h_da
HOCHSCHULE DARMSTADT
UNIVERSITY OF APPLIED SCIENCES

Hochschule Darmstadt

– Fachbereich Informatik –

Performanceoptimierung eines Incident-Response-Scanners durch taskbasiertes Multithreading-Scheduling

Abschlussarbeit zur Erlangung des akademischen Grades

Bachelor of Science (B. Sc.)

vorgelegt von

Pierre Baron

Matrikelnummer: 751992

Referent	:	Prof. Dr. Lars-Olof Burchard
Korreferent	:	Prof. Dr. Ronald Charles Moore

Pierre Baron: *Performanceoptimierung eines Incident-Response-Scanners durch taskbasiertes Multithreading-Scheduling*, © 22. April 2026

ERKLÄRUNG

Ich versichere hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die im Literaturverzeichnis angegebenen Quellen benutzt habe.

Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder noch nicht veröffentlichten Quellen entnommen sind, sind als solche kenntlich gemacht.

Die Zeichnungen oder Abbildungen in dieser Arbeit sind von mir selbst erstellt worden oder mit einem entsprechenden Quellennachweis versehen.

Diese Arbeit ist in gleicher oder ähnlicher Form noch bei keiner anderen Prüfungsbehörde eingereicht worden.

Darmstadt, 22. April 2026

Pierre Baron

ABSTRACT

APT scanners must be able to analyze compromised or suspicious systems as quickly as possible. In day-to-day use outside acute security incidents, they should combine short scan times with low resource utilization.

This thesis investigates performance optimization of an APT scanner by replacing object-based, recursively synchronous scheduling with a centralized task-based scheduling model.

Three scheduling algorithms were implemented and evaluated: First-In, First-Out (FIFO), Shortest-Subtree-Time-First (SSTF), and Work-Stealing with SSTF prioritization. The evaluation was conducted experimentally in a controlled test environment on Debian 13 and Windows 11 with one, four, and ten threads and 20 repetitions per configuration. The measured metrics were runtime, CPU and RAM utilization, waiting time, burst time, and the share of synchronously executed tasks.

On Windows, the new schedulers show clear runtime improvements compared with the reference at four and ten threads, ranging from 5.69 % to 8.39 % at four threads and from 8.89 % to 10.38 % at ten threads, depending on the method. Under Linux, no consistent runtime advantage can be demonstrated. SSTF provides the most stable overall balance of runtime, waiting time, and memory behavior.

The results show that the architectural change is feasible and enables reproducible performance gains, especially in scenarios under heavy system load.

ZUSAMMENFASSUNG

APT-Scanner müssen kompromittierte oder verdächtige Systeme möglichst schnell analysieren können. Im täglichen Einsatz außerhalb akuter Sicherheitsvorfälle sollen sie neben einer kurzen Scanzeit auch eine geringe Ressourcenauslastung gewährleisten.

Diese Arbeit untersucht die Performance-Optimierung eines APT-Scanners durch die Umstellung von einem objektbasierten, rekursiv-synchronen Scheduling auf ein zentrales, taskbasiertes Scheduling.

Implementiert und evaluiert wurden drei Scheduling-Algorithmen: First-In, First-Out (FIFO), Shortest-Subtree-Time-First (SSTF) und Work-Stealing mit SSTF-Priorisierung. Die Bewertung erfolgte experimentell in einer kontrollierten Testumgebung auf Debian 13 und Windows 11 mit einem, vier und zehn Threads sowie 20 Wiederholungen je Konfiguration. Gemessen wurden Laufzeit, CPU- und RAM-Utilization, Waiting Time, Burst Time und der Anteil synchron ausgeführter Tasks.

Unter Windows zeigen die neuen Scheduler bei vier und zehn Threads klare Laufzeitgewinne gegenüber der Referenz. Bei vier Threads betragen diese Laufzeitgewinne je nach Verfahren 5,69 % bis 8,39 % und bei zehn Threads 8,89 % bis 10,38 %. Unter Linux lässt sich dagegen kein konsistenter Laufzeitvorteil nachweisen. SSTF liefert die stabilste Gesamtbalance aus Laufzeit, Waiting Time und Speichereigenschaften.

Die Ergebnisse belegen, dass der Wechsel zu einem zentralen, taskbasierten Scheduling möglich ist. Reproduzierbare Leistungsgewinne, besonders unter hohen Lastprofilen, werden ermöglicht.

INHALTSVERZEICHNIS

I Thesis

1 Einleitung 2

2 Grundlagen 4

2.1 Performance 4

2.2 Arbeitsweise des Scanners 5

2.3 Stand der Technik: Scheduling-Verfahren 7

2.3.1 Work-Sharing und Work-Stealing 7

2.3.2 First-In, First-Out 8

2.3.3 Shortest-Job-First 10

2.4 Motivation und Forschungsziel 11

3 Methodik 12

3.1 Überblick über den Untersuchungsablauf 12

3.2 Metriken und Messungen 13

3.2.1 Scanner-interne Metriken 13

3.2.2 CPU-Utilization 14

3.2.3 RAM-Utilization 14

3.2.4 Parameter der Messungen 16

3.3 Die Testumgebung 16

3.3.1 Speicher-Optimierungen 18

3.3.2 Storage- und I/O-Optimierungen 18

3.3.3 CPU-Optimierungen 19

3.3.4 System-Optimierungen 19

3.4 Vorbereitung des Scanners 20

3.4.1 Das Fertigstellungsproblem 21

3.5 Die Auswahl und Implementierung der Scheduling-Algorithmen 22

3.5.1 Formale Notation und Vergleichsrahmen 23

3.5.2 FIFO-Scheduler 24

3.5.3 Shortest-Subtree-Time-First (SSTF) 25

3.5.4 Work-Stealing mit SSTF-Priorisierung 26

3.5.5 Validitätsaspekte und Reproduzierbarkeit 27

4 Ergebnisse und Diskussion 29

4.1 Rekursiv-synchroner, objektbasierter Scheduler (Referenz) . . . 29

4.2 First-In, First-Out 31

4.2.1 Diskussion der FIFO-Ergebnisse 33

4.3 Shortest-Subtree-Time-First 34

4.3.1 Diskussion der SSTF-Ergebnisse 36

4.4 Work-Stealing mit SSTF-Priorisierung 37

4.4.1 Diskussion der Work-Stealing-Ergebnisse 38

4.5 Diskussion der Ergebnisse 39

5 Fazit 47

6 Ausblick 49

II Appendix
A Anhang 51
A.1 Referenzmessung (Alter Scheduler) 51
A.1.1 Ein Thread 51
A.1.2 Vier Threads 53
A.1.3 Zehn Threads 56
A.1.4 Vergleich 58
A.2 FIFO_50 60
A.2.1 Ein Thread 60
A.2.2 Vier Threads 62
A.2.3 Zehn Threads 65
A.2.4 Vergleich 67
A.3 FIFO_500 69
A.3.1 Ein Thread 69
A.3.2 Vier Threads 71
A.3.3 Zehn Threads 74
A.3.4 Vergleich 76
A.4 Shortest-Subtree-Time-First (SSTF) 78
A.4.1 Ein Thread 78
A.4.2 Vier Threads 80
A.4.3 Zehn Threads 83
A.4.4 Vergleich 85
A.5 Work-Stealing mit SSTF-Priorisierung 87
A.5.1 Ein Thread 87
A.5.2 Vier Threads 89
A.5.3 Zehn Threads 92
A.5.4 Vergleich 94

Literatur 96

ABBILDUNGSVERZEICHNIS

Abbildung 2.1 Arbeitsablauf des Scanners 6
Abbildung 3.1 Arbeitsablauf mit entkoppeltem Scheduler 21
Abbildung 4.1 Referenz CPU, 1 Thread 30
Abbildung 4.2 Referenz CPU, 10 Threads 31
Abbildung 4.3 Scheduler-Vergleich Laufzeit, 1 Thread 40
Abbildung 4.4 Scheduler-Vergleich CPU, 1 Thread 41
Abbildung 4.5 Scheduler-Vergleich RAM, 1 Thread 41
Abbildung 4.6 Scheduler-Vergleich Laufzeit, 4 Threads 42
Abbildung 4.7 Scheduler-Vergleich CPU, 4 Threads 43
Abbildung 4.8 Scheduler-Vergleich RAM, 4 Threads 43
Abbildung 4.9 Scheduler-Vergleich Laufzeit, 10 Threads 44
Abbildung 4.10 Scheduler-Vergleich CPU, 10 Threads 45
Abbildung 4.11 Scheduler-Vergleich RAM, 10 Threads 45
Abbildung A.1 Referenz Laufzeit, 1 Thread 51
Abbildung A.2 Referenz RAM-Zeitverlauf, 1 Thread 51
Abbildung A.3 Referenz RAM, 1 Thread 52
Abbildung A.4 Referenz CPU-Zeitverlauf, 1 Thread 52
Abbildung A.5 Referenz CPU, 1 Thread 53
Abbildung A.6 Referenz Laufzeit, 4 Threads 53
Abbildung A.7 Referenz RAM-Zeitverlauf, 4 Threads 54
Abbildung A.8 Referenz RAM, 4 Threads 54
Abbildung A.9 Referenz CPU-Zeitverlauf, 4 Threads 55
Abbildung A.10 Referenz CPU, 4 Threads 55
Abbildung A.11 Referenz Laufzeit, 10 Threads 56
Abbildung A.12 Referenz RAM-Zeitverlauf, 10 Threads 56
Abbildung A.13 Referenz RAM, 10 Threads 57
Abbildung A.14 Referenz CPU-Zeitverlauf, 10 Threads 57
Abbildung A.15 Referenz CPU, 10 Threads 58
Abbildung A.16 Referenz CPU-Vergleich 58
Abbildung A.17 Referenz RAM-Vergleich 59
Abbildung A.18 Referenz Laufzeit-Vergleich 59
Abbildung A.19 FIFO_50 Laufzeit, 1 Thread 60
Abbildung A.20 FIFO_50 CPU, 1 Thread 60
Abbildung A.21 FIFO_50 RAM, 1 Thread 61
Abbildung A.22 FIFO_50 CPU-Zeitverlauf, 1 Thread 61
Abbildung A.23 FIFO_50 RAM-Zeitverlauf, 1 Thread 62
Abbildung A.24 FIFO_50 Laufzeit, 4 Threads 62
Abbildung A.25 FIFO_50 CPU, 4 Threads 63
Abbildung A.26 FIFO_50 RAM, 4 Threads 63
Abbildung A.27 FIFO_50 CPU-Zeitverlauf, 4 Threads 64
Abbildung A.28 FIFO_50 RAM-Zeitverlauf, 4 Threads 64

Abbildung A.29 FIFO_50 Laufzeit, 10 Threads 65
Abbildung A.30 FIFO_50 CPU, 10 Threads 65
Abbildung A.31 FIFO_50 RAM, 10 Threads 66
Abbildung A.32 FIFO_50 CPU-Zeitverlauf, 10 Threads 66
Abbildung A.33 FIFO_50 RAM-Zeitverlauf, 10 Threads 67
Abbildung A.34 FIFO_50 Laufzeit-Vergleich 67
Abbildung A.35 FIFO_50 CPU-Vergleich 68
Abbildung A.36 FIFO_50 RAM-Vergleich 68
Abbildung A.37 FIFO_500 Laufzeit, 1 Thread 69
Abbildung A.38 FIFO_500 CPU, 1 Thread 69
Abbildung A.39 FIFO_500 RAM, 1 Thread 70
Abbildung A.40 FIFO_500 CPU-Zeitverlauf, 1 Thread 70
Abbildung A.41 FIFO_500 RAM-Zeitverlauf, 1 Thread 71
Abbildung A.42 FIFO_500 Laufzeit, 4 Threads 71
Abbildung A.43 FIFO_500 CPU, 4 Threads 72
Abbildung A.44 FIFO_500 RAM, 4 Threads 72
Abbildung A.45 FIFO_500 CPU-Zeitverlauf, 4 Threads 73
Abbildung A.46 FIFO_500 RAM-Zeitverlauf, 4 Threads 73
Abbildung A.47 FIFO_500 Laufzeit, 10 Threads 74
Abbildung A.48 FIFO_500 CPU, 10 Threads 74
Abbildung A.49 FIFO_500 RAM, 10 Threads 75
Abbildung A.50 FIFO_500 CPU-Zeitverlauf, 10 Threads 75
Abbildung A.51 FIFO_500 RAM-Zeitverlauf, 10 Threads 76
Abbildung A.52 FIFO_500 Laufzeit-Vergleich 76
Abbildung A.53 FIFO_500 CPU-Vergleich 77
Abbildung A.54 FIFO_500 RAM-Vergleich 77
Abbildung A.55 SSTF Laufzeit, 1 Thread 78
Abbildung A.56 SSTF CPU, 1 Thread 78
Abbildung A.57 SSTF RAM, 1 Thread 79
Abbildung A.58 SSTF CPU-Zeitverlauf, 1 Thread 79
Abbildung A.59 SSTF RAM-Zeitverlauf, 1 Thread 80
Abbildung A.60 SSTF Laufzeit, 4 Threads 80
Abbildung A.61 SSTF CPU, 4 Threads 81
Abbildung A.62 SSTF RAM, 4 Threads 81
Abbildung A.63 SSTF CPU-Zeitverlauf, 4 Threads 82
Abbildung A.64 SSTF RAM-Zeitverlauf, 4 Threads 82
Abbildung A.65 SSTF Laufzeit, 10 Threads 83
Abbildung A.66 SSTF CPU, 10 Threads 83
Abbildung A.67 SSTF RAM, 10 Threads 84
Abbildung A.68 SSTF CPU-Zeitverlauf, 10 Threads 84
Abbildung A.69 SSTF RAM-Zeitverlauf, 10 Threads 85
Abbildung A.70 SSTF Laufzeit-Vergleich 85
Abbildung A.71 SSTF CPU-Vergleich 86
Abbildung A.72 SSTF RAM-Vergleich 86
Abbildung A.73 Work-Stealing Laufzeit, 1 Thread 87
Abbildung A.74 Work-Stealing CPU, 1 Thread 87

Abbildung A.75 Work-Stealing RAM, 1 Thread 88
Abbildung A.76 Work-Stealing CPU-Zeitverlauf, 1 Thread 88
Abbildung A.77 Work-Stealing RAM-Zeitverlauf, 1 Thread 89
Abbildung A.78 Work-Stealing Laufzeit, 4 Threads 89
Abbildung A.79 Work-Stealing CPU, 4 Threads 90
Abbildung A.80 Work-Stealing RAM, 4 Threads 90
Abbildung A.81 Work-Stealing CPU-Zeitverlauf, 4 Threads 91
Abbildung A.82 Work-Stealing RAM-Zeitverlauf, 4 Threads 91
Abbildung A.83 Work-Stealing Laufzeit, 10 Threads 92
Abbildung A.84 Work-Stealing CPU, 10 Threads 92
Abbildung A.85 Work-Stealing RAM, 10 Threads 93
Abbildung A.86 Work-Stealing CPU-Zeitverlauf, 10 Threads 93
Abbildung A.87 Work-Stealing RAM-Zeitverlauf, 10 Threads 94
Abbildung A.88 Work-Stealing Laufzeit-Vergleich 94
Abbildung A.89 Work-Stealing CPU-Vergleich 95
Abbildung A.90 Work-Stealing RAM-Vergleich 95

TABELLENVERZEICHNIS

Tabelle 3.1 Erhobene RAM-Metriken und ihre Datenquellen pro Betriebssystem 15
Tabelle 3.2 Konfiguration des Testsystems 17
Tabelle 4.1 Übersicht der Laufzeit-, CPU-, RAM- und Runtime-CV-Metriken des alten Schedulers für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte $\pm$ Standardabweichung aus $n = 20$ Läufen 30
Tabelle 4.2 Laufzeit-, CPU- und RAM-Metriken des FIFO_50-Schedulers für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte $\pm$ Standardabweichung aus $n = 20$ Läufen 31
Tabelle 4.3 Laufzeit-, CPU- und RAM-Metriken des FIFO_500-Schedulers für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte $\pm$ Standardabweichung aus $n = 20$ Läufen 32
Tabelle 4.4 Runtime CV sowie Queue Time, Execution Time und Anteil synchroner Tasks des FIFO_50-Schedulers für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte $\pm$ Standardabweichung aus $n = 20$ Läufen 32
Tabelle 4.5 Runtime CV sowie Queue Time, Execution Time und Anteil synchroner Tasks des FIFO_500-Schedulers für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte $\pm$ Standardabweichung aus $n = 20$ Läufen 33
Tabelle 4.6 Laufzeit-, CPU- und RAM-Metriken des SSTF-Schedulers für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte $\pm$ Standardabweichung aus $n = 20$ Läufen 35
Tabelle 4.7 Runtime CV sowie Queue Time, Execution Time und Anteil synchroner Tasks des SSTF-Schedulers für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte $\pm$ Standardabweichung aus $n = 20$ Läufen 35
Tabelle 4.8 Laufzeit-, CPU- und RAM-Metriken des Work-Stealing-Schedulers mit SSTF-Priorisierung für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte $\pm$ Standardabweichung aus $n = 20$ Läufen 37

Tabelle 4.9 Runtime CV sowie Queue Time, Execution Time und Anteil synchroner Tasks des Work-Stealing-Schedulers mit SSTF-Priorisierung für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte ± Standardabweichung aus $n = 20$ Läufen 38

ABKÜRZUNGSVERZEICHNIS

APT	Advanced Persistent Threat
ASLR	Address Space Layout Randomization
CPU	Central Processing Unit
CSV	Comma-Separated Values
CV	Coefficient of Variation
FCFS	First-Come, First-Served
FIFO	First-In, First-Out
GPU	Graphics Processing Unit
I/O	Input/Output
KVM	Kernel-based Virtual Machine
LPDDR5	Low Power Double Data Rate 5
NVMe	Non-Volatile Memory Express
OOM	Out-of-Memory
OS	Operating System
QEMU	Quick Emulator
RAM	Random Access Memory
SJF	Shortest Job First
SSH	Secure Shell
SSD	Solid-State Drive
SSTF	Shortest-Subtree-Time-First
TLB	Translation Lookaside Buffer
VM	Virtuelle Maschine
vCPU	virtuelle CPU
WS	Work-Stealing

Teil I

THESIS

1 EINLEITUNG

Ein Incident-Response-Scanner ist eine Sicherheitssoftware, die auf die schnelle Analyse und Bewertung kompromittierter oder verdächtiger Systeme ausgelegt ist. Sie wird insbesondere dann eingesetzt, wenn nach einem Vorfall möglichst zügig belastbare Informationen über den Zustand eines Systems benötigt werden. Im Unterschied zu klassischen Schutzmechanismen steht dabei nicht nur die Prävention, sondern auch die forensisch verwertbare Erkennung und Einordnung von Auffälligkeiten im Vordergrund.

Ein zentraler Anwendungsfall solcher Systeme ist die Erkennung von Advanced Persistent Threats (APTs). Der Begriff APT bezeichnet eine Form des Cyberangriffs, bei der sich ein Angreifer unbefugten Zugang zu einem Computersystem verschafft und potenziell über einen langen Zeitraum unentdeckt darin verbleibt. Im Gegensatz zu herkömmlichen Angriffen handelt es sich bei APTs um menschliche Akteure mit hoher Motivation und hohen Ressourcen, nicht um einfachen, automatisierten Schadcode [1, S. 1–2, 7].

Ein Advanced-Persistent-Threat-Scanner (APT-Scanner), oder präziser ein APT-Angriffserkennungssystem, ist eine spezialisierte Sicherheitssoftware, die auf die Identifikation der Aktivitäten fortgeschrittener und andauernder Bedrohungen ausgelegt ist [1, S. 1–2].
Während herkömmliche Virenscanner oft auf bekannten Signaturen basieren, müssen APT-Scanner deutlich komplexere Methoden anwenden, um die typischerweise subtilen und über lange Zeiträume gestreckten Angriffe zu finden.

Da APT-Akteure häufig unbekannte Schwachstellen (Zero-Day-Exploits) nutzen oder legitime Systemwerkzeuge missbrauchen (Living-off-the-Land), helfen signaturbasierte Detektionsmechanismen oft nicht weiter [1, S. 6–7]. Moderne APT-Angriffserkennungssysteme nutzen daher Anomalieerkennung [1, S. 8–9]. Die Anomalieerkennung identifiziert ungewöhnliche Muster oder Instanzen in Daten, die nicht dem erwarteten Verhalten entsprechen [1, S. 8–9, 24]. Sie unterscheidet im Gegensatz zu herkömmlichen Virenscannern nicht zwischen bekannten und unbekannten Angriffen. Da die Anomalieerkennung auf dem Systemverhalten basiert, um Abweichungen zu erkennen, sind APT-Angriffserkennungssysteme mit einem hohen Wartungsaufwand verbunden. Das APT-Angriffserkennungssystem lernt zunächst das "normale" Verhalten des Rechners. Jede erhebliche Abweichung wird als potenzieller Angriff gemeldet. Analysten bewerten die potenziellen Angriffe und nehmen die finale Unterscheidung zwischen True- und

False-Positive vor, welche ab diesem Zeitpunkt automatisch von dem APT-Angriffserkennungssystem als solche eingeordnet werden.

APT-Angriffserkennungssysteme werden auf einer hohen Anzahl von produktiven Systemen, häufig auf kritischen Servern oder in ressourcenbeschränkten Umgebungen, eingesetzt. Hierbei werden regelmäßig Scans ausgeführt. Auch bei Incident-Response-Szenarien kommen APT-Angriffserkennungssysteme zum Einsatz. Die Performance des APT-Angriffserkennungssystems ist von entscheidender Bedeutung. Lange Scan-Zeiten und hohe Ressourcenauslastung stellen ein relevantes Problem dar. Das Risiko von Performanceeinbußen während des Scans auf produktiven Systemen muss möglichst gering gehalten werden. Gleichzeitig muss ein Scan auf einem potenziell befallenen System eine niedrige Scan-Zeit aufweisen und die zur Verfügung stehenden Ressourcen möglichst weitreichend nutzen. Um diese Voraussetzungen an die Performance zu erfüllen, ist eine effiziente Parallelisierung durch einen Scheduler von hoher Relevanz.

In dieser Arbeit wird der Versuch vorgenommen, die Performance des APT-Scanners Thor der Firma Nextron Systems GmbH, im Folgenden als Scanner bezeichnet, zu optimieren, indem ein taskbasierter Scheduler implementiert wird.

Daraus ergibt sich die Forschungsfrage, ob ein zentraler, taskbasierter Scheduler die Laufzeit des Scanners gegenüber dem bisherigen objektbasierten, rekursiv-synchronen Scheduling verbessern kann, ohne CPU- und RAM-Auslastung sowie Wartezeiten unvertretbar zu verschlechtern. Die durchgeführten Untersuchungen zeigen, dass unter Windows bei paralleler Ausführung ab vier Threads reproduzierbare Laufzeitvorteile erreichbar sind. Im Gegensatz dazu ist unter Linux kein konsistenter Laufzeitvorteil nachweisbar. Unter den betrachteten Verfahren zeigt Shortest-Subtree-Time-First (SSTF), insbesondere in den Mehrthread-Konfigurationen, die ausgewogenste Kombination aus Laufzeit, Wartezeit und Speichereigenschaften.

Der weitere Aufbau der Arbeit folgt dieser Fragestellung: Zunächst werden die fachlichen und technischen Grundlagen sowie die Arbeitsweise des Scanners erläutert, anschließend die Methodik und der Versuchsaufbau dargestellt. Darauf aufbauend werden die Messergebnisse präsentiert und diskutiert, bevor die Arbeit mit einer Zusammenfassung und einem Ausblick abschließt.

2 GRUNDLAGEN

In diesem Kapitel wird beschrieben, was Performance für den Anwendungszweck des Scanners bedeutet. Um ein besseres Verständnis für die Arbeitsweise des Scanners zu schaffen, wird diese anschließend näher erläutert.

Dieses Kapitel ist in folgende Themen unterteilt:

- Die für diese Arbeit relevanten Performance-Metriken und ihre Bedeutung für den Scanner.
- Die Arbeitsweise des Scanners und das bisherige objektbasierte Scheduling-Modell.
- Der Stand der Technik zu den für diese Arbeit relevanten Scheduling-Verfahren und Lastverteilungsstrategien.
- Die Motivation und das Forschungsziel der Arbeit als Grundlage für die Methodik in Kapitel 3.

2.1 PERFORMANCE

Wichtige Metriken für die Bewertung von Scheduling-Algorithmen sind [2, S. 204–205], [3, S. 154]:

CPU-UTILIZATION: Die CPU-Utilization beschreibt die Auslastung der CPU, gemessen als Prozentsatz der Zeit, in der die CPU aktiv arbeitet. Bei mehreren Kernen wird die durchschnittliche Auslastung über alle Kerne betrachtet. Das Ziel ist es, den Prozessor so beschäftigt wie möglich zu halten. In realen Systemen sollte die Auslastung zwischen 40 % (leicht belastet) und 90 % (stark belastet) liegen.

THROUGHPUT: Der Throughput beschreibt die Anzahl der Tasks, die pro Zeiteinheit abgeschlossen werden.

TURNAROUND TIME: Die Turnaround Time beschreibt die Zeitspanne zwischen dem Einreihen eines Tasks in die Warteschlange und seinem Abschluss.

WAITING TIME: Die Waiting Time beschreibt die Zeitspanne, die ein Task in der Warteschlange verbringt.

RAM-UTILIZATION: Die RAM-Utilization beschreibt die Auslastung des Hauptspeichers in Prozent. Für Ressourcen, die Speicher bereitstellen, bezieht sich die Auslastung auf die verbrauchte Kapazität. [4, S. 22].

RUNTIME: Die Runtime beschreibt die gesamte Ausführungszeit des Prozesses. Hennessy und Patterson [5, S. 39–40] führen aus, dass die Ausführungszeit als das einzig konsistente und zuverlässige Maß für die Leistung gilt.

2.2 ARBEITSWEISE DES SCANNERS

Der Scanner basiert auf zwei zentralen Komponententypen, deren Zusammenwirken den Arbeitsablauf definiert.

MODUL: Module sind autonome Funktionseinheiten, die sequenziell abgearbeitet werden. Ein Modul (z. B. "User Account Check") identifiziert spezifische Systemressourcen, instanziiert diese als Objekte und initiiert deren Untersuchung [6].

FEATURE: Features werden kontextabhängig aktiviert, sobald ein Objekt eines bestimmten Typs untersucht wird. Sie können ihrerseits neue Objekte generieren. Ein "File System Scan"-Modul kann beispielsweise ein Objekt für eine Archivdatei (.zip) erzeugen. Bei dessen Analyse wird das "Archive Scan"-Feature aktiv, welches den Inhalt entpackt und für jede enthaltene Datei ein neues Objekt instanziiert. Dieser Prozess setzt sich rekursiv fort [7].

Der Scanner nutzt ein objektbasiertes, synchrones Multithreading-Scheduling. Jedes Modul kapselt hierfür einen eigenen primitiven Scheduler, bestehend aus einer Warteschlange und einem Threadpool. Während der Systemuntersuchung generiert das Modul initiale Objekte und reiht diese in die Warteschlange ein. Die Worker, also die ausführenden Threads des Threadpools, entnehmen die Objekte und führen die für den jeweiligen Objekttyp registrierten Features aus.

Die Ausführung eines Features kann weitere Objekte (nachfolgend Kindobjekte genannt) hervorbringen. Auch auf diesen Kindobjekten werden typspezifisch Features ausgeführt, was zu einer tief verschachtelten Objekthierarchie führen kann. Ein Worker arbeitet sowohl das Ursprungsobjekt als auch alle daraus resultierenden Nachkommen vollständig synchron ab. Dadurch entstehen Szenarien, in denen einzelne Threads über lange Zeiträume blockiert sind und eine erhebliche Menge an Hauptspeicher beanspruchen. Abbildung 2.1 zeigt den beschriebenen Arbeitsablauf des Scanners.

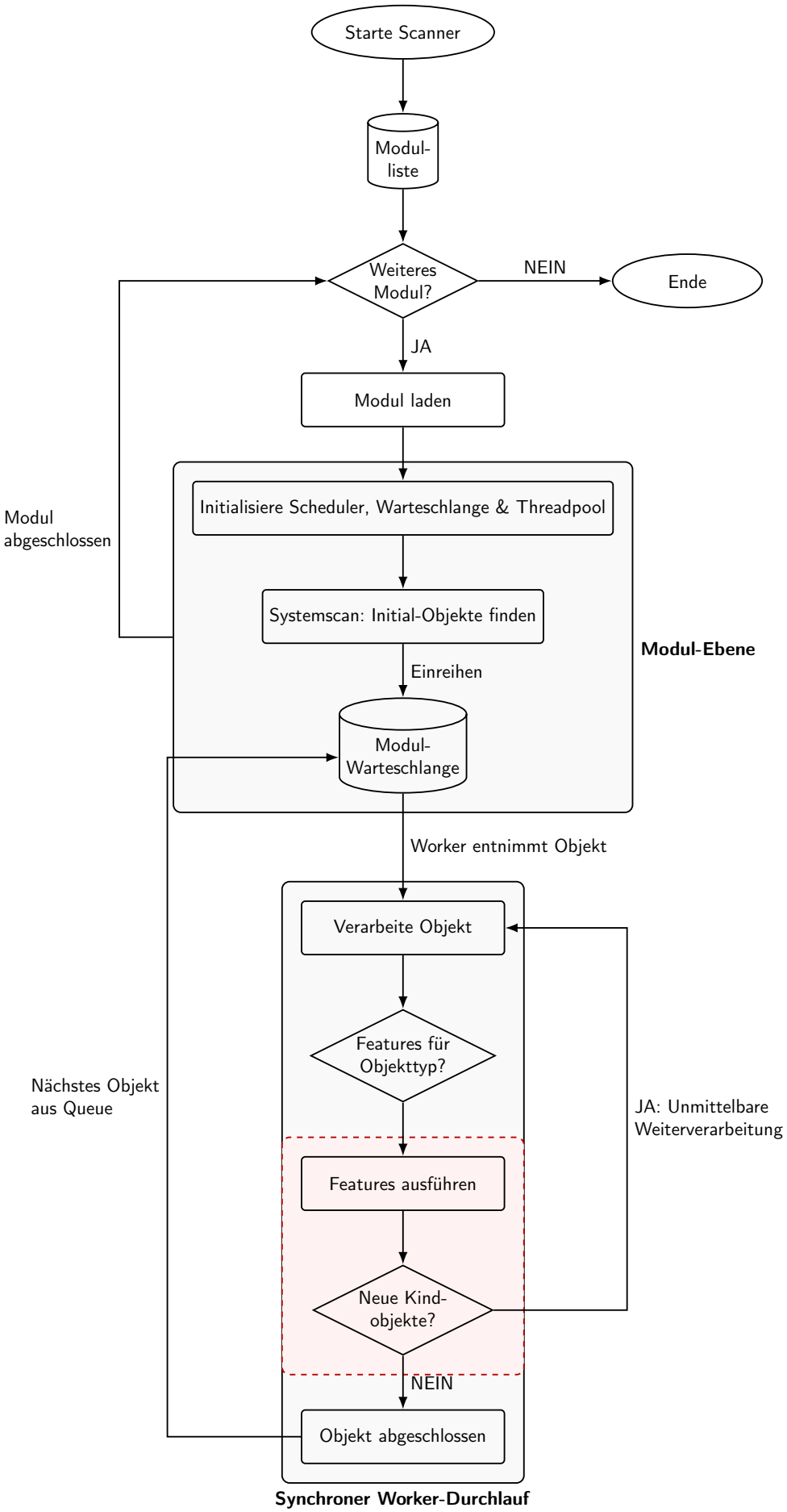

Abbildung 2.1: Arbeitsablauf des Scanners

2.3 STAND DER TECHNIK: SCHEDULING-VERFAHREN

Scheduling-Algorithmen für Multithreading in Prozessen sind seit mehreren Jahrzehnten Gegenstand intensiver Forschung. Bereits das Standardwerk Theory of Scheduling aus dem Jahr 1967 [8] legte zentrale mathematische Grundlagen für das Scheduling in modernen Betriebssystemen. Darin analysieren Conway et al. grundlegende Konzepte wie Prioritätsregeln, Präemption (Unterbrechbarkeit) von Tasks sowie warteschlangentheoretische Modelle im Kontext von Mehrprozessor-Systemen. Auf dieser Basis wurden die Eigenschaften klassischer Scheduling-Strategien wie Round Robin oder Shortest Job First formal beschrieben und hinsichtlich ihrer Leistungsmerkmale analysiert. Die Funktionsweise deterministischer Verfahren sowie grundlegender Priorisierungsschemata gelten daher heute als umfassend untersucht und theoretisch fundiert.

Der Fokus dieses Abschnitts liegt auf den für diese Arbeit relevanten Scheduling-Verfahren, deren Einordnung in den spezifischen Kontext des Scanners, sowie grundlegenden Informationen zu einem Paradigma der Lastverteilung.

2.3.1 *Work-Sharing und Work-Stealing*

Die effiziente Planung dynamisch wachsender, paralleler Berechnungen auf modernen Multiprozessorsystemen erfordert ein Gleichgewicht zwischen einer hohen Prozessor-Auslastung sowie einem geringen Speicherbedarf und Kommunikations-Overhead. In der Literatur zum dynamischen Task-Scheduling haben sich hierfür zwei grundlegende Lastverteilungsstrategien etabliert: Work-Sharing, bei dem Arbeit aktiv auf Worker verteilt wird, und Work-Stealing, bei dem inaktive Worker Aufgaben von anderen Workern übernehmen.

Die Erläuterungen in diesem Abschnitt basieren auf der Arbeit Scheduling Multithreaded Computations by Work Stealing von Blumofe und Leiserson [9, S. 721–746]. Diese untersucht das Scheduling von Threads in parallelen Programmen und ist auf den Kontext des in dieser Arbeit betrachteten Scanners übertragbar.

Beim Work-Sharing versucht der Scheduler aktiv, neu erzeugte Tasks auf andere, potenziell unterausgelastete Worker zu verteilen. Im Gegensatz dazu geht beim Work-Stealing die Initiative von den leerlaufenden Workern aus: Ein Worker, der keine Arbeit mehr besitzt, versucht Aufgaben aus der Warteschlange eines anderen Workers zu übernehmen. Dieser Ansatz gilt als kommunikationseffizienter, da Migrationen von Tasks nur auftreten, wenn ein Worker tatsächlich untätig ist.

Der Work-Stealing-Ansatz wurde in den 1980er Jahren als Scheduling-Modell eingeführt. 1999 erfolgte durch Blumofe und Leiserson eine formale Analyse mit beweisbar optimalen Schranken für Ausführungszeit, Speicherbedarf und Kommunikationsaufwand. Der beschriebene Algorithmus basiert auf einer dezentralen Datenstruktur, der Ready Deque. Hierbei verwaltet jeder Worker seine Tasks in einer Double-Ended Queue (Deque), auf die über zwei Enden, top und bottom, zugegriffen wird.

Der jeweilige Worker arbeitet sein Deque nach einem Stack-Prinzip ab: Neue Kind-Tasks werden am unteren Ende eingefügt und auch von dort wieder entnommen. Dieses Vorgehen begünstigt die Datenlokalität.
Ein untätiger Worker hingegen stiehlt Aufgaben vom oberen Ende der Deque eines zufällig ausgewählten Workers. Dadurch werden Konflikte beim Zugriff auf die Datenstruktur reduziert und gleichzeitig die Datenlokalität begünstigt.

Die Klasse der sogenannten fully strict computations beschreibt Berechnungsstrukturen, bei denen alle Abhängigkeiten eines Threads ausschließlich zu seinem direkten Elternthread führen. An diesen konnten starke Effizienzgarantien gezeigt werden. Für die Ausführungszeit T_P auf P Workern gilt

$$T_P = T_1/P + O(T_\infty), \tag{2.1}$$

wobei T_1 die gesamte serielle Ausführungszeit (Work) und T_∞ die kritische Pfadlänge der Berechnung (Critical Path Length) beschreibt.

Darüber hinaus weist der Algorithmus beweisbar effiziente Eigenschaften bezüglich des Speicherbedarfs auf. Der benötigte Speicherbedarf ist durch $S_1 \cdot P$ nach oben beschränkt, wobei S_1 den minimalen seriellen Speicherbedarf bezeichnet. Diese Schranke ist theoretisch optimal (bis auf einen konstanten Faktor) und garantiert eine lediglich lineare Expansion des Speichers im Verhältnis zur Anzahl der eingesetzten Worker P. Dies verhindert ein unkontrolliertes Wachstum der Speicherressourcen. Dadurch wird sichergestellt, dass der Bedarf nicht, wie bei naiven Scheduling-Strategien oder Work-Sharing-Ansätzen möglich, unkontrolliert ansteigt.

2.3.2 *First-In, First-Out*

In diesem Abschnitt wird auf die Erläuterungen von Hansen in Operating System Principles von 1973 [10] Bezug genommen.

Der First-In, First-Out (FIFO)-Scheduling-Algorithmus, auch als First-Come, First-Served (FCFS) bezeichnet, stellt einen nicht-präemptiven Scheduling-

Ansatz dar. Sein grundlegendes Prinzip besteht in der strikt sequenziellen Abarbeitung von Tasks entsprechend ihrer Ankunftsreihenfolge. Ein Task wird genau dann zur Ausführung ausgewählt, wenn alle zuvor eingetroffenen Tasks vollständig abgearbeitet wurden.

Die Implementierung erfolgt typischerweise über eine FIFO-Warteschlange, in welche neu eintreffende Tasks am Ende eingefügt werden. Der Scheduler selektiert deterministisch stets das am Kopf der Warteschlange befindliche Element. Aufgrund des nicht-präemptiven Charakters wird ein Task nach seiner Disposition ohne Unterbrechung ausgeführt, bis entweder seine Ausführung abgeschlossen ist oder eine externe Terminierung erfolgt.

Das Verhalten des FIFO-Schedulers lässt sich im Rahmen der Warteschlangentheorie analysieren. Für die mittlere Wartezeit W ergibt sich dabei folgender Zusammenhang:

$$W = \frac{W_0}{1 - \rho} \tag{2.2}$$

wobei W_0 eine systemspezifische Konstante darstellt und ρ die Systemauslastung beschreibt. Diese Beziehung verdeutlicht, dass die mittlere Wartezeit für $\rho \to 1$ nichtlinear gegen unendlich wächst.

Ein grundlegender Zusammenhang zwischen der mittleren Anzahl von Prozessen im System n, der Ankunftsrate λ und der mittleren Verweildauer W wird zudem durch das Gesetz von Little beschrieben:

$$n = \lambda \cdot W \tag{2.3}$$

Darüber hinaus lässt sich die erwartete Anzahl von Prozessen in der Warteschlange L wie folgt ausdrücken:

$$L = \frac{\rho^2}{1 - \rho} \tag{2.4}$$

Auch hier zeigt sich, dass für hohe Auslastungen ein starkes Anwachsen der Warteschlangenlänge zu erwarten ist.

Trotz seiner konzeptionellen Einfachheit und deterministischen Ausführungsreihenfolge weist FIFO signifikante Nachteile hinsichtlich der Wartezeiten auf. Insbesondere kann es zum sogenannten Convoy-Effekt kommen, bei dem lange laufende Tasks nachfolgende kurze Tasks blockieren. Dieser Effekt führt zu erhöhten mittleren Wartezeiten und hoher Varianz. Im Vergleich zu optimalen Scheduling-Strategien wie Shortest Job First ist FIFO da-

her hinsichtlich der Minimierung der durchschnittlichen Wartezeit suboptimal.

Ein Vorteil des Verfahrens liegt in seiner einfachen Implementierbarkeit sowie in der strikten Einhaltung der Ankunftsreihenfolge, was eine formale Form der Fairness gewährleistet. Diese Fairness geht jedoch nicht mit einer optimalen Ressourcenausnutzung einher.

2.3.3 *Shortest-Job-First*

In diesem Abschnitt wird auf die Erläuterungen von Abraham Silberschatz et al. in Operating System Concepts [2] Bezug genommen.

Der Shortest-Job-First (SJF)-Scheduler ist ein Scheduling-Algorithmus, der Tasks basierend auf der Länge ihrer Burst Time, also ihrer erwarteten Rechenzeit, priorisiert. Jedem Task wird die Länge seiner Burst Time zugeordnet. Sobald ein Worker frei ist, weist der Scheduler ihm den Task zu, welcher die kürzeste Burst-Time benötigt. Falls zwei Tasks identische Burst Time haben, wird in der Regel das First-Come, First-Served (FCFS) Verfahren als Entscheidungshilfe genutzt.

SJF ist unter der Annahme bekannter Burst-Zeiten beweisbar optimal, da es für eine gegebene Menge von Tasks die minimale durchschnittliche Wartezeit liefert [2]. Indem kurze Tasks vor langen ausgeführt werden, verringert sich die Wartezeit der kurzen Tasks stärker, als sich die der langen Aufgaben erhöht.

Der Algorithmus kann in zwei Formen implementiert werden:

1. Nicht-präemptiv (Nonpreemptive): Ein laufender Task bleibt auf dem Worker, bis er freiwillig beendet oder blockiert.
2. Präemptiv (Preemptive): Wenn ein neuer Task in die Warteschlange eintritt, dessen nächster Burst kürzer ist als die verbleibende Restzeit des aktuell laufenden Tasks, wird dieser unterbrochen.

Hierzu können je nach verfügbaren Daten verschiedene Ansätze gewählt werden. Da die exakte Dauer der nächsten Burst Time im Voraus nicht bekannt ist, wird SJF in der Praxis oft durch Vorhersagen approximiert. Hierzu wird ein exponentieller Durchschnitt der vergangenen Bursts verwendet, um den nächsten Wert zu schätzen. Häufig erfolgt die Berechnung nach der Formel:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n \tag{2.5}$$

Dabei stellt t_n die Länge des jüngsten Bursts dar, während τ_{n+1} den vorhergesagten Wert für den nächsten Burst repräsentiert. Der Parameter α (typischerweise $\frac{1}{2}$) steuert das relative Gewicht der jüngsten Vergangenheit gegenüber der historischen Schätzung.

2.4 MOTIVATION UND FORSCHUNGSZIEL

Die in diesem Kapitel dargestellten Scheduling-Verfahren verdeutlichen, dass zwischen Fairness, geringer Wartezeit, hoher Auslastung und moderatem Speicherbedarf Zielkonflikte bestehen. Für den in dieser Arbeit betrachteten Scanner ist dieser Zielkonflikt besonders relevant, da bei der rekursiven Verarbeitung von Kindobjekten stark unterschiedliche Task-Laufzeiten auftreten. Dadurch können Lastungleichgewichte entstehen, die sowohl die Gesamtlaufzeit als auch die Ressourceneffizienz negativ beeinflussen.

Vor diesem Hintergrund besteht die Motivation dieser Arbeit darin, das bestehende Scheduling-Konzept des Scanners systematisch zu überarbeiten und dessen Performance messbar zu verbessern. Im Zentrum steht die Entwicklung und Evaluation eines taskbasierten Schedulers, der eine effizientere Verteilung der anfallenden Arbeit ermöglicht. Die wissenschaftliche Zielsetzung ist es, belastbar zu bestimmen, welche Scheduling-Strategie unter den gegebenen Randbedingungen die beste Gesamtwirkung auf Laufzeit- und Ressourceneigenschaften erzielt.

Daraus leiten sich folgende Forschungsfragen ab:

1. In welchem Ausmaß lässt sich die Laufzeit des Scanners durch alternative Scheduling-Strategien verbessern?
2. Wie wirken sich die verschiedenen Strategien auf CPU- und RAM-Utilization sowie auf die Waiting Time aus?

Die Beantwortung dieser Fragen bildet die Grundlage für die methodische Umsetzung in Kapitel 3, in dem die Implementierung der Scheduler-Varianten und deren experimentelle Evaluation beschrieben werden.

3 METHODIK

Um die Performance der verschiedenen Scheduling-Algorithmen objektiv bewerten und vergleichen zu können, folgt diese Arbeit einem quantitativ-experimentellen Design, das auf kontrolliertem Benchmarking basiert. Ziel ist es, durch eine möglichst deterministische Testumgebung die kausalen Auswirkungen der Algorithmen auf die Systemleistung zu isolieren und statistisch abzusichern.

3.1 ÜBERBLICK ÜBER DEN UNTERSUCHUNGSABLAUF

Der methodische Prozess gliederte sich in vier zentrale Phasen:

1. **Softwareseitige Vorbereitung:** Der Scanner wurde für die nebenläufige Verarbeitung optimiert, um als konsistente Testbasis zu dienen.
2. **Konfiguration der Testumgebung:** Um eine hohe Validität zu erreichen, wurde ein dediziertes Testsystem unter kontrollierten und weitgehend von externen Störeinflüssen bereinigten Bedingungen konfiguriert.
3. **Implementierung der Testvarianten:** Es wurden verschiedene Iterationen des Scanners entwickelt, die sich primär in der Logik der eingesetzten Scheduling-Algorithmen unterscheiden.
4. **Datenerhebung und Metriken:** Jede Scanner-Variante durchlief eine Messreihe von $n = 20$ Testläufen, um eine statistische Grundlage für die anschließende komparative Analyse zu schaffen.

Dieses Kapitel ist in folgende Themen unterteilt:

- Die Auswahl der Metriken und die Parameter der Messungen.
- Die Vorbereitung und Optimierung des Testsystems für die Benchmarks.
- Die Anpassungen, um dem Scanner die Arbeit mit einem taskbasierten Multithreading-Scheduler zu ermöglichen.
- Die Auswahl und Implementierung der Scheduling-Algorithmen.

Es ist zu beachten, dass die Vorgehensweise spezifisch auf das zu optimierende System anzupassen ist.

3.2 METRIKEN UND MESSUNGEN

Die primären Bewertungskriterien sind die tatsächlich vergangene Zeit in Verbindung mit der CPU-Utilization, da sie die Gesamtperformance des Scanners am unmittelbarsten abbilden. Der Throughput dient dabei als Kontrollwert.

Die Waiting Time und die RAM-Utilization haben sekundäre Priorität. Die Waiting Time zeigt direkt, wie effizient der Scheduler die Tasks priorisiert und abarbeitet, während die RAM-Utilization Aufschluss über die Speichereffizienz gibt. Insbesondere sollen RAM-Spitzenlasten vermieden werden, da das Auslagern von Speicher auf die Festplatte die Performance drastisch verschlechtern kann. Darüber hinaus sind die Auswirkungen von Anpassungen der Thread-Anzahl und der Warteschlangen-Größe auf die durchschnittliche RAM-Utilization von Interesse, da sich hieraus möglicherweise weitere Optimierungen ableiten lassen.

Die übrigen Metriken werden ergänzend erhoben, um Unstimmigkeiten und potenzielle Probleme frühzeitig zu identifizieren.

3.2.1 *Scanner-interne Metriken*

Die folgenden Metriken werden automatisiert innerhalb des Scanner-Codes ermittelt und nach jedem Scan-Vorgang persistiert. Hierbei werden jeweils die kumulierten Gesamtwerte sowie die spezifischen Zeitspannen pro Feature erhoben.

THROUGHPUT (DURCHSATZ): Der Throughput quantifiziert die Anzahl der erfolgreich verarbeiteten Tasks pro Zeiteinheit und dient als Maßstab für die Leistungsfähigkeit der jeweiligen Implementierung. Die Erhebung des Durchsatzes ist als relative Vergleichsgröße essentiell, da die absolute Anzahl der zu scannenden Objekte trotz aller Maßnahmen zur Varianzminimierung schwanken kann. Bei einer hohen Abweichung der Anzahl gescannter Objekte pro Feature in verschiedenen Scans müssen Maßnahmen ergriffen werden, um die Varianz zu minimieren.

TURNAROUND TIME (GESAMTDURCHLAUFZEIT PRO TASK): Es werden die durchschnittliche, minimale und maximale Turnaround Time erhoben. Diese Metrik ist von großer Bedeutung, da sie die Effizienz des Gesamtsystems abbildet. Eine Reduktion der Turnaround Time führt zu einer schnelleren Bereitstellung der Ergebnisse und optimiert somit die Antwortzeit gegenüber dem Endnutzer.

BURST TIME (AUSFÜHRUNGSZEIT PRO TASK): Es werden die durchschnittliche, minimale und maximale Burst Time erhoben. Diese Metrik ist

essentiell, um den tatsächlichen Rechenaufwand der implementierten Algorithmen unabhängig von Scheduling-Einflüssen zu bewerten. Darüber hinaus kann sie für die Implementierung von Scheduling-Algorithmen, beispielsweise des Shortest-Job-First-Schedulers, herangezogen werden.

WAITING TIME (WARTEZEIT PRO TASK): Es werden die durchschnittliche, minimale und maximale Waiting Time erhoben. Die Erhebung der Wartezeit ist notwendig, um die Effizienz des Schedulers bei der Task-Priorisierung sowie mögliche Engpässe in der Task-Queue zu identifizieren. Sie ist der primäre Ansatzpunkt für den Scheduler, um die Turnaround Time zu reduzieren und eine schnelle Bereitstellung von Ergebnissen zu ermöglichen.

REAL TIME (GESAMTDURCHLAUFZEIT PRO SCAN): Die Real Time wird bei jedem Scan-Vorgang als kumulative Metrik erfasst. Sie repräsentiert die tatsächlich verstrichene Zeit vom Start bis zum Ende des gesamten Scans. Da diese Kennzahl unmittelbar die Systemperformance aus Sicht des Endanwenders widerspiegelt, stellt sie ein entscheidendes Qualitätsmerkmal für die Praxistauglichkeit des Scanners dar.

3.2.2 *CPU-Utilization*

Die CPU-Utilization wird extern mit einer Abtastrate von 1 Hz erfasst. Die Erhebung dieser Metrik dient dazu, die Effektivität des Schedulers bei der Minimierung von Leerlaufzeiten zu bewerten. Ziel ist die Quantifizierung der Ressourceneffizienz, um nachzuweisen, inwieweit die verfügbare Hardwarekapazität durch die gewählte Scheduling-Strategie optimal ausgelastet wird.

3.2.3 *RAM-Utilization*

Die RAM-Utilization wird extern mit einer Abtastrate von 1 Hz erfasst. Anpassungen der Größe der Queue-Datenstruktur, welche die Tasks für die Worker vorhält, haben direkte Auswirkungen auf die RAM-Utilization. Eine passende Größe für die Queue-Datenstruktur zu wählen, bei welcher die Laufzeit nicht verschlechtert wird, die RAM-Utilization aber möglichst minimal ist, hat eine hohe Priorität, um die Speichereffizienz des Systems zu gewährleisten. Des Weiteren ermöglicht diese Metrik die Identifikation von RAM-Spitzenlasten (Peaks). Deren Vermeidung ist essentiell, da diese im günstigsten Fall die System-Performance beeinträchtigen und im schlechtesten Fall, bei deaktiviertem Swap, den Out-of-Memory-Killer (OOM) auslösen können, was zum Abbruch des Scans oder anderer kritischer Systemprozesse führt.

Da der Memory-Ballooning-Treiber zur Stabilisierung der Messumgebung deaktiviert wurde (vgl. Abschnitt 3.3.1), steht dem Hypervisor keine direkte Einsicht in die Speichernutzung der Virtuellen Maschine (VM) zur Verfügung. Die Datenerhebung erfolgt daher über das Kommando `virsh qemu-agent-command`, welches Befehle innerhalb des isolierten Gast-Betriebssystems ausführt, ohne die VM-Topologie zu beeinflussen.

Auf der Debian-VM wird per SSH ein Messprozess gestartet, der im Sekundentakt die Kernel-Schnittstellen `/proc/$pid/status` und `/proc/meminfo` ausliest und die Ergebnisse lokal in eine CSV-Datei schreibt. Erst nach Abschluss des Benchmarks wird diese Datei einmalig per `scp` zum Host übertragen.

Auf der Windows-VM wird analog eine PowerShell-Schleife via SSH gestartet, die prozessspezifische Metriken über die .NET-Klasse `System.Diagnostics.Process` erhebt. Tabelle 3.1 stellt die erhobenen Metriken und ihre jeweiligen Datenquellen gegenüber.

Tabelle 3.1: Erhobene RAM-Metriken und ihre Datenquellen pro Betriebssystem

Metrik	**Datenquelle**	**Beschreibung**
Debian 13 (Linux-VM)		
Working Set	VmRSS	Aktuell im physischen RAM gehaltener Anteil des Scanner-Prozesses
Private Bytes	VmRSS + VmSwap	Gesamte dem Prozess zugeordnete Speichermenge, inklusive ausgelagerter Seiten
Freier Systemspeicher	MemAvailable	Vom Betriebssystem als verfügbar gemeldeter Arbeitsspeicher
Windows 11 (Windows-VM)		
Working Set	WorkingSet64	Aktuell im physischen RAM gehaltener Anteil des Scanner-Prozesses
Private Bytes	PrivateMemorySize64	Gesamte dem Prozess zugeordnete Speichermenge, inklusive ausgelagerter Seiten
Freier Systemspeicher	FreePhysicalMemory	Vom Betriebssystem als verfügbar gemeldeter Arbeitsspeicher

Da die Abfragen lediglich Kernel-Schnittstellen auslesen, ist die Komplexität der Messoperationen gering. Eine messbare Beeinflussung der Benchmark-Ergebnisse durch den Erhebungsvorgang konnte in Vorab-Testläufen nicht festgestellt werden.

3.2.4 *Parameter der Messungen*

Jede Implementierung eines Scheduling-Algorithmus wird systematisch unter Variation mehrerer Parameter evaluiert. Die Messreihen sind so konzipiert, dass sowohl das Skalierungsverhalten als auch die Robustheit der Algorithmen unter unterschiedlichen Lastbedingungen beurteilt werden können. Im Folgenden werden die wichtigsten Parameter aufgeführt.

THREAD-ANZAHL: Pro Algorithmus wird mit drei unterschiedlichen Thread-Konfigurationen gemessen: einen, vier und zehn Threads. Den VMs stehen jeweils sieben Kerne zur Verfügung, sodass die drei Stufen gezielt verschiedene Lastniveaus abbilden:

- **Ein Thread:** Sequenzielle Baseline ohne Parallelisierung.
- **Vier Threads:** Moderate Parallelisierung unterhalb der verfügbaren Kernanzahl. Zeigt das Skalierungsverhalten bei mittlerer Auslastung.
- **Zehn Threads:** Übersubskription der verfügbaren sieben Kerne. Zeigt wie der Algorithmus unter Ressourcenknappheit agiert.

WARTESCHLANGEN-GRÖSSE: Zusätzlich wird jede Thread-Konfiguration mit zwei unterschiedlichen Größen der Warteschlangen-Datenstruktur getestet. Die Größe der Warteschlange beeinflusst unmittelbar, wie viele Tasks vorab geladen und im Speicher gehalten werden.

BETRIEBSSYSTEME: Sämtliche Konfigurationen werden auf beiden Gast-Systemen, Debian 13 und Windows 11, durchgeführt. Dies ermöglicht eine plattformübergreifende Bewertung der Algorithmen und deckt potenzielle betriebssystemspezifische Unterschiede in der Thread- und Speicherverwaltung auf.

WIEDERHOLUNGEN UND STICHPROBENUMFANG: Jede Einzelkonfiguration wird 20 Mal wiederholt. Durch die Wiederholungen können Ausreißer identifiziert und die Varianz der Messwerte quantifiziert werden.

3.3 DIE TESTUMGEBUNG

Zur Gewährleistung einer hohen Reproduzierbarkeit sowie zur Minimierung externer Artefakte und Varianz in den Antwortzeiten wurden sowohl das Host-System als auch die Gast-Systeme gezielt optimiert. Grundsätzlich ließe sich argumentieren, dass bei einer ausreichend großen Anzahl an

Messungen ein Betrieb der Systeme im Standardzustand sinnvoll ist. Eine höhere Varianz (High-Fidelity) bildet die reale Betriebsumgebung präziser ab [4, S. 647]. Für den Vergleich der Stabilität wird der Variationskoeffizient herangezogen, da dieser die Streuung relativ zum Mittelwert normiert und somit eine vergleichbare Bewertung über unterschiedliche Laufzeitniveaus hinweg ermöglicht.

Da im Rahmen dieser Arbeit nur begrenzte Ressourcen zur Verfügung stehen, wurde stattdessen eine weitgehende Homogenisierung der Testumgebung angestrebt. Hierbei wird ein bewusster Laboreffekt in Kauf genommen. Die Resultate folgen dem Ansatz des Best-Case-Benchmarkings und sind daher als Idealwerte zu interpretieren, die von der durchschnittlichen Performance im Realbetrieb abweichen können.

Die im Folgenden aufgeführten Modifikationen wurden zur Stabilisierung der Messumgebung vorgenommen.

Tabelle 3.2: Konfiguration des Testsystems

Hardware	
Modell	Beelink SER5 MAX
Prozessor	AMD Ryzen 7 6800U (8 Kerne, 16 Threads, bis zu 4,7 GHz)
Arbeitsspeicher	24 GB LPDDR5 RAM
Speichermedium	500 GB M.2 2280 NVMe SSD
Grafikeinheit	AMD Radeon Graphics (12 Cores, 2200 MHz)
Virtualisierungs-Stack	
QEMU/KVM	Emulator und Kernel-basierte virtuelle Maschine als Hypervisor-Backend
libvirtd	Management-Daemon zur Steuerung der QEMU-Prozesse und Snapshots
virsh	Kommandozeilenschnittstelle zur Verwaltung der virtuellen Infrastruktur
Betriebssysteme	
Host-System	Debian 13.2.0
Gast-Systeme (VM)	Windows 11 Professional (Build 25H2) sowie Debian 13.2.0

3.3.1 *Speicher-Optimierungen*

DEAKTIVIERUNG DES MEMORY-BALLOONING: Die dynamische Speicheranpassung wurde deaktiviert, um zu verhindern, dass der Host-Kernel zur Laufzeit RAM-Seiten der VM zurückfordert oder neu zuweist. Dies unterbindet unvorhersehbare Latenzen, die durch Pagefaults oder Reclaim-Operationen entstehen können. Des Weiteren gibt es Informationen dazu, dass es Probleme mit Windows-VMs und den Ballooning-Treibern geben kann, welche somit umgangen werden. [11], [12], [13]

SPEICHERALLOKATION: Durch Angleichung von `currentMemory` und `memory` in der VM-Konfiguration wird der gesamte Arbeitsspeicher bereits beim Startvorgang der VM reserviert. Dies ist aufgrund der Deaktivierung des Memory-Balloonings theoretisch nicht nötig, die Einstellung wurde aber zur Sicherheit trotzdem vorgenommen. [14]

HUGEPAGES: In Kombination mit Hugepages (2 MB) auf dem Host (`/proc/sys/vm/nr_hugepages`) wird die Effizienz des Translation Lookaside Buffers (TLB) gesteigert und der Overhead der Adressübersetzung reduziert. [15], [16]

TRANSPARENT HUGE PAGES: Transparent Huge Pages sind deaktiviert. Dieser Hintergrunddienst des Kernels setzt kurzzeitig Locks auf den Speicher, was zu unvorhersehbaren Latenz-Spitzen führen kann. Um zu verhindern, dass der Kernel Speicherbereinigungen während der Messung durchführen muss, wird ihm mit `vm.min_free_kbytes` 1 GB zugewiesen, sodass er ausreichend freien Spielraum hat. [16], [17]

SWAPPINESS: Durch das Setzen des Parameters `swappiness` auf `0` wird verhindert, dass der Kernel frühzeitig ungenutzte Speicherseiten aus dem RAM auf die Festplatte auslagert. Der Wert 0 weist den Kernel an, das Auslagern so lange wie absolut möglich zu vermeiden. Dies sorgt dafür, dass die Benchmarks nicht durch langsame Festplatten-I/O ausgebremst werden. [17]

3.3.2 *Storage- und I/O-Optimierungen*

SPEICHER-CACHE: Für die virtuellen Datenträger wurde der Modus `cache='none'` gewählt. Dies erzwingt die Nutzung von `O_DIRECT`, wodurch der Page-Cache des Host-Betriebssystems umgangen wird. Messungen spiegeln somit die tatsächliche Performance des Storage-Backends wider, ohne durch Caching-Effekte des Hosts verfälscht zu werden. [16], [18]

I/O-SCHNITTSTELLE: Die Nutzung von `io='io_uring'` ermöglicht eine hocheffiziente, asynchrone Verarbeitung von I/O-Anfragen mit minimalem

Systemcall-Overhead, was besonders bei hohen Lasten die Varianz der Antwortzeiten senkt. [16]

DISCARD-MODUS: Der Discard-Modus wurde auf `ignore` gesetzt, um zu verhindern, dass TRIM-Befehle (`unmap`) während der Benchmarks rechenintensive Metadaten-Updates im Host-Dateisystem auslösen. [19]

3.3.3 *CPU-Optimierungen*

CPU-PINNING UND ISOLIERUNG: Um Kontextwechsel und Cache-Interferenzen durch den Host-Scheduler zu vermeiden, wurden die vCPUs der Gast-Systeme fest an physikalische Kerne gebunden (Pinning). Zusätzlich wurde ein `emulatorpin` konfiguriert, um QEMU-Hintergrundprozesse auf dedizierte Service-Kerne auszulagern, sodass die für den Benchmark genutzten Kerne exklusiv dem Gast zur Verfügung stehen [20]

HOST CPU GOVERNOR: Der Linux-Scaling-Governor wurde auf `performance` gesetzt. Dies fixiert die Taktfrequenz auf dem Maximum und verhindert Latenzspitzen, die durch das Hochfahren der CPU-Frequenz aus Energiesparzuständen (P-States) entstehen würden. [21]

DEAKTIVIERUNG VON HYPER-THREADING: Hyper-Threading wurde deaktiviert, sodass die VM exklusiven Zugriff auf die Ressourcen des gepinnten Kerns hat. Hyper-Threading kann durch Lastverteilungsprobleme zu Jitter führen, da die CPU-Pipeline je nach Auslastung des zweiten logischen Threads unterschiedlich schnell arbeitet. Dies ist ein weiterer Schritt, das Verhalten der CPU deterministischer zu machen. [22]

3.3.4 *System-Optimierungen*

DEAKTIVIERUNG VON ASLR: Address Space Layout Randomization (ASLR) wurde im Gast-System deaktiviert (`kernel.randomize_va_space=0`). Da ASLR Speicheradressen bei jedem Programmlauf zufällig anordnet, können minimale Unterschiede in den Cache-Trefferraten und der Branch-Prediction entstehen. Die Deaktivierung sorgt für eine deterministische Speicherausführung. [23]

HINTERGRUNDPROZESSE: Nicht kritische Systemdienste und geplante Aufgaben wurden deaktiviert, um kurzzeitige CPU-Lastspitzen während der Messreihen auszuschließen. Auf dem Host und der Debian-VM sind `cron` und `anacron` gestoppt. Nicht essentielle Dienste wie beispielsweise der Bildschirmschoner sind ebenfalls deaktiviert. Auf der Windows-VM wurden zusätzlich folgende Dienste deaktiviert:

- **Windows Search:** Die ständige Indizierung von Dateien im Hintergrund wurde beendet.

- **Aufgabenplanung:** Über `taskschd.msc` wurden geplante Wartungsaufgaben (z. B. Diagnoseläufe) eingesehen und deaktiviert.
- **Benachrichtigungen:** Das Benachrichtigungssystem wurde vollständig deaktiviert, um periodische Unterbrechungen durch das Betriebssystem zu unterbinden.

WINDOWS UPDATES UND DEFENDER: Automatische Updates sowie der Echtzeitschutz von Windows Defender wurden deaktiviert. Beide Dienste erzeugen im Hintergrund Festplatten- und CPU-Last durch das Herunterladen von Updates, das Scannen von Dateien sowie das Aktualisieren von Signaturdatenbanken. Dies hätte die Messungen durch unvorhersehbare I/O- und CPU-Spitzen verfälschen können.

VISUELLE EFFEKTE: Auf der Windows-VM wurden sämtliche Animationseffekte und Transparenzeffekte deaktiviert. Über die erweiterten Systemeinstellungen wurde die Option "Für optimale Leistung anpassen"gewählt. Dies entlastet GPU und CPU und eliminiert eine weitere Quelle nichtdeterministischer Hintergrundaktivität.

DEAKTIVIERUNG DES INTERNETZUGRIFFS: Durch Updates oder andere Internetzugriffe der Betriebssysteme können unerwünschte Aktivitäten auf der VM stattfinden. Der Internetzugriff wurde auf dem Host-System deaktiviert.

3.4 VORBEREITUNG DES SCANNERS

Durch die Implementierung eines Schedulers gibt es einen entscheidenden Architekturwechsel, weg von der engen Kopplung, in welcher jedes Modul seinen eigenen Scheduler steuert, hin zu einem zentralen Service-Modell mit dem Scheduler als eigenständige Komponente. Es gibt eine strikte Trennung. Die restliche Programmlogik weiß nicht, wie der Scheduler arbeitet, sie übergibt lediglich Tasks. Wenn ein Feature Kindobjekte erstellt, werden diese nicht mehr direkt verarbeitet, stattdessen werden sie als neue Tasks in die Warteschlange gegeben. Für den Scheduler ist es egal, ob ein Task von einem Modul oder einem Feature kommt, alles landet in derselben Warteschlange. Der restliche Scanner muss entsprechend angepasst werden, was je nach Komplexität und Kopplung eine Überarbeitung von sehr viel Code bedeuten kann.

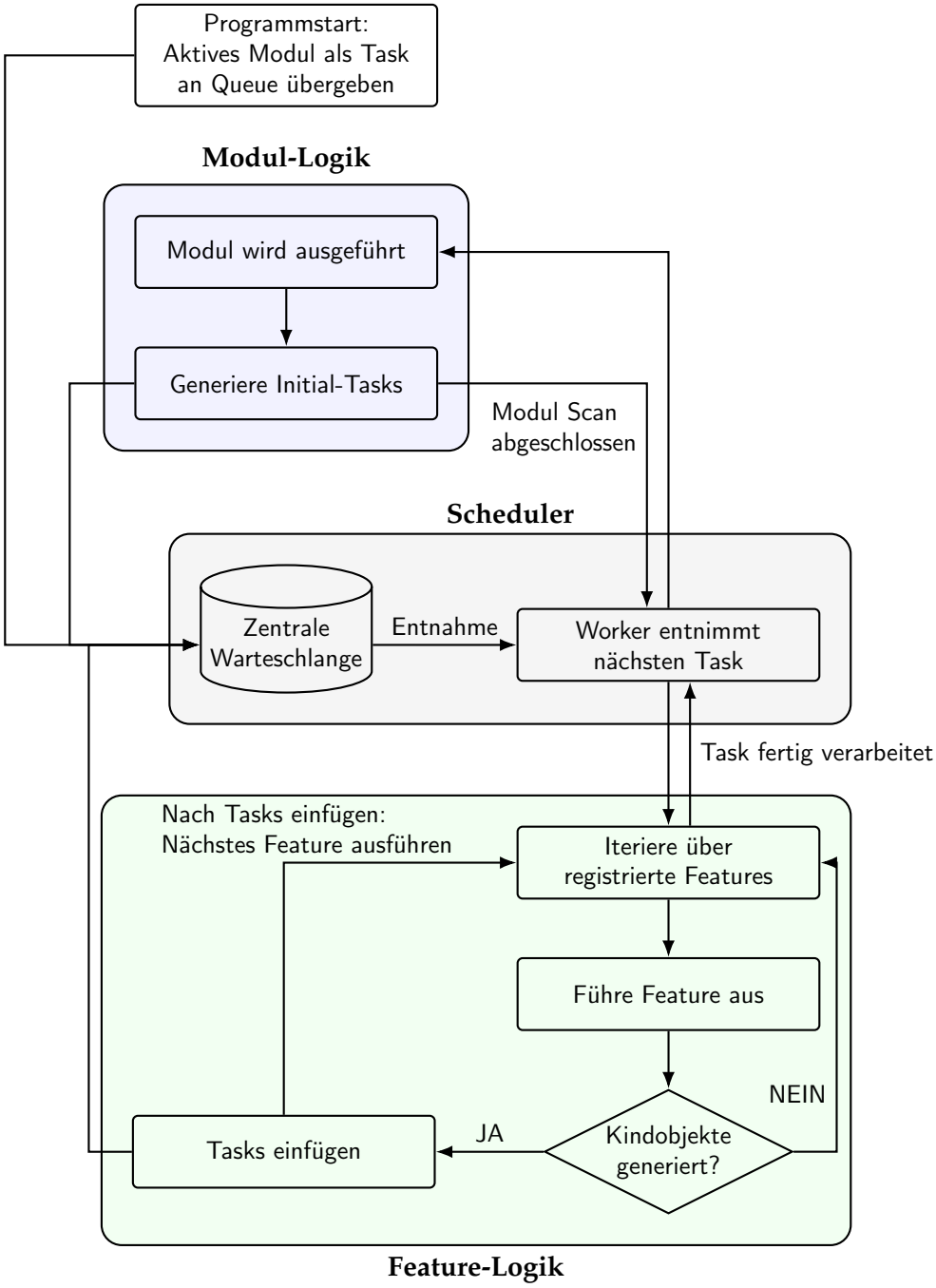

Abbildung 3.1: Arbeitsablauf mit entkoppeltem Scheduler

3.4.1 *Das Fertigstellungsproblem*

SYNCHRONE FERTIGSTELLUNG IM BISHERIGEN MODELL: Im bisherigen objektbasierten Scheduling-Modell verarbeitet ein Worker ein Objekt sowie sämtliche daraus rekursiv entstehenden Kindobjekte synchron innerhalb desselben Threads. Sobald der Worker terminiert ist, ist garantiert, dass das Objekt und sein gesamter Nachkommenbaum vollständig abgearbeitet sind. Code, der auf die Fertigstellung eines Objekts angewiesen ist, kann da-

her unmittelbar auf den Rückgabewert des Workers zugreifen, eine explizite Synchronisation ist nicht erforderlich.

VERLUST DER IMPLIZITEN FERTIGSTELLUNGSGARANTIE: Mit der Umstellung auf den taskbasierten Scheduler entfällt diese implizite Garantie. Ein Task wird abgearbeitet, reiht dabei seine Kindobjekte als eigenständige Tasks in die Warteschlange ein und meldet sich anschließend als abgeschlossen, obwohl die Kinder zu diesem Zeitpunkt vielleicht noch nicht einmal mit der Verarbeitung begonnen haben. Die Fertigstellung eines einzelnen Tasks impliziert somit nicht mehr die Fertigstellung des gesamten Teilbaums.

PROGRESS-TRACKING: Um dennoch feststellen zu können, ob ein Task und alle seine Nachkommen vollständig verarbeitet wurden, ist eine zusätzliche Datenstruktur erforderlich: ein Progress-Tracker. Dieser verwaltet für jeden Task einen Zähler über die Anzahl offener Kindtasks. Erst wenn ein Task selbst abgeschlossen ist und alle seine direkten sowie transitiven Nachkommen ebenfalls terminiert sind, gilt der Teilbaum als vollständig verarbeitet. Wartende Threads können mittels einer Condition Variable benachrichtigt werden, sobald dieser Zustand eintritt. Funktional entspricht dies einer Barrier Synchronization, also dem gemeinsamen Warten bis zu einem definierten Synchronisationspunkt.

ANPASSUNGEN IM BESTEHENDEN CODE: Der Scheduler stellt hierfür eine dedizierte `Wait()`-Funktion bereit. Sämtliche Stellen im Scanner, die bisher implizit davon ausgingen, dass nach Rückkehr eines Workers das gesamte Objekt inklusive aller Nachkommen verarbeitet ist, müssen identifiziert und auf die explizite Nutzung dieser Funktion umgestellt werden. Dies betrifft insbesondere Module, die nach einem Scan-Vorgang aggregierte Ergebnisse auswerten oder nachgelagerte Verarbeitungsschritte anstoßen.

3.5 DIE AUSWAHL UND IMPLEMENTIERUNG DER SCHEDULING-ALGORITHMEN

Die Auswahl der Scheduling-Algorithmen basiert auf einer zielgerichteten Optimierung bestimmter Metriken in einem gegebenen System. Es liegt mit dem Scanner ein System vor, in welchem die Tasks asynchron und kontinuierlich dem Scheduler übergeben werden. Die Ausführung dieser Tasks soll wenn möglich schnell geschehen, es werden allerdings keine Deadline-Garantien benötigt. Präemption wäre theoretisch möglich, allerdings mit einem erheblichen Implementierungsaufwand verbunden. Da keine Deadline-Anforderungen bestehen und durch die Nutzung von präemptiven Scheduling-Algorithmen ein Kontextwechsel- und Synchronisations-Overhead entsteht, wurde darauf verzichtet. Die Last-Verteilungsstrategie ist frei wählbar.

Die Anzahl an Scheduling-Algorithmen, welche diese Anforderungen erfüllen, ist extrem hoch. Weitere Empfehlungen für vielversprechende Scheduling-Algorithmen, welche aufgrund der begrenzten Ressourcen dieser Arbeit nicht getestet werden konnten, sind in Kapitel 6 zu finden.

Die Auswahl ist auf folgende Scheduling-Algorithmen gefallen:

1. First-In, First-Out (FIFO)
2. Shortest-Subtree-Time-First (SSTF)
3. Work-Stealing kombiniert mit Shortest-Subtree-Time-First (WS+SSTF)

Es wurde des Weiteren ein adaptiver Multi-Level-Feedback-Warteschlangen-Scheduler und ein FIFO-Scheduler mit adaptivem Backpressure getestet. Diese zwei Scheduling-Algorithmen wurden aufgrund von schlechten Ergebnissen bei ersten Benchmarks verworfen und es wurden keine Testreihen angelegt.

3.5.1 *Formale Notation und Vergleichsrahmen*

Zur Sicherstellung der Vergleichbarkeit verwenden alle Scheduler-Varianten denselben formalen Rahmen.

NOTATION:

$$\begin{aligned} W &:= \max(1, \text{konfigurierter Threadcount}) \\ N(t) &:= \text{Anzahl akzeptierter, noch nicht abgeschlossener Tasks zum Zeitpunkt } t \end{aligned}$$

Für jede Scheduler-Variante $x \in \{\text{FIFO}, \text{SSTF}, \text{WS}\}$ wird die Queue-Kapazität separat als C_x angegeben, da sie implementierungsabhängig ist.
Für jeden Task τ werden folgende Zeitpunkte betrachtet:

$$t_{\text{enqueue}}(\tau), \quad t_{\text{start}}(\tau), \quad t_{\text{end}}(\tau).$$

Die Warteschlangenzeit ist definiert als

$$t_{\text{queue}}(\tau) = \begin{cases} t_{\text{start}}(\tau) - t_{\text{enqueue}}(\tau), & \text{bei asynchroner Queue-Ausführung} \\ 0, & \text{bei synchroner Inline-Ausführung.} \end{cases}$$

Die exklusive Laufzeit lautet

$$t_{\text{exec}}^{\text{exkl}}(\tau) = \max\Big(0,\ (t_{\text{end}}(\tau) - t_{\text{start}}(\tau)) - t_{\text{sync-child}}(\tau)\Big),$$

wobei $t_{\text{sync-child}}(\tau)$ die kumulierte Laufzeit synchron ausgeführter Kind-Tasks derselben Ausführungskette bezeichnet.

WORKER- UND PARALLELITÄTSMODELL: Es werden langlebige Worker-Goroutines betrieben. Die effektive Zahl gleichzeitig laufender Task-Ausführungen wird durch eine Semaphore auf höchstens W begrenzt. Damit sind Einreihungs- und Entnahmeordnung von Abschlussordnung zu unterscheiden: Abschlussreihenfolgen können bei Parallelität grundsätzlich nicht deterministisch bestimmt werden.

ABSCHLUSSBEDINGUNG: Ein Task erhöht beim erfolgreichen Akzeptieren den Zähler N um 1 und dekrementiert N genau einmal beim Abschluss (normal, abgebrochen oder Fallback (Inline-Ausführung im Caller-Kontext) beendet). Ein Scan ist genau dann abgeschlossen, wenn $N = 0$ erreicht wird; die Wartesynchronisation erfolgt über eine Condition Variable.

BACKPRESSURE: Bei Enqueue-Fehlschlag erfolgt ein synchroner Fallback. Ein Enqueue-Fehlschlag kann je nach Implementierung durch volle oder geschlossene Queue entstehen. Dies begrenzt Queue-Wachstum, verändert aber globale Ausführungsordnung und Fairnesseigenschaften, da die Nachkommen dieses Tasks direkt synchron abgearbeitet werden.

3.5.2 *FIFO-Scheduler*

ZIELSETZUNG: Der FIFO-Scheduler dient als Baseline mit minimaler struktureller Komplexität.

CHARAKTERISTISCHE EIGENSCHAFTEN:

- Einfache Baseline mit klarer Queue-Semantik
- FIFO-Ordnung im Queue-Pfad
- Potenziell erhöhte Wartezeiten bei stark heterogenen Taskkosten
- Keine globale strikte FIFO-Abschlussreihenfolge

DATENSTRUKTUREN: Es wird eine zentrale thread-sichere FIFO-Warteschlange verwendet. Für den FIFO-Scheduler wurden zwei Implementierungsvarianten getestet, die in ihrer Logik identisch sind und sich ausschließlich in der Kapazität der zentralen Warteschlange unterscheiden:

$$C_{\text{FIFO50}} = 50 \cdot W$$

und

$$C_{\text{FIFO500}} = 500 \cdot W$$

ORDNUNGS- UND DETERMINISMUSEIGENSCHAFT: Die zentrale Warteschlange arbeitet nach dem FIFO-Prinzip. Eine globale strikte FIFO-Ordnung über alle Tasks gilt nicht, da mehrere Worker parallel arbeiten und bei Enqueue-Fehlschlag der synchrone Fallback aktiv wird.

EINREICHUNGSPFAD:

1. Falls Kontext beendet: Rückgabe mit abgeschlossenem Handle.
2. Sonst: $N \leftarrow N + 1$.
3. Bei Sync-Kontext: Inline-Ausführung, $N \leftarrow N - 1$.
4. Andernfalls: nicht-blockierender Enqueue-Versuch in Warteschlange.
5. Bei Enqueue-Fehlschlag: synchroner Fallback, $N \leftarrow N - 1$.

AUSFÜHRUNGSPFAD: Worker warten blockierend auf Queue-Elemente. Nach Entnahme wird ein Semaphore-Token angefordert. Bei Fehlschlag, also zum Beispiel bei abgebrochenem Kontext, wird der Task abgeschlossen. Bei Erfolg: Queue-Zeit messen, Kontext anreichern, Task ausführen, exklusive Laufzeit berechnen, Statistik schreiben, Handle schließen, Semaphore freigeben und $N \leftarrow N - 1$.

3.5.3 *Shortest-Subtree-Time-First (SSTF)*

ZIELSETZUNG: SSTF priorisiert Tasks anhand einer vordefinierten Laufzeitheuristik, sodass Tasks mit kürzerer erwarteter Gesamtausführungszeit ihres Teilbaums bevorzugt ausgeführt werden. Die Priorität eines Tasks entspricht der Summe aus seiner eigenen Ausführungszeit und der kumulierten Ausführungszeit aller Nachkommen-Tasks. Eine dynamische Anpassung der Prioritätstabelle während der Laufzeit wäre prinzipiell realisierbar, wurde in dieser Arbeit jedoch aufgrund begrenzter zeitlicher Ressourcen nicht berücksichtigt. Dieser Scheduling-Algorithmus wurde gewählt, um die mittlere Wartezeit zu reduzieren und zugleich den Anteil synchron ausgeführter Tasks gering zu halten.

CHARAKTERISTISCHE EIGENSCHAFTEN:

- Geringere mittlere Wartezeit bei gut passender Prioritätsheuristik
- Bevorzugung kurzer Subtrees gemäß Prioritätstabelle
- Starvation-Risiko für lange Tasks
- Empfindlich gegenüber Fehlklassifikation in Prioritätenliste P

PRIORITÄTSMODELL: Mit

$$P : \mathcal{T} \to \mathbb{N}$$

ergibt sich für Task τ:

$$prio(\tau) = \begin{cases} P(type(\tau)), & \text{falls bekannt} \\ P_{\text{default}}, & \text{sonst.} \end{cases}$$

Die Prioritäten stammen aus einer statischen Priority-Map, die aus mehreren Scans des FIFO-Schedulers abgeleitet und über diese gemittelt wurde.

DATENSTRUKTUR: Zentrale, thread-sichere Prioritätswarteschlange auf Basis eines Min-Heaps mit Kapazität

$$C_{\text{SSTF}} = 50 \cdot W,$$

die ein effizientes Einfügen und Entnehmen von Tasks in $O(\log n)$ ermöglicht.

EINREICHUNG UND AUSFÜHRUNG: Einreichungslogik entspricht dem gemeinsamen Rahmen (Kontextprüfung, N-Tracking, Sync-Bypass, Fallback). Ausführung entnimmt stets einen Task mit höchster Priorität.

3.5.4 *Work-Stealing mit SSTF-Priorisierung*

ZIELSETZUNG: Kombination aus lokaler Queue-Verarbeitung und Stealing zur Verbesserung von Skalierbarkeit und Lastverteilung bei Beibehaltung der SSTF-Priorisierung. Neben den Vorteilen, welche bereits durch den SSTF-Scheduler entstehen, soll hier der Overhead durch Locks auf eine zentrale Warteschlange verhindert werden.

CHARAKTERISTISCHE EIGENSCHAFTEN:

- Reduzierte zentrale Lock-Contention
- Gute Lastverteilung bei ungleichmäßiger Last
- Höhere Systemkomplexität

DATENSTRUKTUREN: Jeder Worker w_i besitzt eine eigene thread-sichere Min-Heap-Queue Q_i. Die Prioritätsordnung folgt SSTF. Die Kapazität pro Worker-Queue beträgt

$$C_{\text{WS,pro Worker}} = 100,$$

also insgesamt

$$C_{\text{WS,gesamt}} = 100 \cdot W.$$

EINREICHUNGSPFAD:

1. Kontextprüfung, danach $N \leftarrow N + 1$.
2. Prioritätsbestimmung.
3. Zielqueue-Wahl: bevorzugt lokale Queue (bei Worker-Kontext), sonst definierte Fallback-Queue.

4. Falls der Aufrufer ein Worker ist und in die eigene Queue schreibt: zunächst nicht-blockierender TryPush, dann Spillover (andere Worker-Queues, danach Modul-Queue), erst danach synchroner Fallback.
5. Falls der Aufrufer extern ist oder der Modul-Worker einreiht: blockierender Push auf die Zielqueue (Backpressure bei voller Queue). Bei geschlossener Queue wird der synchrone Fallback verwendet.

AUSFÜHRUNGSPFAD: Worker suchen in folgender Stufenreihenfolge:

1. eigene Queue,
2. Fallback-/Modul-Queue,
3. andere Worker-Queues (Stealing mit zufälligem Startindex und zyklischer Suche).

Bei gefundenem Task erfolgt Ausführung analog zum gemeinsamen Rahmen (Semaphore, Messung, Abschluss, N-Dekrement).

ORDNUNGS- UND FAIRNESSEIGENSCHAFT: Lokale Prioritätsordnung ist stark, globale Prioritätsordnung nur approximativ. Fairness kann durch lokale Heuristiken und Stealing-Reihenfolgen sinken.

3.5.5 *Validitätsaspekte und Reproduzierbarkeit*

Es können folgende Konfigurationsparameter, je nach Scheduling-Algorithmus, angepasst werden:

KONFIGURATIONSPARAMETER:

- W (Threadcount-basiert),
- Warteschlange-Kapazitäten je Scheduler-Variante,
- Prioritätsmap P und P_{default}

INTERNE VALIDITÄT: Messwerte werden durch Laufzeit-Scheduling, Lastmischung und Fallback-Häufigkeit beeinflusst. Vergleiche zwischen Schedulern sind daher nur bei identischer Workload, identischen Stopkriterien und identischer Instrumentierung valide.

EXTERNE VALIDITÄT: Ergebnisse sind auf vergleichbare Task-Granularität, Subtree-Struktur und Systemlast übertragbar. Bei stark abweichenden Lastprofilen ist gegebenenfalls die Verwendung einer adaptiven Implementierung oder eine erneute Kalibrierung von P sinnvoll.

REPRODUZIERBARKEIT: Um die Reproduzierbarkeit der Messergebnisse zu bewerten, wird der Variationskoeffizient (engl. *Coefficient of Variation*, CV) der Laufzeit als relatives Maß für die Streuung herangezogen. Im Gegensatz zur Standardabweichung erlaubt der CV den Vergleich der Variabilität von Datensätzen mit unterschiedlichen Mittelwerten, indem er die Streuung normiert. Er ist definiert als das Verhältnis der Standardabweichung σ zum arithmetischen Mittelwert μ:

$$\text{CV} = \frac{\sigma}{\mu} \cdot 100\% \tag{3.1}$$

Ein niedriger CV-Wert signalisiert eine hohe Präzision des Testaufbaus, während ein steigender Wert auf eine zunehmende Instabilität oder eine stärkere gegenseitige Beeinflussung der Threads hindeutet.

4 ERGEBNISSE UND DISKUSSION

In diesem Kapitel werden die Ergebnisse der Messungen für die verschiedenen Scheduler vorgestellt, miteinander verglichen und im Anschluss diskutiert. Im Anhang befinden sich alle Grafiken zu den Messungen in nach Scheduling-Algorithmen geordneter Reihenfolge.

Die Ergebnisse werden in vier Abschnitten präsentiert:

1. Die Ergebnisse des alten Schedulers werden als Referenz vorgestellt.
2. Die Ergebnisse des FIFO-Schedulers werden vorgestellt und mit der Referenz verglichen, um die Auswirkungen der Einführung eines taskbasierten Schedulers zu bewerten.
3. Die Ergebnisse des SSTF-Schedulers werden vorgestellt und mit dem FIFO-Scheduler verglichen, um die Auswirkungen der Priorisierung von Tasks zu bewerten.
4. Die Ergebnisse des Work-Stealing-Schedulers mit SSTF-Priorisierung werden vorgestellt und mit dem SSTF-Scheduler verglichen, um die Auswirkungen der Kombination von Work-Stealing mit einer Priorisierungsstrategie zu bewerten.

Die Algorithmen bauen aufeinander auf, sodass die Ergebnisse des vorherigen Schedulers jeweils als Baseline für die Bewertung des nachfolgenden Schedulers herangezogen werden. Nach jeder Vorstellung der Ergebnisse eines Schedulers erfolgt eine Diskussion der Ergebnisse. Am Ende des Kapitels werden die Ergebnisse aller Scheduler miteinander verglichen und diskutiert.

4.1 REKURSIV-SYNCHRONER, OBJEKTBASIERTER SCHEDULER (REFERENZ)

Der nicht optimierte Scanner wurde als Referenz für die Messungen mit dem neuen Scheduler verwendet. In der Tabelle 4.1 sind die Mittelwerte der Laufzeit, CPU-Auslastung, RAM-Auslastung (Mittel und Max) sowie der Variationskoeffizient der Laufzeit für die Messungen mit dem alten Scheduler aufgelistet. Die internen Metriken wurden bei diesem nicht erhoben, um die Performance des alten Schedulers nicht zu beeinträchtigen.

Tabelle 4.1: Übersicht der Laufzeit-, CPU-, RAM- und Runtime-CV-Metriken des alten Schedulers für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte ± Standardabweichung aus $n = 20$ Läufen

Threads	OS	Runtime [s]	CPU [%]	RAM Mittel [MB]	RAM Max [MB]	Runtime CV [%]
1	Linux	643 ± 3.89	24.0 ± 0.00	1069 ± 17.8	1670 ± 38.0	0.60
4	Linux	266 ± 2.61	55.0 ± 0.32	1492 ± 22.5	2612 ± 79.5	0.98
10	Linux	221 ± 3.07	63.0 ± 0.74	1866 ± 36.9	3374 ± 86.9	1.39
1	Windows	2514 ± 8.57	21.0 ± 0.	1350 ± 2.15	5937 ± 82.2	0.34
4	Windows	1214 ± 27.0	46.6 ± 0.80	2019 ± 50.5	7306 ± 248.2	2.22
10	Windows	826 ± 20.5	68.5 ± 1.96	2879 ± 281	12190 ± 1055.5	2.49

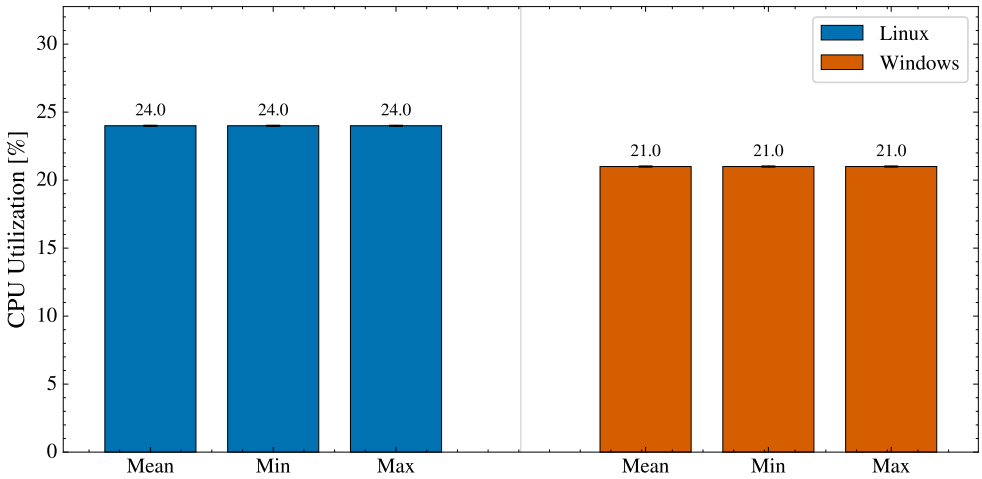

Abbildung 4.1: CPU-Auslastung des Referenz-Schedulers auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

Für die Bewertung der Stabilität des Testsystems ist die CPU-Auslastung maßgeblich. Bei der Ausführung mit einem Thread ist ihre Standardabweichung sowohl unter Linux als auch unter Windows extrem gering, wie in Abbildung 4.1 dargestellt. Bei vier Threads steigt sie auf 0.32 bzw. 0.80 und bei zehn Threads auf 0.74 bzw. 1.96 (siehe Abb. 4.2). Zugleich steigt auch der Variationskoeffizient der Laufzeit (*Runtime CV*) in Tabelle 4.1 von 0.60 auf 0.98 und 1.39 unter Linux sowie von 0.34 auf 2.22 und 2.49 unter Windows. Damit lässt sich anhand der Referenzwerte interpretieren, dass die Streuung mit steigender Thread-Anzahl zunimmt.

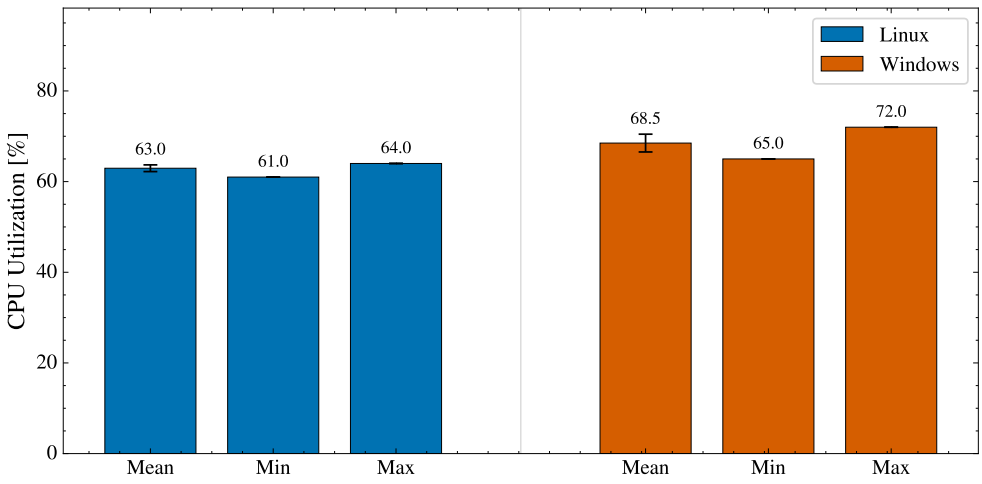

Abbildung 4.2: CPU-Auslastung des Referenz-Schedulers auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

4.2 FIRST-IN, FIRST-OUT

In diesem Abschnitt werden die Ergebnisse der Messungen mit dem FIFO-Scheduler vorgestellt und mit der Referenz verglichen. Für den FIFO-Scheduler wurden zwei verschiedene Warteschlangenlängen (50- und 500-mal die Anzahl der Threads) getestet, um die Auswirkungen der Warteschlangenlänge auf die Performance zu untersuchen. Im Unterschied zur Referenz wurden hier auch die internen Metriken wie Queue Time, Execution Time und der Anteil der synchronen Tasks erhoben.

Tabelle 4.2: Laufzeit-, CPU- und RAM-Metriken des FIFO_50-Schedulers für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte $\pm$ Standardabweichung aus $n = 20$ Läufen

Threads	OS	Runtime [s]	CPU [%]	RAM Mittel [MB]	RAM Max [MB]
1	Linux	656 $\pm$ 3.15	23.6 $\pm$ 0.497	1065 $\pm$ 33.8	1705 $\pm$ 57.1
4	Linux	271 $\pm$ 3.09	54.0 $\pm$ 0.447	1603 $\pm$ 22.7	2891 $\pm$ 74.1
10	Linux	220 $\pm$ 3.34	62.2 $\pm$ 1.09	1954 $\pm$ 29.7	3968 $\pm$ 163.2
1	Windows	2506 $\pm$ 8.22	21.0 $\pm$ 0	1393 $\pm$ 6.47	6015 $\pm$ 102.9
4	Windows	1126 $\pm$ 23.3	48.6 $\pm$ 0.917	2188 $\pm$ 85.1	7766 $\pm$ 369.9
10	Windows	751 $\pm$ 25.4	71.1 $\pm$ 2.43	3213 $\pm$ 282	11672 $\pm$ 837.3

Tabelle 4.3: Laufzeit-, CPU- und RAM-Metriken des FIFO_500-Schedulers für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte ± Standardabweichung aus $n = 20$ Läufen

Threads	OS	Runtime [s]	CPU [%]	RAM Mittel [MB]	RAM Max [MB]
1	Linux	655 ± 3.82	23.7 ± 0.458	1057 ± 20.3	1630 ± 46.7
4	Linux	274 ± 4.04	54.3 ± 0.843	1607 ± 30	3013 ± 125.3
10	Linux	216 ± 3.34	63.8 ± 0.942	1922 ± 36.9	3649 ± 136.2
1	Windows	2502 ± 11.5	21.0 ± 0	1396 ± 4.84	6041 ± 111.7
4	Windows	1112 ± 29.2	49.9 ± 1.18	2294 ± 237	10782 ± 932.2
10	Windows	740 ± 31.5	72.2 ± 2.72	3107 ± 160	10672 ± 742.7

Die Intervalle aus Mittelwert ± Standardabweichung für Laufzeit, CPU-Auslastung und mittlere RAM-Auslastung überlappen zwischen FIFO_50 und FIFO_500 in allen gezeigten Konfigurationen. Für diese drei Metriken lässt sich damit kein belastbarer Unterschied zwischen beiden Warteschlangenlängen belegen. Für die RAM-Maximalwerte zeigt sich nur bei Windows und vier Threads ein klarer Unterschied: FIFO_500 erreicht mit 10782 ± 932.2 MB höhere Maximalwerte als FIFO_50 mit 7766 ± 369.9 MB. In den übrigen Konfigurationen überlappen die Standardabweichungsintervalle.

Verglichen mit der Referenz ergibt sich auf Windows bei vier und zehn Threads für beide FIFO-Konfigurationen ein erheblicher Laufzeitvorteil. Auf Linux lässt sich ein solcher Vorteil nicht zeigen, bei einem Thread sind beide FIFO-Konfigurationen und bei vier Threads FIFO_500 sogar langsamer als die Referenz.

Tabelle 4.4: Runtime CV sowie Queue Time, Execution Time und Anteil synchroner Tasks des FIFO_50-Schedulers für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte ± Standardabweichung aus $n = 20$ Läufen

Threads	OS	Runtime CV [%]	Queue [ms]	Exec [ms]	Sync Tasks [%]
1	Linux	0.48	23.52 ± 0.12	0.98 ± 0.01	99.97 ± 0.00
4	Linux	1.14	27.98 ± 0.44	0.66 ± 0.01	54.06 ± 1.96
10	Linux	1.52	44.08 ± 1.49	1.08 ± 0.02	41.47 ± 2.06
1	Windows	0.33	34.68 ± 0.12	1.09 ± 0.00	99.98 ± 0.00
4	Windows	2.07	46.89 ± 1.04	1.09 ± 0.02	63.84 ± 5.46
10	Windows	3.38	51.67 ± 2.22	1.43 ± 0.03	58.78 ± 4.66

Tabelle 4.5: Runtime CV sowie Queue Time, Execution Time und Anteil synchroner Tasks des FIFO_500-Schedulers für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte ± Standardabweichung aus $n = 20$ Läufen

Threads	OS	Runtime CV [%]	Queue [ms]	Exec [ms]	Sync Tasks [%]
1	Linux	0.58	228.61 ± 1.51	0.97 ± 0.01	99.85 ± 0.00
4	Linux	1.48	278.90 ± 8.87	0.69 ± 0.01	65.44 ± 0.97
10	Linux	1.55	426.67 ± 10.08	1.10 ± 0.02	53.57 ± 2.30
1	Windows	0.46	337.52 ± 1.60	1.09 ± 0.01	99.88 ± 0.00
4	Windows	2.62	481.52 ± 13.97	1.10 ± 0.02	79.56 ± 2.98
10	Windows	4.26	510.97 ± 17.40	1.43 ± 0.02	71.92 ± 3.80

Die Queue Time ist bei FIFO_50 in allen Konfigurationen deutlich geringer als bei FIFO_500. Beim Anteil synchron ausgeführter Tasks zeigt sich der gleiche Effekt insbesondere bei vier und zehn Threads deutlich, auch hier überlappen die Standardabweichungsintervalle der beiden Konfigurationen nicht.

4.2.1 *Diskussion der FIFO-Ergebnisse*

STABILITÄT: Wie beim Referenz-Scheduler ist die Standardabweichung der CPU-Auslastung bei einem Thread klein (Linux 0.50, Windows 0.00), steigt bei vier Threads auf 0.45 bzw. 0.92 und bei zehn Threads auf 1.09 bzw. 2.43. Es steigt auch der Runtime CV von 0.48 auf 1.14 und 1.52 unter Linux sowie von 0.33 auf 2.07 und 3.38 unter Windows. Damit nimmt die Streuung auch beim FIFO-Scheduler mit wachsender Thread-Anzahl zu. Eine Erklärung ist die mit mehr Threads steigende Ressourcenkonkurrenz.

LAUFZEIT: Ein belegbarer Laufzeitvorteil des FIFO-Schedulers gegenüber der Referenz zeigt sich nur auf Windows bei vier und zehn Threads. Für Linux lässt sich kein entsprechender Vorteil belegen, vielmehr sind beide FIFO-Konfigurationen bei einem Thread und FIFO_500 zusätzlich bei vier Threads gesichert langsamer als die Referenz. Die Ursachen dieser plattformspezifischen Unterschiede müssen gesondert untersucht werden.

WAITING TIME UND BURST TIME: Die Auswirkungen der Warteschlangenlänge auf die Queue Time sind statistisch klar: FIFO_500 weist in allen Konfigurationen eine klar höhere Queue Time auf als FIFO_50. Für Laufzeit, CPU-Auslastung und mittlere RAM-Auslastung dagegen lässt sich kein belastbarer Unterschied zwischen den Queue-Längen erkennen. Weitere Tests mit kleineren Warteschlangen könnten zeigen, ob sich die Queue Time weiter reduzieren lässt, ohne einen belegbaren Nachteil bei den übrigen Metriken zu erzeugen.

SYNCHRON AUSGEFÜHRTE TASKS UND RAM-AUSLASTUNG: Bei vier und zehn Threads ist der Anteil synchron ausgeführter Tasks bei FIFO_500 messbar höher als bei FIFO_50. Für die RAM-Maximalwerte zeigt sich ein klar höherer Wert von FIFO_500 gegenüber FIFO_50 nur bei Windows und vier Threads. In den übrigen Konfigurationen lässt sich kein Unterschied belegen.

VORTEILE UND NACHTEILE DER FIFO-SCHEDULER IM VERGLEICH ZUR REFERENZ:

- Vorteile:
  - Auf Windows zeigen beide FIFO-Konfigurationen bei vier und zehn Threads einen belegbaren Laufzeitvorteil gegenüber der Referenz.
  - FIFO_50 zeigt in allen Konfigurationen eine deutlich geringere Queue Time als FIFO_500 und bei vier und zehn Threads zudem messbar weniger synchron ausgeführte Tasks.
- Nachteile:
  - Auf Linux lässt sich kein konsistenter Laufzeitvorteil gegenüber der Referenz belegen, einzelne Konfigurationen sind sogar klar langsamer.
  - Für CPU-Auslastung und mittlere RAM-Auslastung ergibt sich zwischen FIFO_50 und FIFO_500 kein belastbarer Unterschied, sodass die größere Queue keine belastbaren Vorteile zeigt.
  - Bei Windows und vier Threads ist der RAM-Maximalwert von FIFO_500 messbar höher als der von FIFO_50. Bei zehn Threads lässt sich dieser Unterschied nicht belegen.

4.3 SHORTEST-SUBTREE-TIME-FIRST

In diesem Abschnitt werden die Ergebnisse der Messungen mit dem SSTF-Scheduler vorgestellt. Das Ziel dieser Implementierung war es, die mittlere Wartezeit zu reduzieren und zugleich den Anteil synchron ausgeführter Tasks gering zu halten. Dazu werden Tasks mit kürzerer erwarteter Gesamtausführungszeit ihres Teilbaums bevorzugt ausgeführt. Der SSTF-Scheduler wird mit den FIFO-Schedulern verglichen, um die Auswirkungen des Priorisierungsansatzes auf die Performance zu bewerten.

Tabelle 4.6: Laufzeit-, CPU- und RAM-Metriken des SSTF-Schedulers für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte ± Standardabweichung aus $n = 20$ Läufen

Threads	OS	Runtime [s]	CPU [%]	RAM Mittel [MB]	RAM Max [MB]
1	Linux	658 ± 3.27	23.9 ± 0.357	1068 ± 25.8	1688 ± 47.6
4	Linux	273 ± 3.5	53.9 ± 0.436	1595 ± 36.4	3136 ± 132.2
10	Linux	223 ± 4.11	62.3 ± 0.781	1922 ± 48.4	3631 ± 131.5
1	Windows	2526 ± 12.1	21.0 ± 0	1393 ± 5.3	5999 ± 99.5
4	Windows	1124 ± 24.4	48.9 ± 1.11	2209 ± 60.6	7844 ± 380.4
10	Windows	752 ± 25.6	71.8 ± 2.41	3210 ± 210	9592 ± 641.6

Tabelle 4.6 zeigt Laufzeit, CPU-Auslastung und RAM-Auslastung des SSTF-Schedulers. Für Laufzeit, CPU-Auslastung und mittlere RAM-Auslastung überlappen die Standardabweichungsintervalle in den meisten Vergleichen mit FIFO_50 und FIFO_500. Für diese Metriken lässt sich daher überwiegend kein belastbarer Unterschied erkennen. Für die maximale RAM-Auslastung ergibt sich ebenfalls kein konsistenter Vorteil: Bei vier Threads auf Linux liegt SSTF mit 3136 ± 132.2 MB messbar über FIFO_50 mit 2891 ± 74.1 MB, bei zehn Threads auf Windows dagegen mit 9592 ± 641.6 MB messbar unter FIFO_50 mit 11672 ± 837.3 MB. Gegenüber FIFO_500 ist SSTF bei vier Threads auf Windows mit 7844 ± 380.4 MB erkennbar günstiger als FIFO_500 mit 10782 ± 932.2 MB. In den übrigen Konfigurationen lässt sich gegenüber FIFO_500 kein belastbarer Unterschied erkennen.

Tabelle 4.7: Runtime CV sowie Queue Time, Execution Time und Anteil synchroner Tasks des SSTF-Schedulers für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte ± Standardabweichung aus $n = 20$ Läufen

Threads	OS	Runtime CV [%]	Queue [ms]	Exec [ms]	Sync Tasks [%]
1	Linux	0.50	23.52 ± 0.12	0.49 ± 0.00	99.97 ± 0.00
4	Linux	1.28	25.71 ± 0.45	0.64 ± 0.01	44.09 ± 2.44
10	Linux	1.84	42.87 ± 1.04	1.08 ± 0.02	31.23 ± 1.16
1	Windows	0.48	34.64 ± 0.18	0.70 ± 0.00	99.95 ± 0.00
4	Windows	2.17	45.06 ± 1.34	1.07 ± 0.02	53.51 ± 4.05
10	Windows	3.40	49.73 ± 1.61	1.44 ± 0.02	40.08 ± 2.65

Tabelle 4.7 enthält Runtime CV, Queue Time, Execution Time und den Anteil synchroner Tasks. Die Queue Time ist gegenüber FIFO_500 in allen Konfigurationen deutlich geringer. Gegenüber FIFO_50 lässt sich ein belastbarer Unterschied nur bei Linux mit vier Threads zeigen, in den übrigen Fällen überlappen die Standardabweichungsintervalle. Für die Execution Time zeigt sich ein belegbarer Vorteil von SSTF bei einem Thread auf beiden Betriebssystemen sowie bei vier Threads unter Linux, nicht jedoch konsistent

bei höherer Last. Der Anteil synchron ausgeführter Tasks ist bei vier und zehn Threads gegenüber beiden FIFO-Konfigurationen erkennbar geringer.

4.3.1 *Diskussion der SSTF-Ergebnisse*

STABILITÄT: Die Stabilität ist mit FIFO vergleichbar. Die Standardabweichung der CPU-Auslastung beträgt bei einem Thread 0.36 unter Linux und 0.00 unter Windows, steigt bei vier Threads auf 0.44 bzw. 1.11 und bei zehn Threads auf 0.78 bzw. 2.41. Zugleich nimmt auch der Runtime CV mit der Thread-Anzahl zu.

LAUFZEIT: Gegenüber FIFO_50 lässt sich in keiner Konfiguration ein belastbarer Laufzeitunterschied erkennen. Gegenüber FIFO_500 ist SSTF nur bei einem Thread auf Windows messbar langsamer, in allen übrigen Konfigurationen überlappen die Standardabweichungsintervalle.

WAITING TIME UND BURST TIME: Gegenüber FIFO_500 ist die Queue Time von SSTF in allen Konfigurationen klar geringer. Gegenüber FIFO_50 lässt sich dieser Vorteil nur bei vier Threads auf Linux belegen, in den übrigen Fällen reichen die Standardabweichungen nicht für eine Trennung der Intervalle.

SYNCHRON AUSGEFÜHRTE TASKS UND RAM-AUSLASTUNG: Bei vier und zehn Threads ist der Anteil synchron ausgeführter Tasks gegenüber beiden FIFO-Konfigurationen erkennbar geringer. Für die mittlere RAM-Auslastung ergibt sich dagegen kein belastbarer Unterschied. Auch bei der RAM-Maximal-Auslastung zeigt sich kein einheitliches Bild: Gegenüber FIFO_50 ist die Auslastung SSTF bei vier Threads unter Linux messbar höher, bei zehn Threads auf Windows jedoch messbar geringer. Die Auslastung gegenüber FIFO_500 ist SSTF bei vier Threads auf Windows erkennbar geringer, in den übrigen Konfigurationen jedoch nicht unterscheidbar.

VORTEILE UND NACHTEILE IM VERGLEICH ZU FIFO:

- Vorteile:
  - Gegenüber FIFO_500 zeigt SSTF in allen Konfigurationen eine klar geringere Queue Time.
  - Bei vier und zehn Threads ist der Anteil synchron ausgeführter Tasks erkennbar geringer als bei FIFO_50 und FIFO_500.
  - Bei einem Thread auf beiden Betriebssystemen und bei vier Threads unter Linux ist die Execution Time messbar geringer als bei FIFO.
  - Bei vier Threads auf Windows ist der RAM-Maximalwert erkennbar geringer als bei FIFO_500.
- Nachteile:

- Ein konsistenter belegbarer Laufzeitvorteil gegenüber FIFO lässt sich nicht zeigen, gegenüber FIFO_500 ist SSTF bei einem Thread auf Windows sogar langsamer.
- Für CPU-Auslastung und mittlere RAM-Auslastung ergibt sich meist kein belastbarer Unterschied.
- Auch bei den RAM-Maxima lässt sich kein konsistenter Vorteil belegen. Bei vier Threads auf Windows ist SSTF zwar gegenüber FIFO_500 erkennbar günstiger, bei zehn Threads auf Windows aber nur gegenüber FIFO_50, nicht jedoch gegenüber FIFO_500.

4.4 WORK-STEALING MIT SSTF-PRIORISIERUNG

Hier werden die Ergebnisse der Messungen des Work-Stealing-Schedulers mit SSTF-Priorisierung vorgestellt. Das Ziel dieser Implementierung war es, die Vorteile des SSTF-Schedulers mit der Flexibilität und Lastverteilung des Work-Stealing-Ansatzes zu kombinieren und Lock-Contention zu reduzieren. Der Work-Stealing-Scheduler mit SSTF-Priorisierung wird primär mit dem SSTF-Scheduler verglichen, um die Auswirkungen des Work-Stealing-Ansatzes auf die Performance zu bewerten, da beide Scheduler die gleiche Priorisierungsstrategie verwenden.

Tabelle 4.8: Laufzeit-, CPU- und RAM-Metriken des Work-Stealing-Schedulers mit SSTF-Priorisierung für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte ± Standardabweichung aus $n = 20$ Läufen

Threads	OS	Runtime [s]	CPU [%]	RAM Mittel [MB]	RAM Max [MB]
1	Linux	660 ± 3.47	23.5 ± 0.5	1065 ± 17.6	1653 ± 30.4
4	Linux	294 ± 3.93	51.2 ± 0.927	1528 ± 31.7	2573 ± 63.1
10	Linux	220 ± 3.16	63.2 ± 0.829	1868 ± 47.7	3751 ± 144.9
1	Windows	2526 ± 10.2	21.0 ± 0	1389 ± 6.46	5991 ± 98.2
4	Windows	1145 ± 34.6	47.8 ± 1.26	2087 ± 54	7648 ± 293.5
10	Windows	747 ± 34.2	71.9 ± 3.37	2904 ± 243	11457 ± 884.0

In Tabelle 4.8 sind die Laufzeit, CPU-Auslastung und RAM-Auslastung für den Work-Stealing mit SSTF-Priorisierung aufgelistet. Insgesamt sinkt die Laufzeit mit steigender Thread-Zahl deutlich, unter Linux von rund 660 s auf 220 s und unter Windows von rund 2526 s auf 747 s. Parallel dazu steigt die CPU-Auslastung von etwa 24 % bzw. 21 % auf 63 % bzw. 72 %, die mittlere RAM-Auslastung von rund 1.1 GB auf 1.9 GB bzw. 2.9 GB. Gegenüber SSTF überlappen die Standardabweichungsintervalle von Laufzeit, CPU-Auslastung und mittlerer RAM-Auslastung in den meisten Konfigurationen. Ein belastbarer Unterschied zeigt sich bei der Laufzeit nur auf Linux mit vier Threads, wo Work-Stealing mit 294 ± 3.93 s langsamer ist als SSTF mit 273 ± 3.50 s. Bei der mittleren RAM-Auslastung ergibt sich nur auf Windows

mit vier Threads ein messbar geringerer Wert. Bei den RAM-Maximalwerten zeigt sich gegenüber SSTF nur bei zehn Threads auf Windows ein klarer Unterschied. Dort liegt Work-Stealing mit 11457 ± 884.0 MB über SSTF mit 9592 ± 641.6 MB.

Tabelle 4.9: Runtime CV sowie Queue Time, Execution Time und Anteil synchroner Tasks des Work-Stealing-Schedulers mit SSTF-Priorisierung für Linux und Windows bei einem, vier und zehn Threads, angegeben als Mittelwerte ± Standardabweichung aus $n = 20$ Läufen

Threads	OS	Runtime CV [%]	Queue [ms]	Exec [ms]	Sync Tasks [%]
1	Linux	0.53	46.26 ± 0.28	0.98 ± 0.01	99.95 ± 0.00
4	Linux	1.33	58.52 ± 1.09	0.91 ± 0.01	80.61 ± 1.21
10	Linux	1.44	84.72 ± 1.28	1.10 ± 0.01	27.09 ± 1.63
1	Windows	0.40	65.91 ± 0.40	1.09 ± 0.00	99.83 ± 0.00
4	Windows	3.03	90.46 ± 2.76	1.11 ± 0.02	53.36 ± 4.64
10	Windows	4.58	97.23 ± 3.15	1.44 ± 0.03	35.31 ± 3.25

In Tabelle 4.9 sind die Runtime CV, Queue Time, Execution Time und der Anteil der synchronen Tasks für den Work-Stealing mit SSTF-Priorisierung aufgelistet. Die Queue Time steigt mit der Thread-Zahl von 46 ms auf 85 ms unter Linux und von 66 ms auf 97 ms unter Windows. Die Execution Time bleibt mit etwa 0.9 bis 1.4 ms vergleichsweise stabil. Beim Anteil synchron ausgeführter Tasks zeigt sich kein einheitliches Bild: Unter Linux liegt Work-Stealing bei vier Threads mit 80.61 ± 1.21 % klar über SSTF, bei zehn Threads mit 27.09 ± 1.63 % dagegen klar darunter. Auf Windows überlappen die Standardabweichungsintervalle bei vier und zehn Threads. Gegenüber SSTF ist die Queue Time in allen Konfigurationen durchgängig höher. Im Vergleich zu FIFO_50 ist sie ebenfalls durchgehend höher, gegenüber FIFO_500 dagegen durchgehend geringer.

4.4.1 *Diskussion der Work-Stealing-Ergebnisse*

STABILITÄT: Die Standardabweichung der CPU-Auslastung ist bei einem Thread sehr gering und steigt mit der Thread-Anzahl auf bis zu 0.83 unter Linux und 3.37 unter Windows. Zugleich wächst auch der Runtime CV von rund 0.5 auf bis zu 1.44 unter Linux und 4.58 unter Windows. Damit zeigt Work-Stealing ebenfalls eine höhere Streuung unter Last.

LAUFZEIT: Gegenüber SSTF lässt sich für die Laufzeit nur auf Linux mit vier Threads ein belastbarer Unterschied zeigen. Dort liegt Work-Stealing bei 294 ± 3.93 s, SSTF dagegen bei 273 ± 3.50 s. In allen übrigen Konfigurationen überlappen die Standardabweichungsintervalle.

WAITING TIME UND BURST TIME: Für die Queue Time zeigt sich dagegen ein konsistenter Nachteil gegenüber SSTF: In allen Konfigurationen liegt

Work-Stealing klar höher. Besonders deutlich ist dies unter Windows bei vier Threads mit 90.46 ± 2.76 ms gegenüber 45.06 ± 1.34 ms für SSTF sowie 46.89 ± 1.04 ms für FIFO_50. Gegenüber FIFO_500 ist Work-Stealing dagegen klar günstiger, bei vier Threads auf Windows etwa mit 90.46 ± 2.76 ms statt 481.52 ± 13.97 ms. Eine angepasste Implementierung könnte diese Wartezeit weiter reduzieren, etwa durch eine Begrenzung der Warteschlangengröße pro Thread, damit Tasks nicht zu lange in den Queues verbleiben, bevor sie ausgeführt werden.

SYNCHRON AUSGEFÜHRTE TASKS UND RAM-AUSLASTUNG: Beim Anteil synchron ausgeführter Tasks ergibt sich kein einheitlicher Vorteil: Bei vier Threads auf Linux liegt Work-Stealing mit 80.61 ± 1.21 % klar über SSTF mit 44.09 ± 2.44 %, bei zehn Threads auf Linux dagegen mit 27.09 ± 1.63 % klar darunter. Auf Windows lässt sich bei vier und zehn Threads kein Unterschied zu SSTF belegen. Für die mittlere RAM-Auslastung zeigt sich nur bei vier Threads auf Windows ein messbar niedrigerer Wert als bei SSTF, nämlich 2087 ± 54 MB gegenüber 2209 ± 60.6 MB. Die RAM-Maximalwerte liegen bei zehn Threads auf Windows dagegen mit 11457 ± 884.0 MB klar über SSTF mit 9592 ± 641.6 MB.

VORTEILE UND NACHTEILE DES WORK-STEALING-SCHEDULERS MIT SSTF-PRIORISIERUNG IM VERGLEICH ZUM SSTF-SCHEDULER:

- Vorteile:
  - Bei vier Threads auf Windows ist die mittlere RAM-Auslastung erkennbar geringer als bei SSTF.
  - Bei zehn Threads auf Linux ist der Anteil synchron ausgeführter Tasks messbar geringer als bei SSTF.
- Nachteile:
  - Die Queue Time ist gegenüber SSTF in allen Konfigurationen klar höher.
  - Bei vier Threads auf Linux ist auch die Laufzeit messbar schlechter als bei SSTF.
  - Auf Windows lässt sich für den Anteil synchroner Tasks kein belastbarer Vorteil gegenüber SSTF zeigen. Bei zehn Threads ist der RAM-Maximalwert sogar klar höher als bei SSTF.

4.5 DISKUSSION DER ERGEBNISSE

Die Ergebnisse der Messungen zeigen, dass die verschiedenen Scheduling-Algorithmen unterschiedliche Auswirkungen auf die Performance von verschiedenen Metriken haben, wie Laufzeit, CPU-Auslastung, RAM-Auslastung, Wartezeit von Tasks und Anzahl der synchron ausgeführten Tasks. Für die Nutzer des Scanners ist die Laufzeit von oberster Priorität, möglichst in Kombination mit einer geringen Ressourcen-Auslastung. Ein Scanner mit

responsivem Verhalten und entsprechend niedriger Wartezeit von Tasks, ist ebenfalls wünschenswert, sollte allerdings nicht auf Kosten einer höheren Laufzeit implementiert werden. Die Anzahl der synchron ausgeführten Tasks sollte möglichst gering sein, um die Vorteile der taskbasierten Ausführung zu nutzen, hat für den Nutzer allerdings keine direkte Auswirkung und somit eine niedrigere Priorität.

Die hier verwendeten Balkendiagramme zeigen die Mittelwerte von jeweils 20 Messungen der Laufzeit, CPU-Auslastung und RAM-Auslastung der verschiedenen Scheduling-Algorithmen bei verschiedenen Thread-Anzahlen auf Windows und Linux. Die Standardabweichungen sind an den Balken mit angegeben.

Bei einem Thread zeigen die CPU-Auslastung und, zwischen den neuen Scheduler-Varianten, auch die mittlere RAM-Auslastung keine konsistenten belastbaren Unterschiede, siehe Abbildung 4.3, 4.4 und 4.5. Gegenüber der Referenz liegt die mittlere RAM-Auslastung auf Windows jedoch bei allen neuen Scheduler-Varianten erkennbar höher. Auch für die Laufzeit ergibt sich kein robuster Vorteil einer neuen Scheduler-Variante: Auf Linux sind alle neuen Scheduler gegenüber der Referenz messbar langsamer, auf Windows unterscheiden sich Referenz und neue Scheduler dagegen nicht belastbar. Innerhalb der neuen Scheduler treten zwar einzelne erkennbare Unterschiede auf, sie bilden jedoch kein konsistentes Muster zugunsten einer Variante. Dies passt zur Erwartung, dass Scheduling-Vorteile erst beim Einsatz mehrerer Threads sichtbar werden.

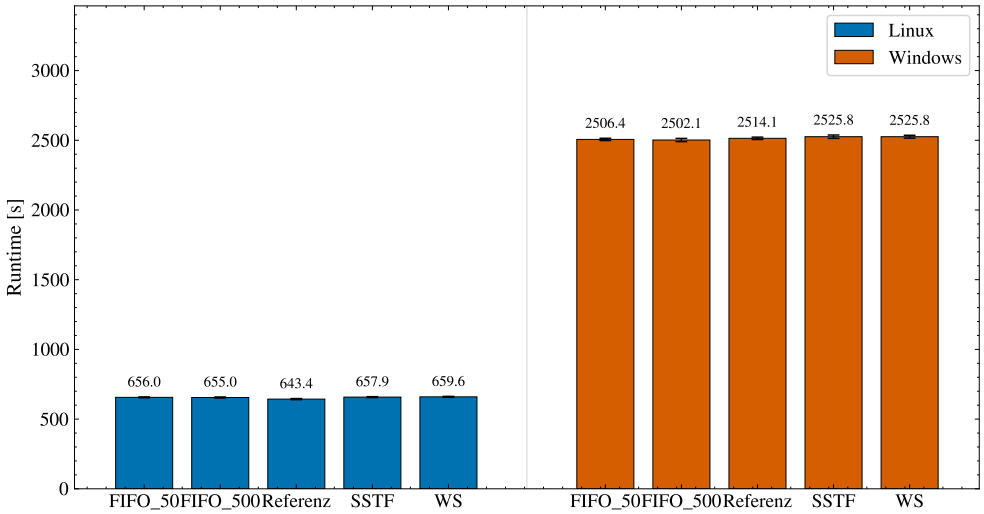

Abbildung 4.3: Vergleich der Laufzeiten von Referenz, FIFO_50, FIFO_500, SSTF und Work-Stealing auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

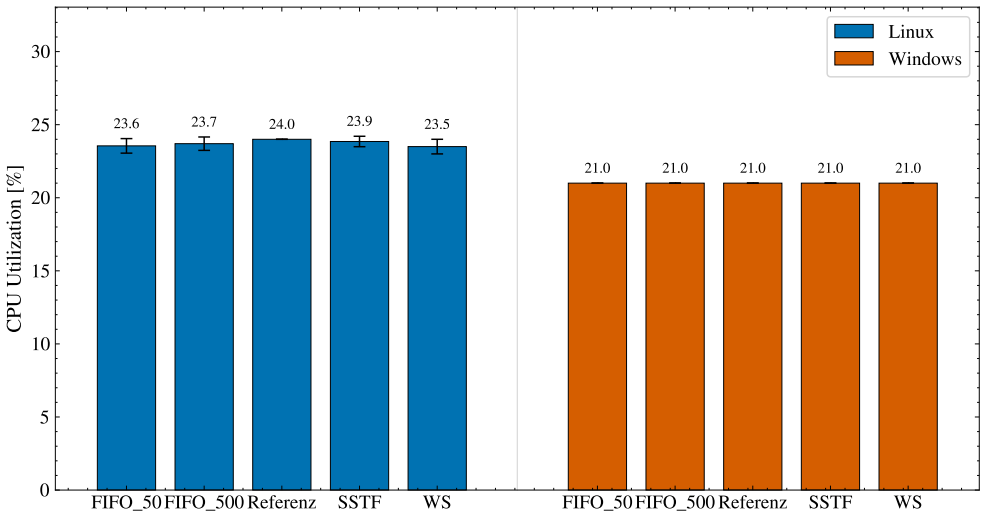

Abbildung 4.4: Vergleich der CPU-Auslastung von Referenz, FIFO_50, FIFO_500, SSTF und Work-Stealing auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

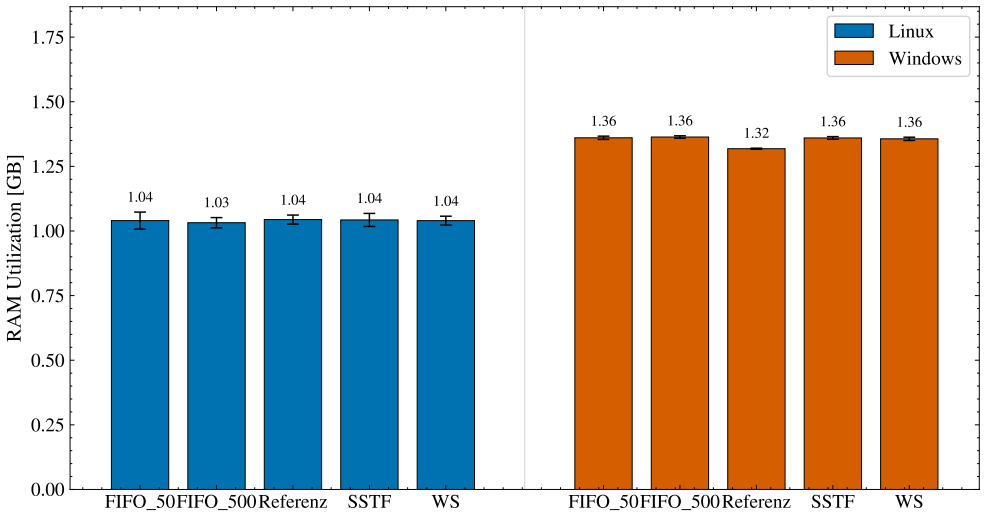

Abbildung 4.5: Vergleich der RAM-Auslastung von Referenz, FIFO_50, FIFO_500, SSTF und Work-Stealing auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

Bei vier Threads zeigen sich auf Windows für alle neuen Scheduler klar geringere Laufzeiten als bei der Referenz (1213.83 $\pm$ 26.95 s): FIFO_50 mit 1126.42 $\pm$ 23.26 s, FIFO_500 mit 1112.04 $\pm$ 29.19 s, SSTF mit 1123.66 $\pm$ 24.39 s und Work-Stealing mit 1144.78 $\pm$ 34.65 s. Auf Linux gilt dies nicht, dort überlappt nur FIFO_50 mit der Referenz, während FIFO_500, SSTF und Work-Stealing messbar langsamer als die Referenz sind.

Zwischen FIFO_50, FIFO_500, SSTF und Work-Stealing lässt sich bei der Laufzeit auf Windows kein belastbarer Unterschied erkennen. Gleiches gilt für die CPU-Auslastung. Bei der mittleren RAM-Auslastung überlappen die Standardabweichungsintervalle ebenfalls weitgehend, nur zwischen SSTF und Work-Stealing zeigt sich ein erkennbarer Unterschied zugunsten von Work-Stealing, siehe Abbildung 4.6, 4.7 und 4.8. Die internen Metriken trennen die Verfahren jedoch deutlicher: FIFO_500 weist mit 481.52 ± 13.97 ms eine klar höhere Queue Time als FIFO_50, SSTF und Work-Stealing auf. Zwischen FIFO_50 (46.89 ± 1.04 ms) und SSTF (45.06 ± 1.34 ms) lässt sich dagegen kein Unterschied belegen. Beim Anteil synchron ausgeführter Tasks liegen SSTF und Work-Stealing mit 53.51 ± 4.05 % bzw. 53.36 ± 4.64 % messbar unter FIFO_50 und FIFO_500, unterscheiden sich untereinander aber nicht belastbar. Die höchste RAM-Spitze zeigt bei vier Threads auf Windows FIFO_500 mit 10782 ± 932.2 MB. Dieser Wert liegt klar über FIFO_50, SSTF und Work-Stealing.

Nach den festgelegten Prioritäten ist SSTF damit auch bei zehn Threads die am besten begründete Gesamtwahl: Gegenüber FIFO_500 und Work-Stealing zeigt SSTF keinen belegbaren Laufzeitnachteil, aber eine messbar geringere Queue Time. Gegenüber FIFO_50 ist die Laufzeit ebenfalls nicht unterscheidbar, während der Anteil synchron ausgeführter Tasks mit 53.51 ± 4.05 % gegenüber 63.84 ± 5.46 % erkennbar geringer ist. Work-Stealing weist zwar eine messbar geringere mittlere RAM-Auslastung als SSTF auf, erkauft dies aber mit einer deutlich höheren Queue Time.

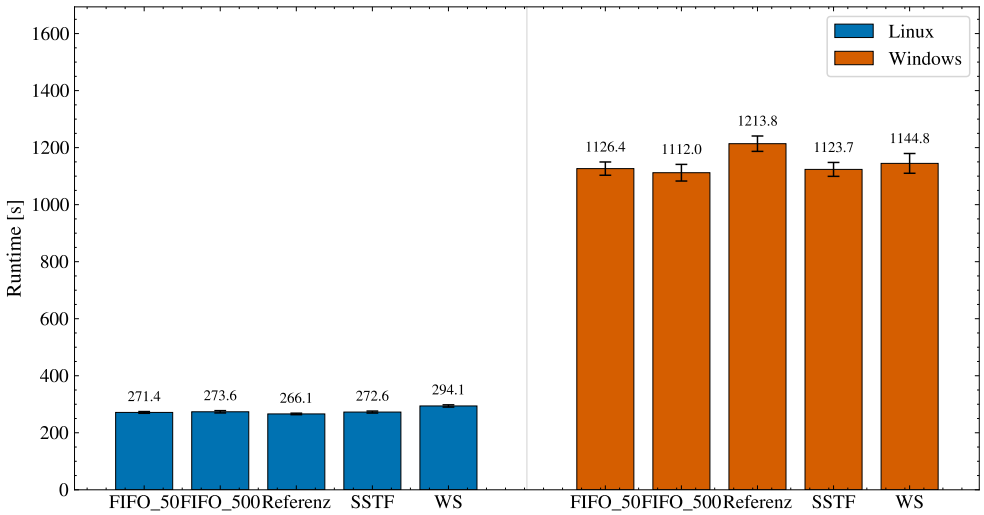

Abbildung 4.6: Vergleich der Laufzeiten von Referenz, FIFO_50, FIFO_500, SSTF und Work-Stealing auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

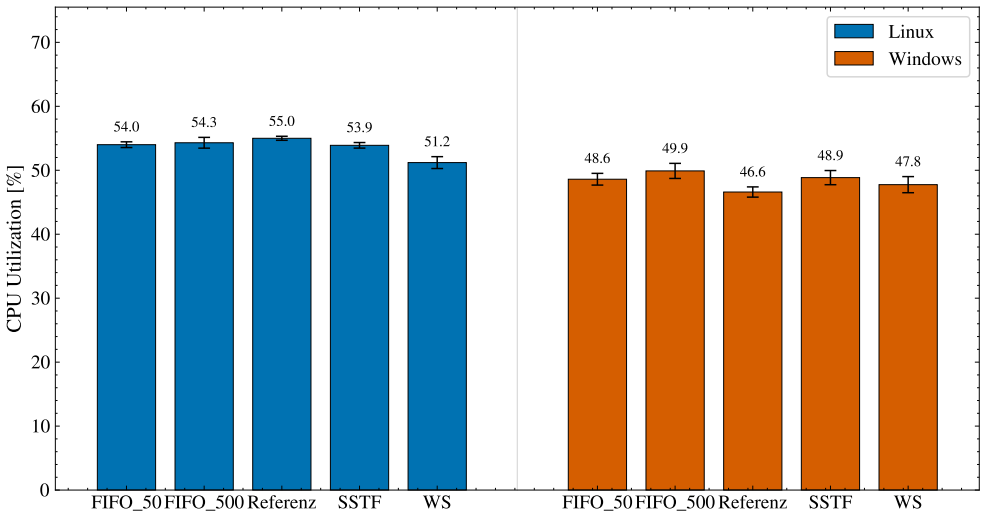

Abbildung 4.7: Vergleich der CPU-Auslastung von Referenz, FIFO_50, FIFO_500, SSTF und Work-Stealing auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

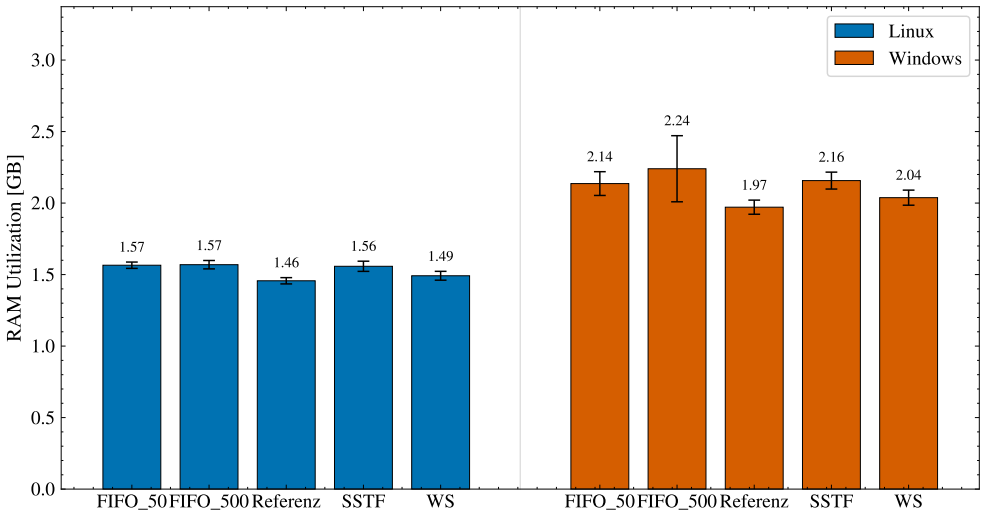

Abbildung 4.8: Vergleich der RAM-Auslastung von Referenz, FIFO_50, FIFO_500, SSTF und Work-Stealing auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

Bei zehn Threads werden die Unterschiede zwischen den Scheduling-Algorithmen weiter sichtbar, siehe Abbildung 4.9, 4.10 und 4.11. Auf Windows zeigen alle neuen Scheduler weiterhin einen belegbaren Laufzeitvorteil gegenüber der Referenz (825.78 ± 20.53 s): FIFO_50 mit 750.65 ± 25.36 s, FIFO_500 mit 740.05 ± 31.51 s, SSTF mit 752.41 ± 25.56 s und Work-Stealing mit 747.43 ± 34.23 s. Auf Linux überlappen die Laufzeitintervalle aller neuen Scheduler mit der Referenz; ein solcher Vorteil ist dort nicht klar erkennbar.

Die internen Metriken differenzieren die Verfahren auch hier stärker als die Laufzeit. SSTF und Work-Stealing reduzieren den Anteil synchron ausgeführter Tasks gegenüber FIFO_50 und FIFO_500 messbar, unterscheiden sich untereinander auf Windows jedoch nicht belastbar. Für die Laufzeit auf Windows lässt sich zwischen Work-Stealing (747.43 ± 34.23 s), SSTF (752.41 ± 25.56 s) und den FIFO-Varianten kein Unterschied belegen, da sich die Intervalle überlappen. Die Queue Time liegt bei Work-Stealing mit 97.23 ± 3.15 ms klar über SSTF und FIFO_50, aber klar unter FIFO_500. Bei der RAM-Spitze ist SSTF auf Windows mit 9592 ± 641.6 MB gegenüber Work-Stealing mit 11457 ± 884.0 MB erkennbar geringer. Gegenüber FIFO_500 ist der Unterschied dagegen nicht belastbar.

Nach den festgelegten Prioritäten ist SSTF damit auch bei zehn Threads die am besten begründete Gesamtwahl: Es zeigt gegenüber FIFO_50, FIFO_500 und Work-Stealing keinen belegbaren Laufzeitnachteil, reduziert aber den Anteil synchron ausgeführter Tasks gegenüber beiden FIFO-Varianten erkennbar und weist gegenüber Work-Stealing eine deutlich geringere Queue Time auf. Zusätzlich liegt der RAM-Maximalwert gegenüber FIFO_50 und Work-Stealing erkennbar niedriger, gegenüber FIFO_500 jedoch nicht belastbar.

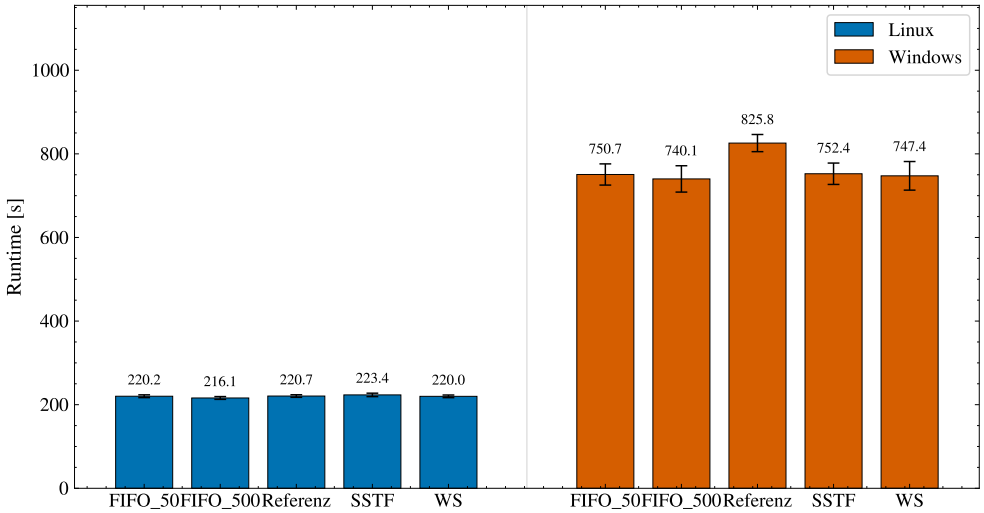

Abbildung 4.9: Vergleich der Laufzeiten von Referenz, FIFO_50, FIFO_500, SSTF und Work-Stealing auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

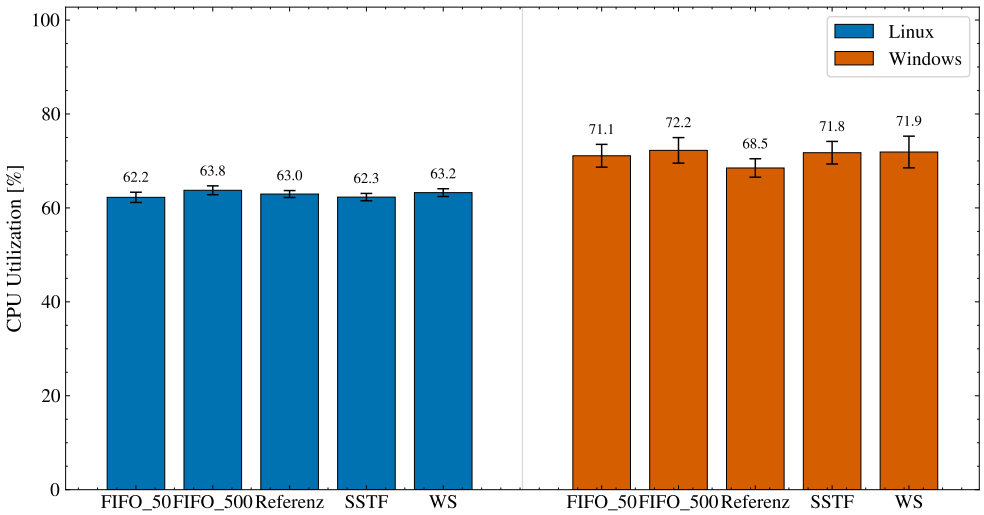

Abbildung 4.10: Vergleich der CPU-Auslastung von Referenz, FIFO_50, FIFO_500, SSTF und Work-Stealing auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

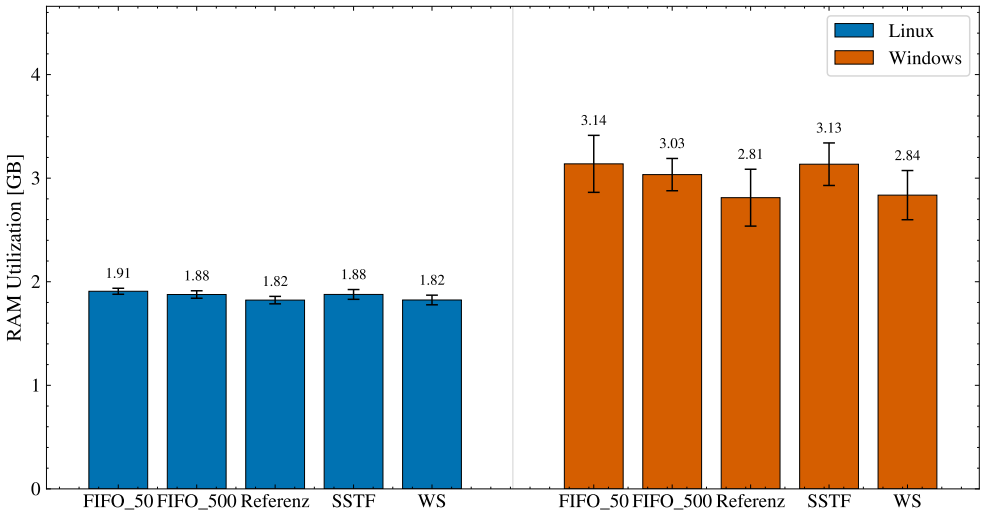

Abbildung 4.11: Vergleich der RAM-Auslastung von Referenz, FIFO_50, FIFO_500, SSTF und Work-Stealing auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

Nach unserer Definition von Performance und im Hinblick auf die festgelegten Prioritäten lässt sich SSTF als die am besten begründete Scheduler-Strategie identifizieren. Unter Last ab vier Threads zeigt sich gegenüber den FIFO-Varianten kein belegbarer Laufzeitnachteil, während der Anteil synchron ausgeführter Tasks bei vier und zehn Threads erkennbar geringer ist. Gegenüber Work-Stealing besteht ebenfalls kein belegbarer Laufzeitnachteil, bei zugleich deutlich geringerer Queue Time. Zudem ist der RAM-Maximalwert bei zehn Threads auf Windows gegenüber FIFO_50 und Work-

Stealing erkennbar geringer, während gegenüber FIFO_500 kein Unterschied belastbar erkennbar ist.
Weitere Tests mit optimierten Implementierungen und auf verschiedenen Systemen sind jedoch empfehlenswert.

5 FAZIT

Diese Arbeit hatte das Ziel, die Performance eines APT-Scanners durch die Einführung eines taskbasierten Schedulers zu verbessern und die Wirkung verschiedener Scheduling-Strategien unter kontrollierten Bedingungen experimentell zu bewerten. Dafür wurde der Scanner architektonisch von einer modulgebundenen, rekursiv-synchronen und objektbasierten Verarbeitung auf ein entkoppeltes Service-Modell mit zentralem Scheduler umgestellt. Auf dieser Basis wurden drei Scheduling-Strategien systematisch untersucht: FIFO, SSTF sowie Work-Stealing mit SSTF-Priorisierung.

Im Vergleich zum alten Scheduler zeigt sich ein differenziertes Bild. Die neuen Scheduler liefern auf Windows bei paralleler Ausführung messbare Laufzeitverbesserungen. Bei vier Threads wurde die Runtime gegenüber der Referenz je nach Verfahren um 5,69 % bis 8,39 % reduziert, bei zehn Threads um 8,89 % bis 10,38 %. Die Ergebnisse zeigen zugleich, dass die CPU-Auslastung in den Multithread-Konfigurationen gesteigert werden konnte, ohne dass die Streuung der Messungen grundsätzlich ihre Aussagekraft beeinträchtigt. Unter Linux zeigt sich dagegen kein konsistenter Laufzeitvorteil: Bei einem Thread sind alle neuen Scheduler langsamer als die Referenz, bei vier Threads gilt dies für FIFO_500, SSTF und Work-Stealing, und bei zehn Threads lässt sich kein nachweisbarer Unterschied zeigen.

Die Arbeit untersuchte erstens, in welchem Ausmaß alternative Scheduling-Strategien die Laufzeit des Scanners verbessern können, und zweitens, wie sich diese Strategien auf CPU- und RAM-Auslastung sowie die Waiting Time auswirken. Auf Basis der Messergebnisse lassen sich beide Fragen präzise beantworten. Hinsichtlich der Laufzeitverbesserung zeigen die getesteten Scheduling-Algorithmen einen deutlichen Vorteil unter Windows bei paralleler Ausführung in den Konfigurationen mit vier und zehn Threads. Unter Linux lässt sich dagegen kein belastbarer Laufzeitvorteil ableiten. Bezüglich der Auswirkungen auf CPU- und RAM-Auslastung sowie Waiting Time ergibt sich ein differenziertes Bild: Die CPU-Auslastung steigt mit der Threadzahl in allen Varianten. Große Warteschlangen erhöhen die Queue Time konsistent und führen in einzelnen Konfigurationen auch zu höheren RAM-Spitzen, zeigen aber keinen belastbaren Vorteil bei Laufzeit, CPU-Auslastung oder mittlerer RAM-Auslastung. SSTF reduziert die Queue Time gegenüber FIFO_500 in allen Konfigurationen und senkt bei vier und zehn Threads den Anteil synchron ausgeführter Tasks gegenüber beiden FIFO-Varianten nachweisbar. Work-Stealing mit SSTF zeigt in der vorliegenden Implementierung keinen konsistenten Vorteil gegenüber SSTF, da die Queue Time in allen Konfigu-

rationen höher ist und bei zehn Threads auf Windows auch ein höherer RAM-Maximalwert auftritt.

Unter den gewählten Prioritäten (Laufzeit vor Ressourcenverbrauch, dahinter erst die Responsivität) ergibt sich SSTF als beste Gesamtlösung. Unter Last ab vier Threads zeigt SSTF keinen belegbaren Laufzeitnachteil gegenüber FIFO_50, FIFO_500 und Work-Stealing, reduziert gegenüber FIFO_500 konsistent die Queue Time und senkt bei vier und zehn Threads den Anteil synchron ausgeführter Tasks gegenüber beiden FIFO-Varianten nachweisbar.

Bei der Interpretation der Ergebnisse sind folgende Punkte zu beachten:
Erstens sind die Laufzeitgewinne plattformabhängig und unter Linux nicht konsistent feststellbar.
Zweitens hängt die Wirksamkeit von SSTF von der Qualität der Prioritätsheuristik ab. Für das Testsystem, welches mithilfe von Snapshots vor jedem Scan zurückgesetzt wurde, war eine statische Priorisierung ausreichend, eine adaptive Lösung ist aber für den produktiven Einsatz empfehlenswert.
Drittens besteht beim Work-Stealing-Ansatz mit SSTF-Priorisierung in der aktuellen Variante Optimierungspotenzial bei den Warteschlangengrößen. Es ist möglich, dass durch gezielte Anpassungen Performancegewinne erzielt werden können.

Diese Arbeit zeigt, dass der Architekturwechsel zu einem zentralen taskbasierten Scheduling für den Scanner möglich ist und unter Windows bei paralleler Last reproduzierbare Leistungsgewinne liefern kann. Die entwickelten Verfahren stellen eine belastbare Grundlage für weiterführende Optimierungen dar, insbesondere bei Lastprofilen mit hoher Parallelität.

6 AUSBLICK

Da die Testumgebung für diese Arbeit auf einem virtuellen Testsystem mit kontrollierten Bedingungen basiert, sind die Ergebnisse nur begrenzt auf reale Einsatzszenarien übertragbar. Es sollten weiterführende Untersuchungen unter realen Bedingungen mit produktionsnahen Systemen durchgeführt werden. Somit könnten auch durch die Testumgebung bedingte systematische Fehler ausgeschlossen werden, um ein realistisches Bild außerhalb von Laborbedingungen zu gewinnen.

Es gibt viele Möglichkeiten, die in dieser Arbeit entwickelten Verfahren zu optimieren und zu erweitern, welche aber aufgrund der begrenzten Ressourcen nicht umgesetzt werden konnten. Hier folgen einige konkrete Ansätze, die sich aus den Ergebnissen und Beobachtungen der vorliegenden Arbeit ableiten lassen:

Die Implementierung des Work-Stealing-Ansatzes mit SSTF-Priorisierung zeigte in der vorliegenden Variante Optimierungsbedarf im Queue-Management und in der Speichercharakteristik. Es ist möglich, dass durch gezielte Anpassungen Performancegewinne erzielt werden können. Die Warteschlangenlängen aller Scheduler-Varianten können weiter optimiert werden, um die Queue-Zeiten zu reduzieren und die durchschnittliche und maximale RAM-Utilization zu senken.

Die Prioritätsheuristik von SSTF kann weiter verbessert werden. Statt einer statischen Implementierung könnte eine adaptive Lösung implementiert werden, welche dynamisch die Prioritäten anpasst.

Backpressure-Mechanismen könnten implementiert werden, um die Anzahl der Fallbacks zu reduzieren. Beispielsweise könnte der Modulthread zeitweise pausiert werden, wenn die Warteschlangen zu voll werden oder die RAM-Auslastung zu hoch ist.

Im Bereich der Priority-Based-Scheduler könnten weitere Strategien untersucht werden, um die Laufzeit weiter zu optimieren, ohne die Responsivität zu stark zu beeinträchtigen. Die Anwendung von Queueing-Theorie kann helfen, die Warteschlangen besser zu verstehen und zu optimieren, um die Performance weiter zu verbessern.

Teil II

APPENDIX

A

ANHANG

A.1 REFERENZMESSUNG (ALTER SCHEDULER)

A.1.1 *Ein Thread*

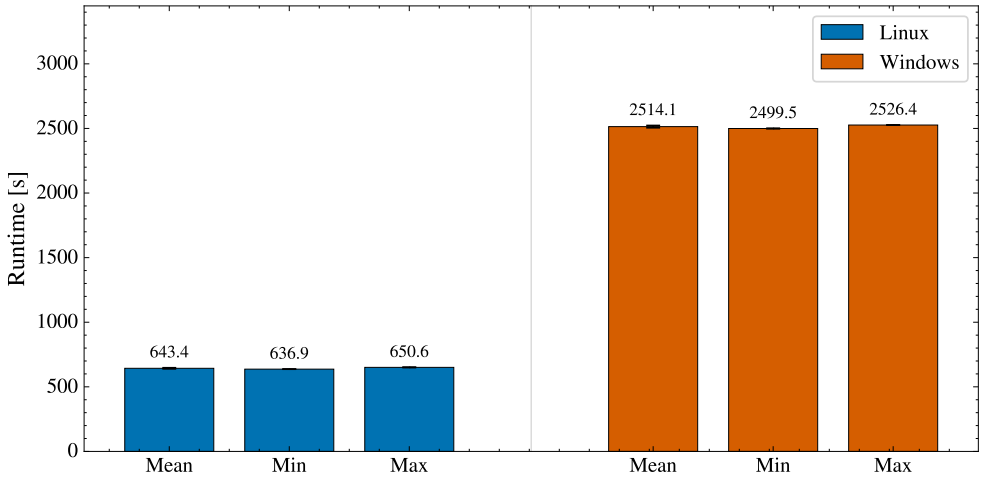

Abbildung A.1: Laufzeiten der Referenzmessung auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

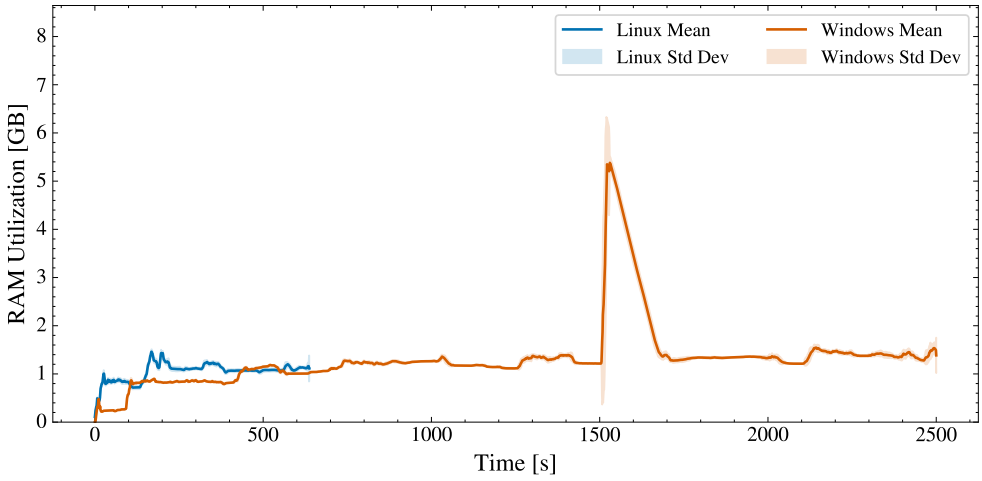

Abbildung A.2: Zeitlicher Verlauf der RAM-Auslastung der Referenzmessung auf Linux und Windows bei einem Thread

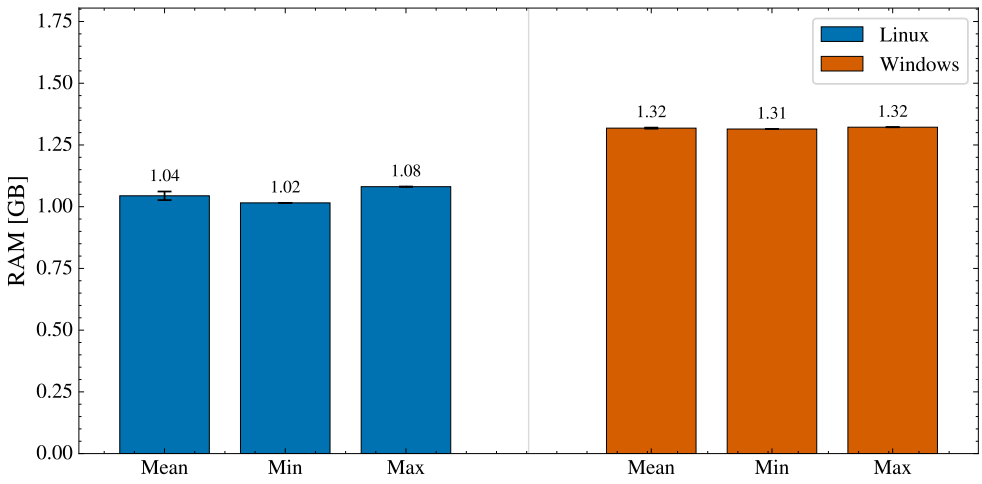

Abbildung A.3: RAM-Auslastung der Referenzmessung auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

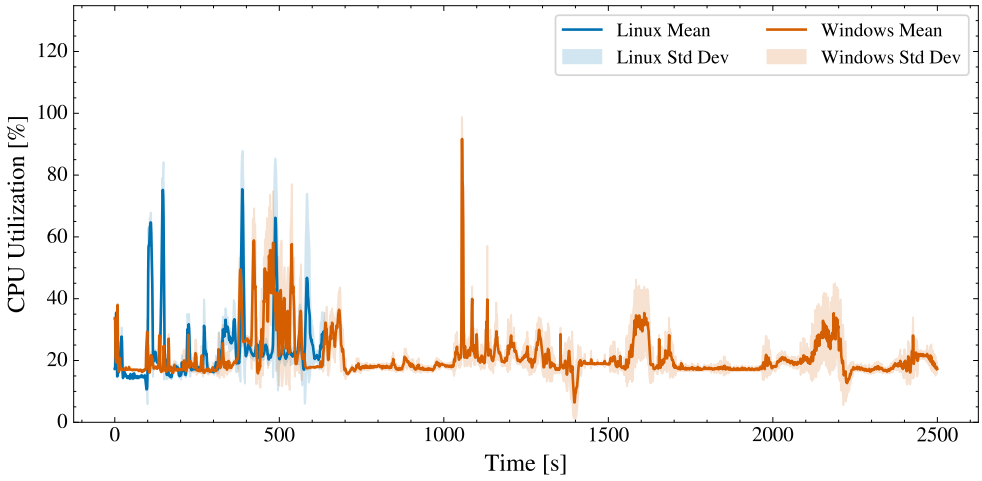

Abbildung A.4: Zeitlicher Verlauf der CPU-Auslastung der Referenzmessung auf Linux und Windows bei einem Thread

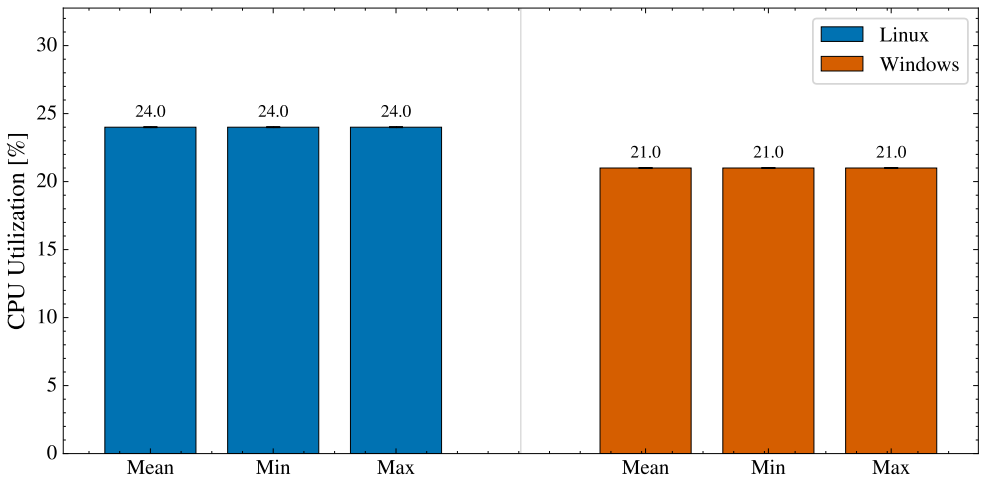

Abbildung A.5: CPU-Auslastung der Referenzmessung auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

A.1.2 *Vier Threads*

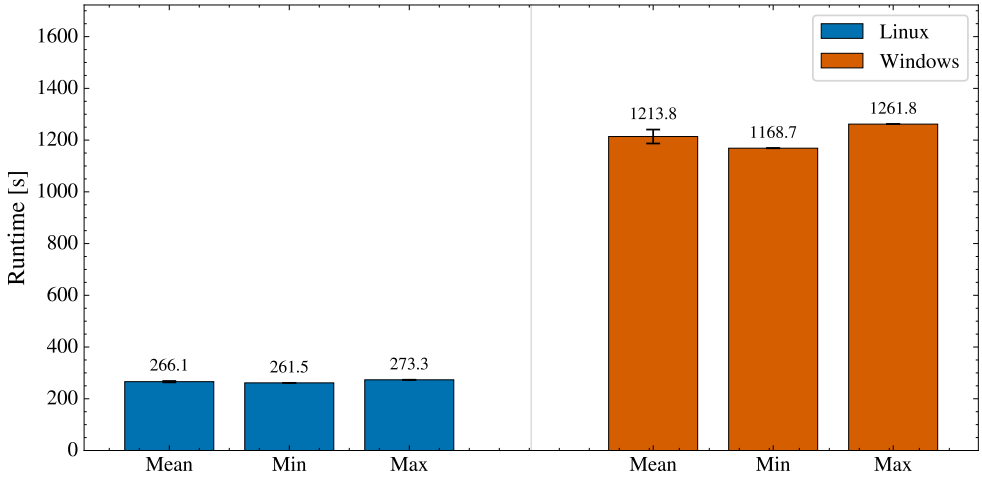

Abbildung A.6: Laufzeiten der Referenzmessung auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

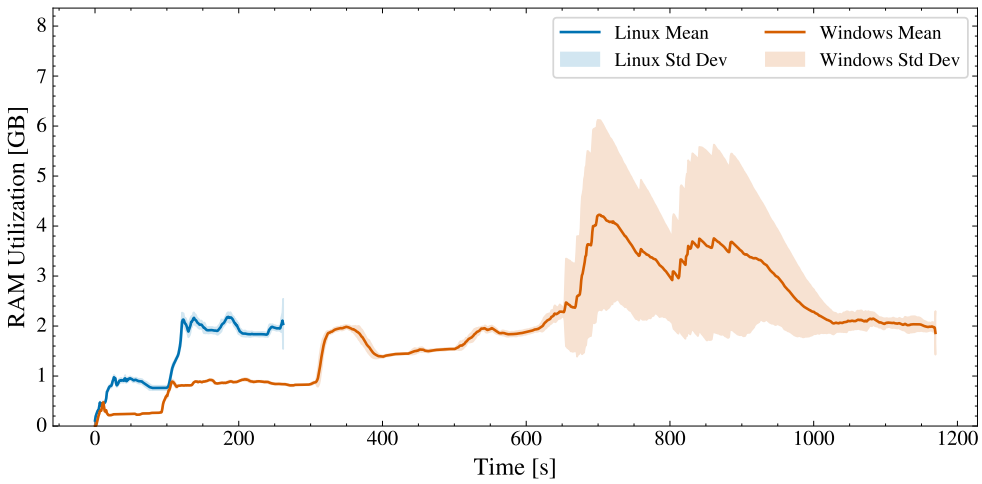

Abbildung A.7: Zeitlicher Verlauf der RAM-Auslastung der Referenzmessung auf Linux und Windows bei vier Threads

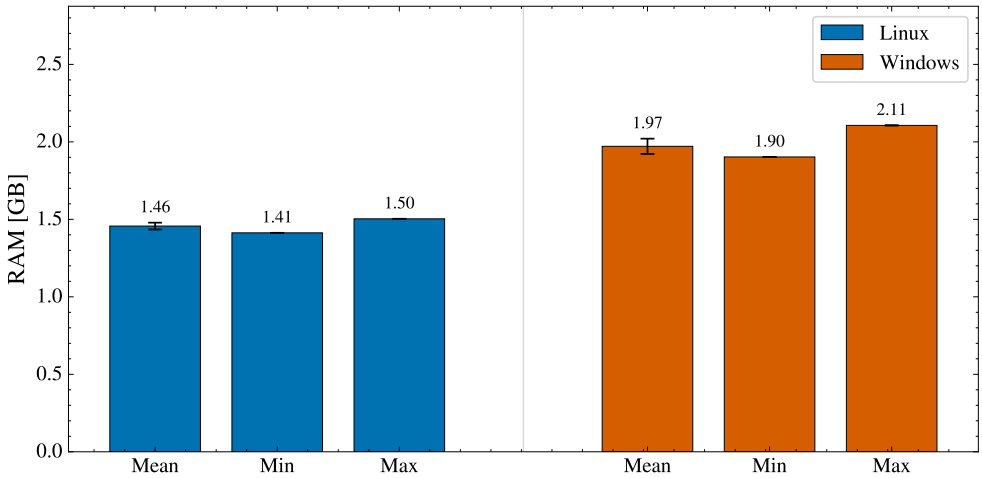

Abbildung A.8: RAM-Auslastung der Referenzmessung auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

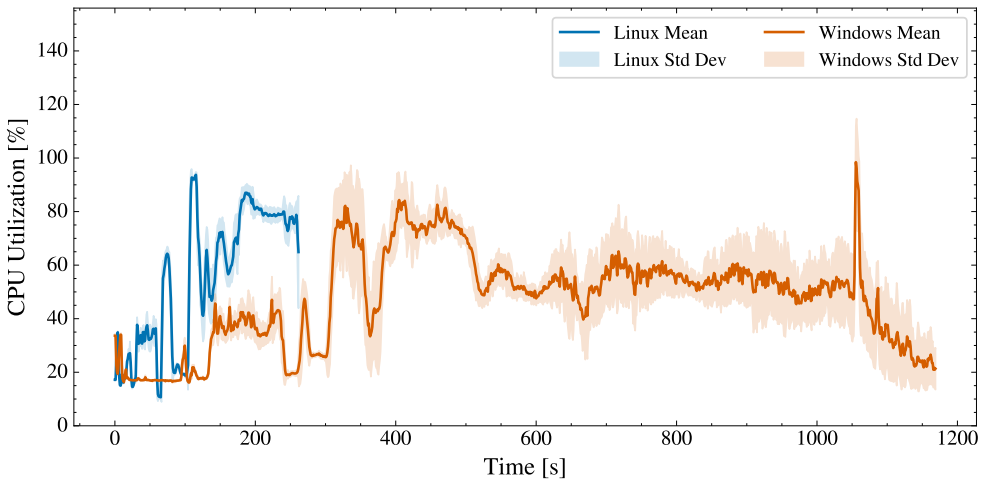

Abbildung A.9: Zeitlicher Verlauf der CPU-Auslastung der Referenzmessung auf Linux und Windows bei vier Threads

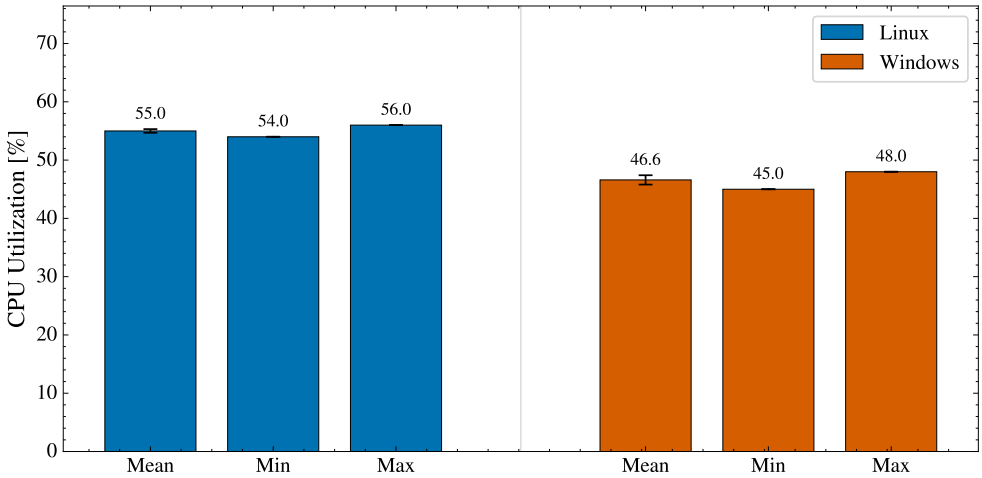

Abbildung A.10: CPU-Auslastung der Referenzmessung auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

A.1.3 *Zehn Threads*

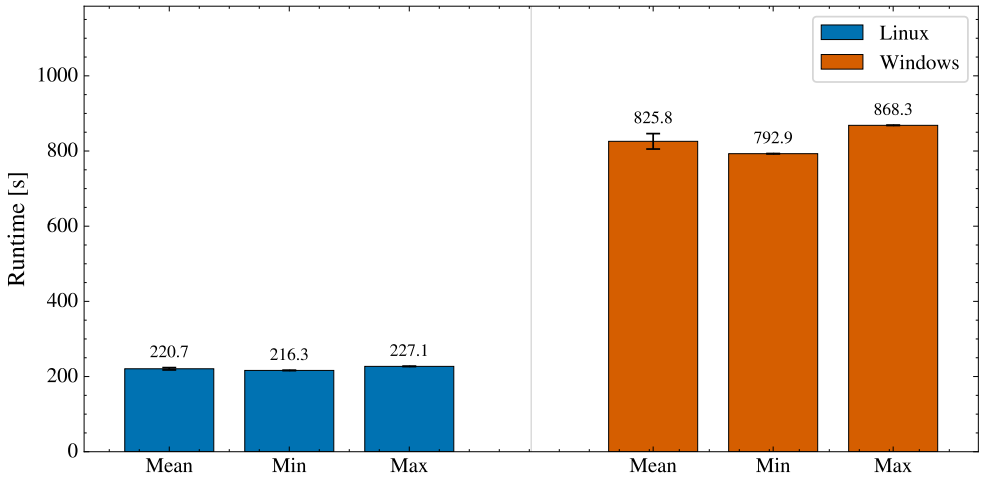

Abbildung A.11: Laufzeiten der Referenzmessung auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

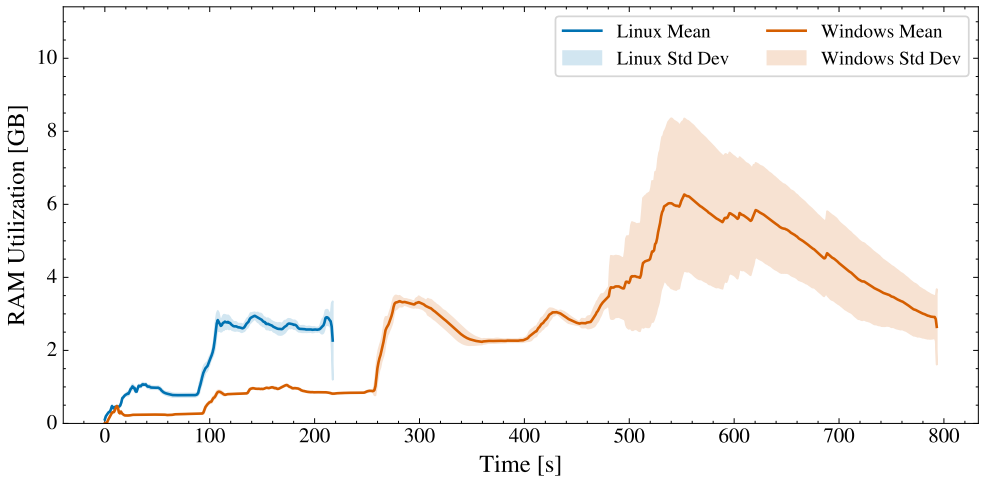

Abbildung A.12: Zeitlicher Verlauf der RAM-Auslastung der Referenzmessung auf Linux und Windows bei zehn Threads

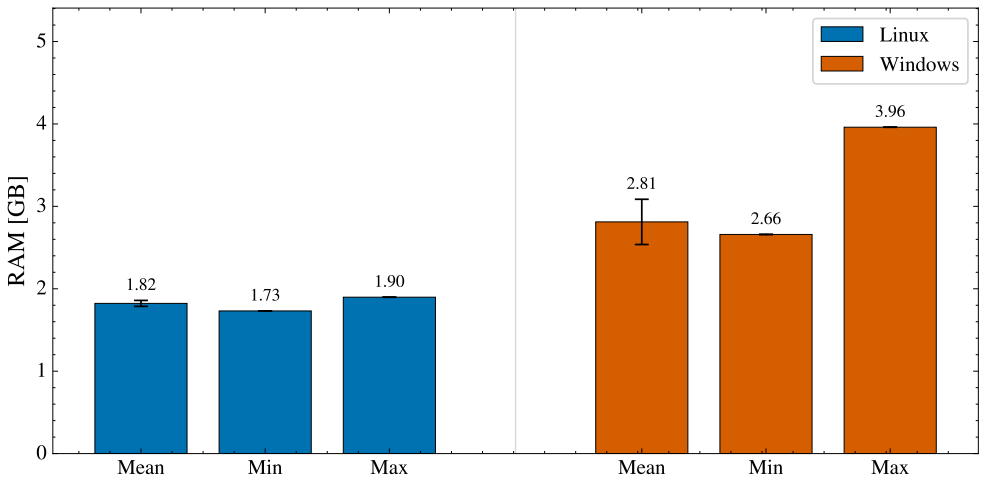

Abbildung A.13: RAM-Auslastung der Referenzmessung auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

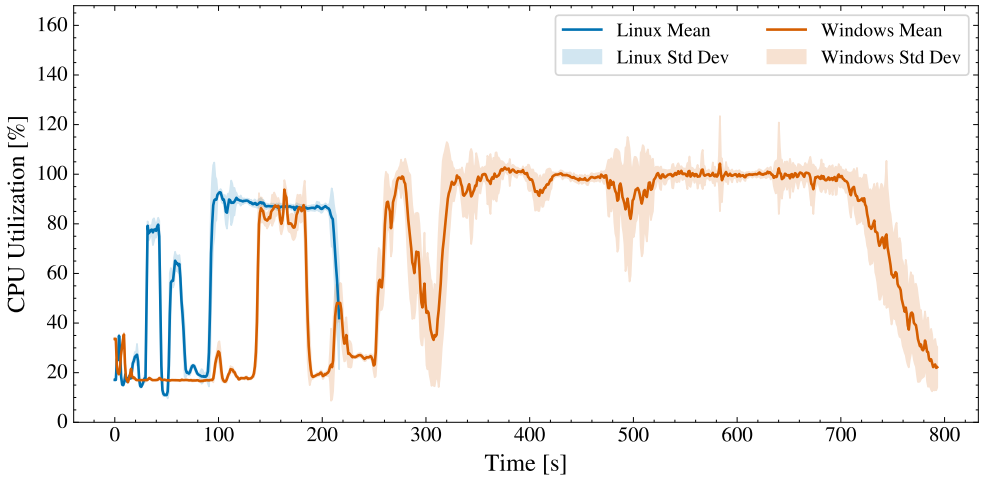

Abbildung A.14: Zeitlicher Verlauf der CPU-Auslastung der Referenzmessung auf Linux und Windows bei zehn Threads

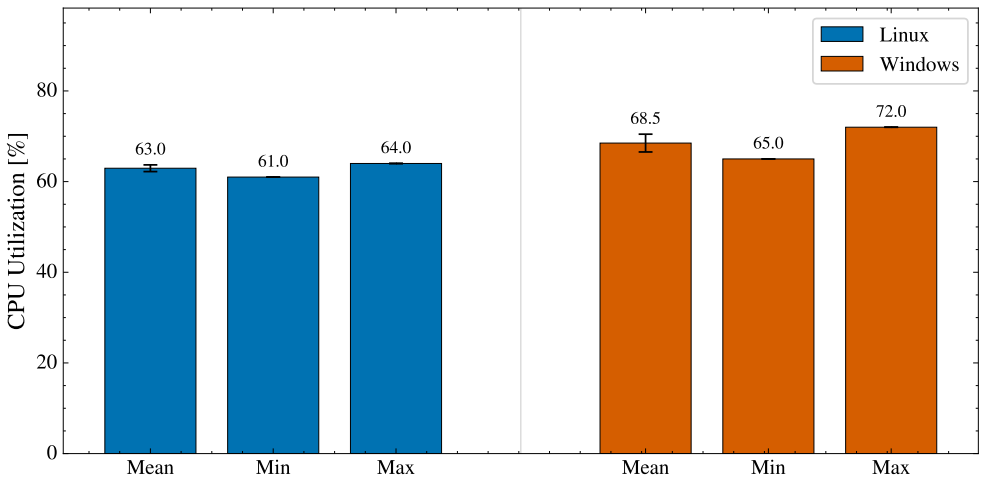

Abbildung A.15: CPU-Auslastung der Referenzmessung auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

A.1.4 *Vergleich*

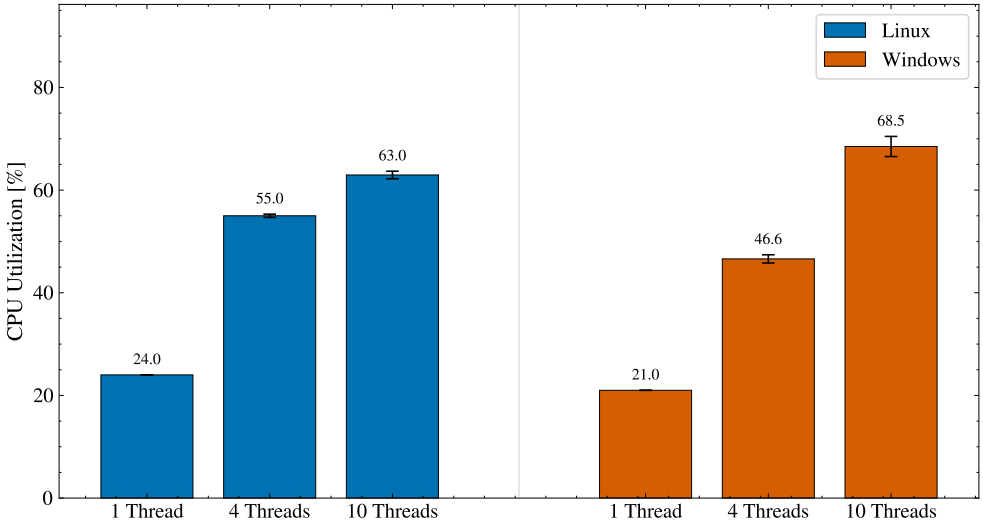

Abbildung A.16: Vergleich der CPU-Auslastung der Referenzmessung auf Linux und Windows über ein, vier und zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

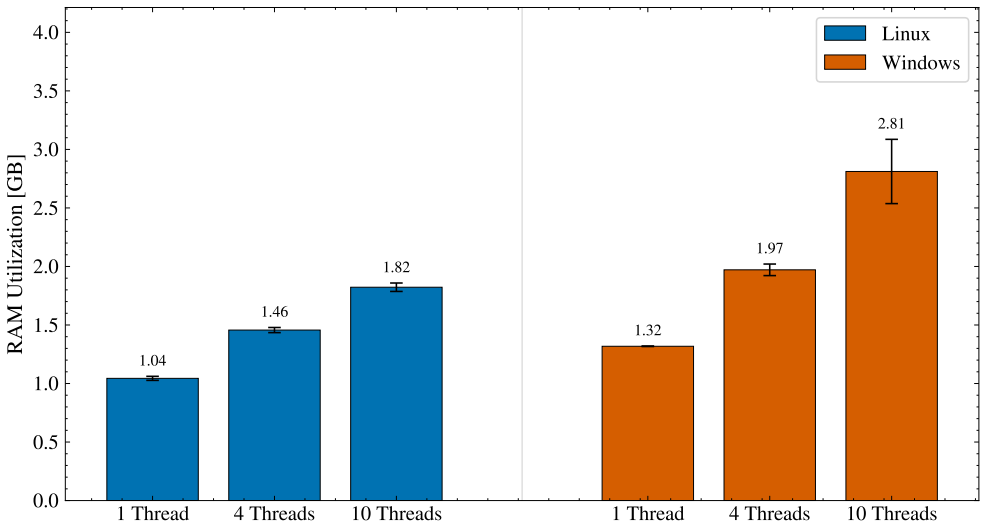

Abbildung A.17: Vergleich der RAM-Auslastung der Referenzmessung auf Linux und Windows über ein, vier und zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

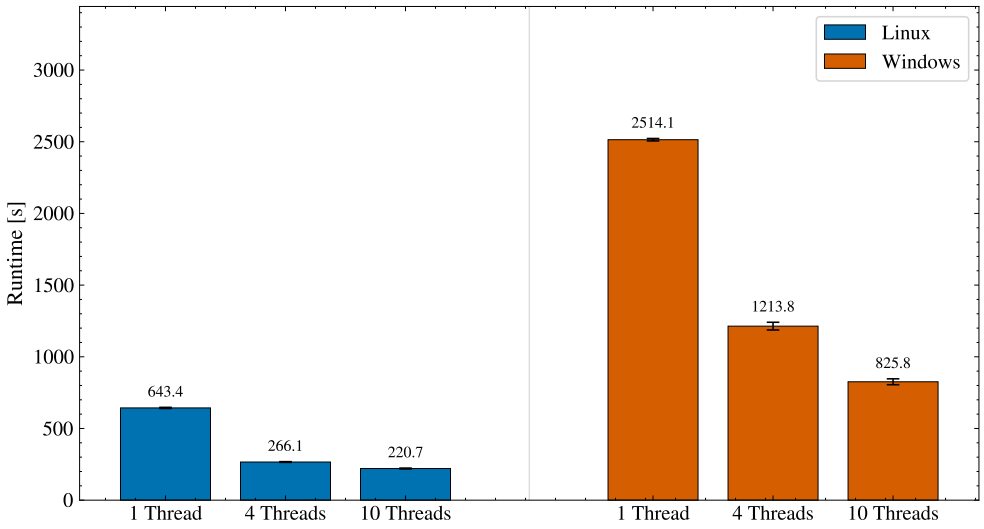

Abbildung A.18: Vergleich der Laufzeiten der Referenzmessung auf Linux und Windows über ein, vier und zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

A.2 FIFO_50

A.2.1 *Ein Thread*

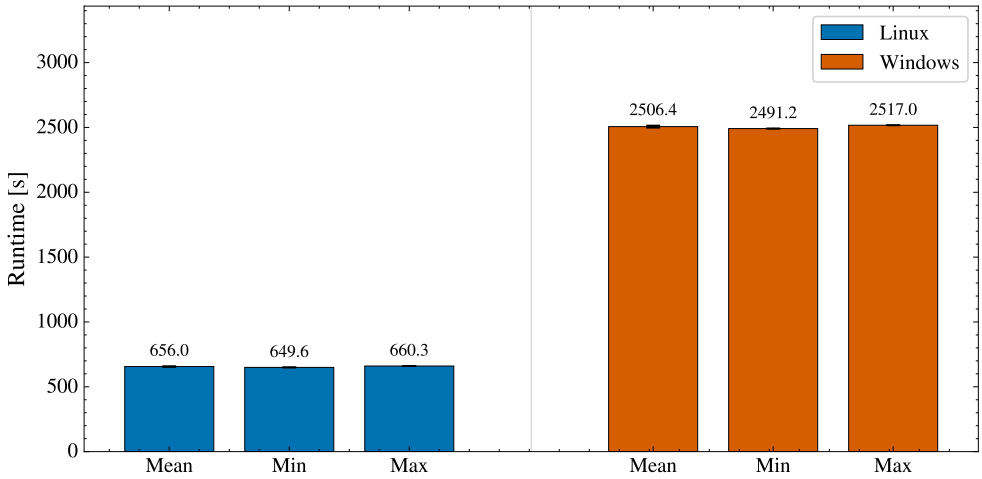

Abbildung A.19: Laufzeiten des FIFO_50-Schedulers auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

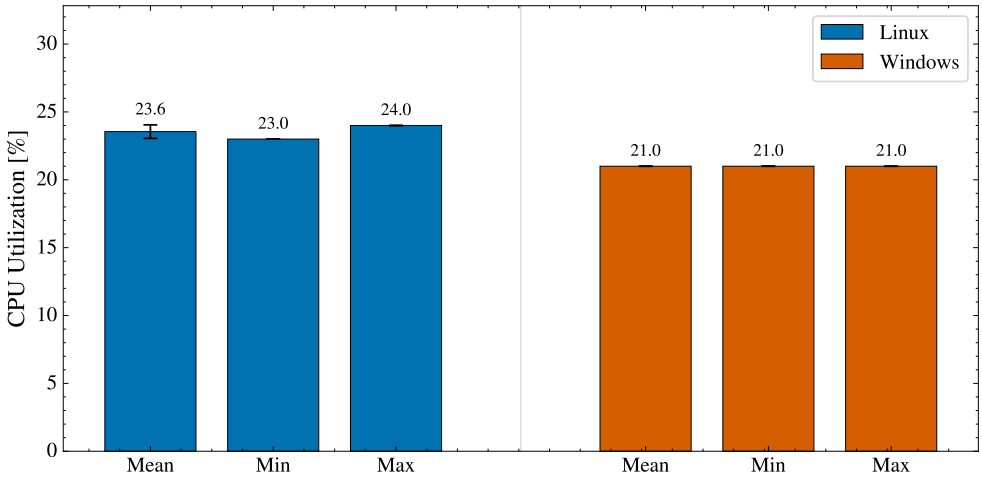

Abbildung A.20: CPU-Auslastung des FIFO_50-Schedulers auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

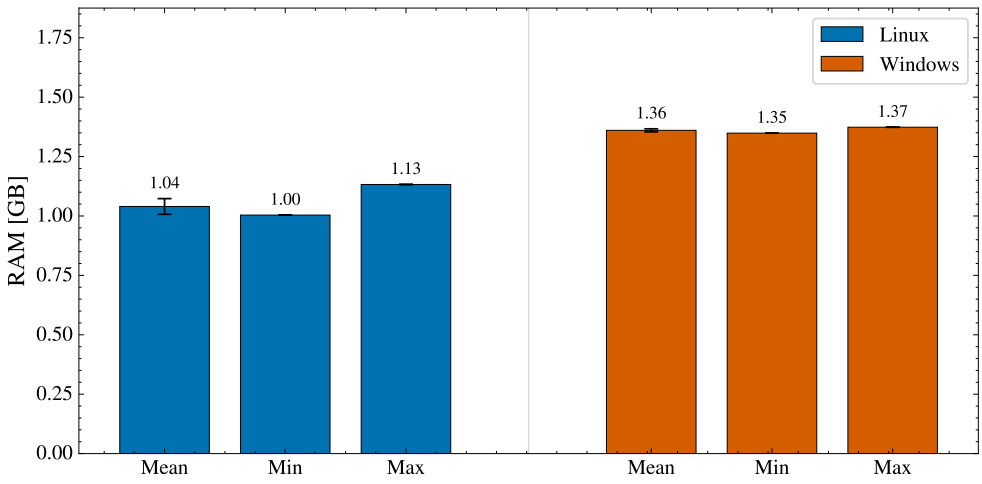

Abbildung A.21: RAM-Auslastung des FIFO_50-Schedulers auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

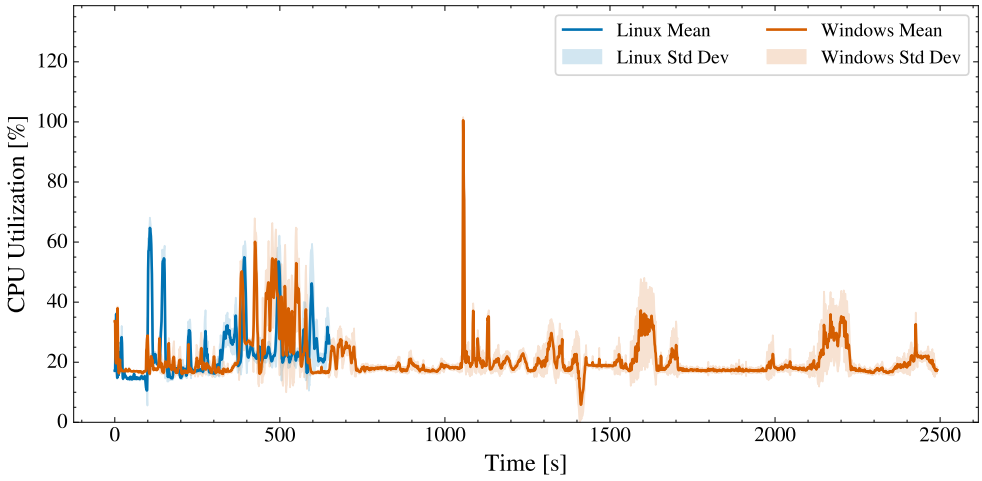

Abbildung A.22: Zeitlicher Verlauf der CPU-Auslastung des FIFO_50-Schedulers auf Linux und Windows bei einem Thread

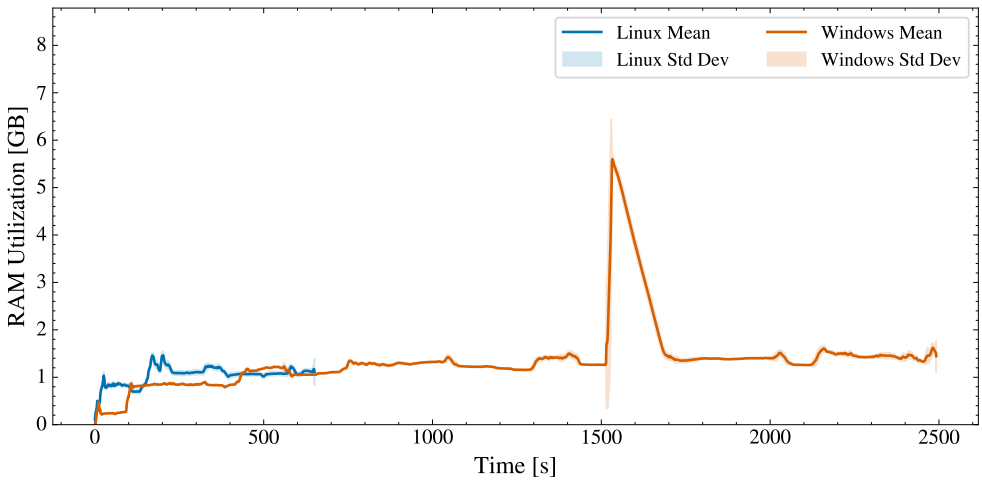

Abbildung A.23: Zeitlicher Verlauf der RAM-Auslastung des FIFO_50-Schedulers auf Linux und Windows bei einem Thread

A.2.2 *Vier Threads*

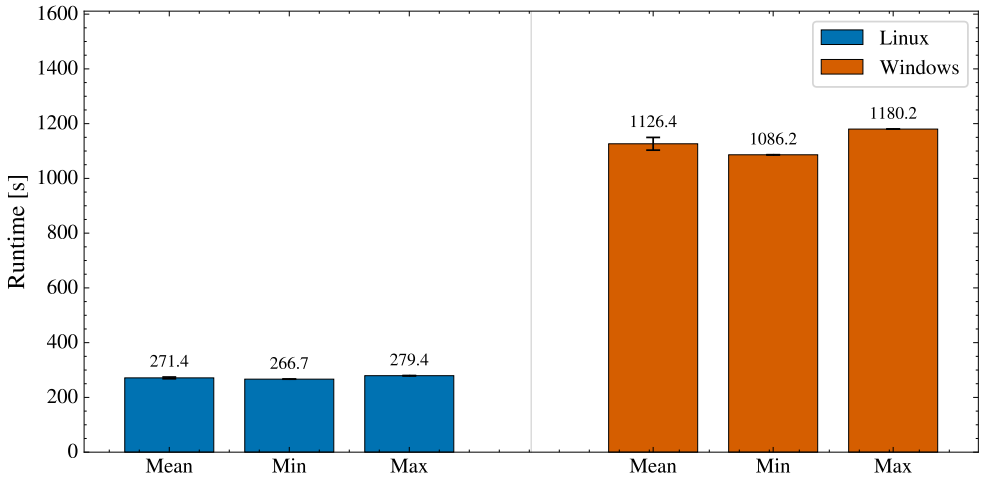

Abbildung A.24: Laufzeiten des FIFO_50-Schedulers auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

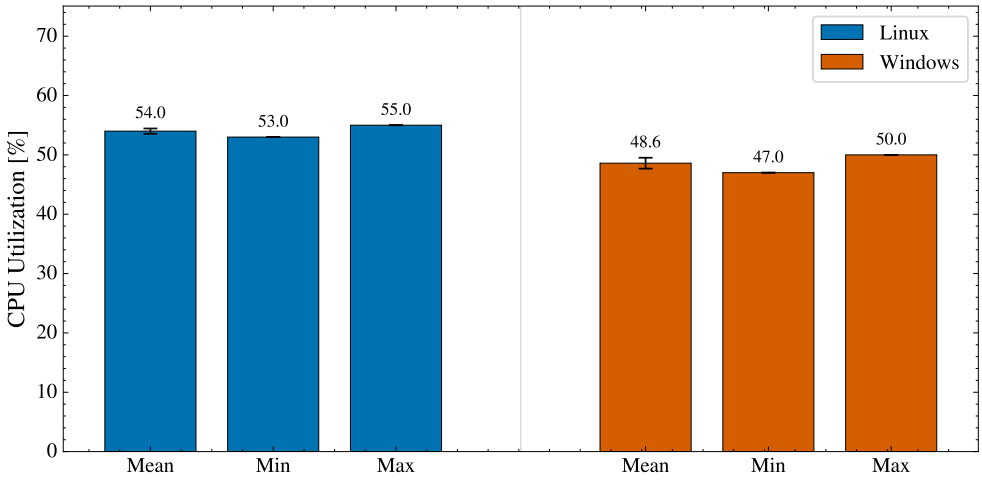

Abbildung A.25: CPU-Auslastung des FIFO_50-Schedulers auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

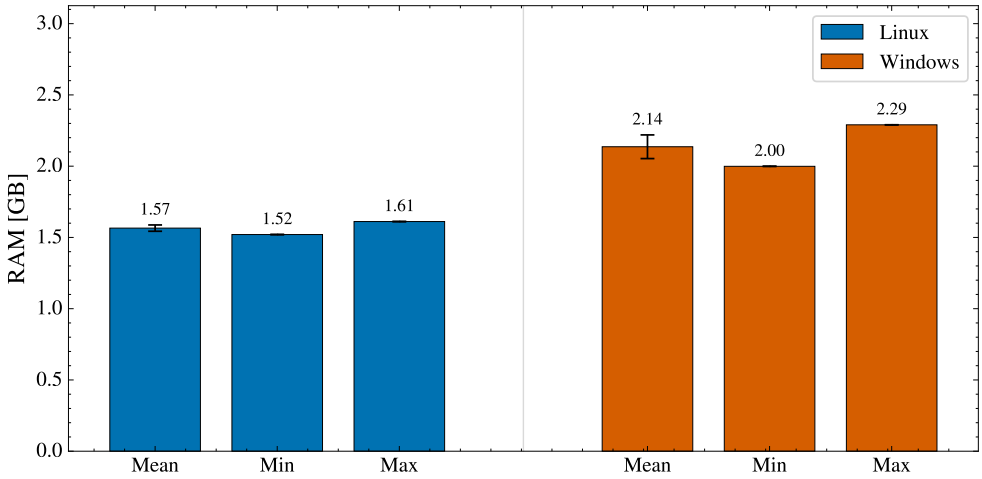

Abbildung A.26: RAM-Auslastung des FIFO_50-Schedulers auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

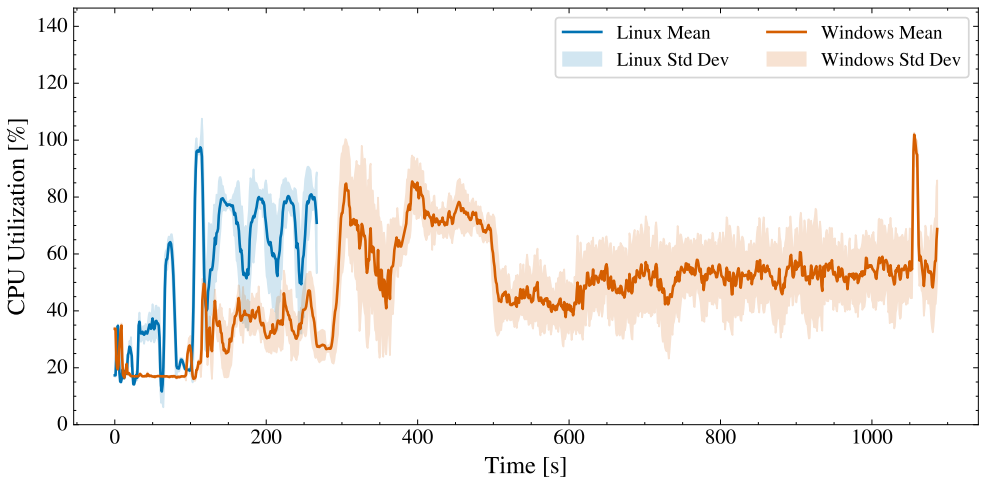

Abbildung A.27: Zeitlicher Verlauf der CPU-Auslastung des FIFO_50-Schedulers auf Linux und Windows bei vier Threads

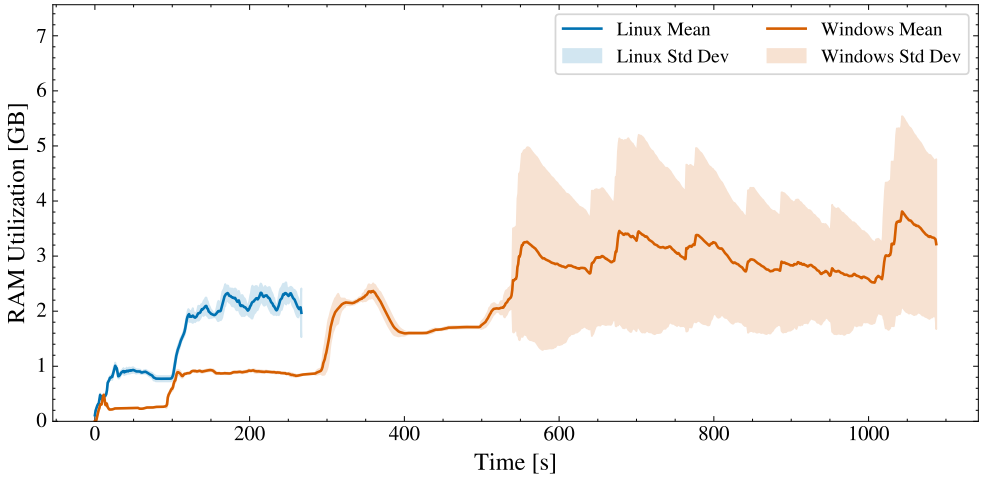

Abbildung A.28: Zeitlicher Verlauf der RAM-Auslastung des FIFO_50-Schedulers auf Linux und Windows bei vier Threads

A.2.3 *Zehn Threads*

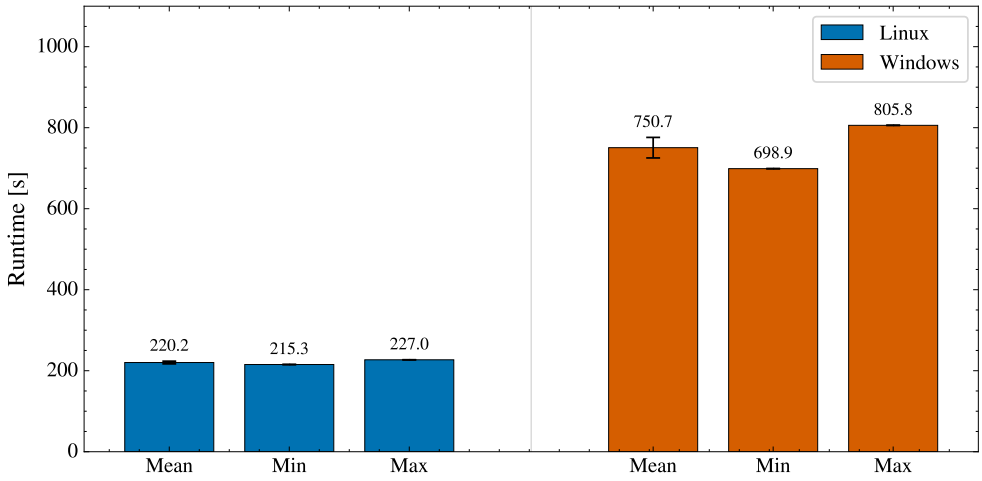

Abbildung A.29: Laufzeiten des FIFO_50-Schedulers auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

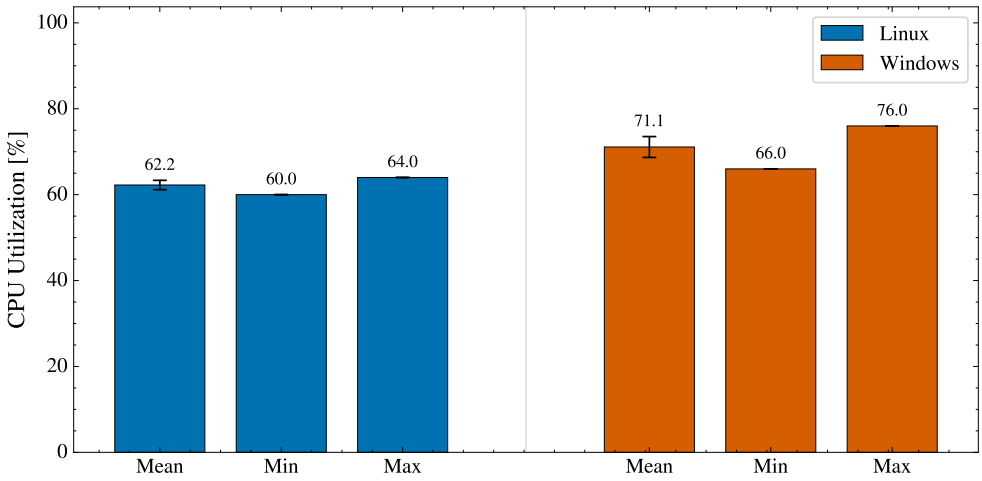

Abbildung A.30: CPU-Auslastung des FIFO_50-Schedulers auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

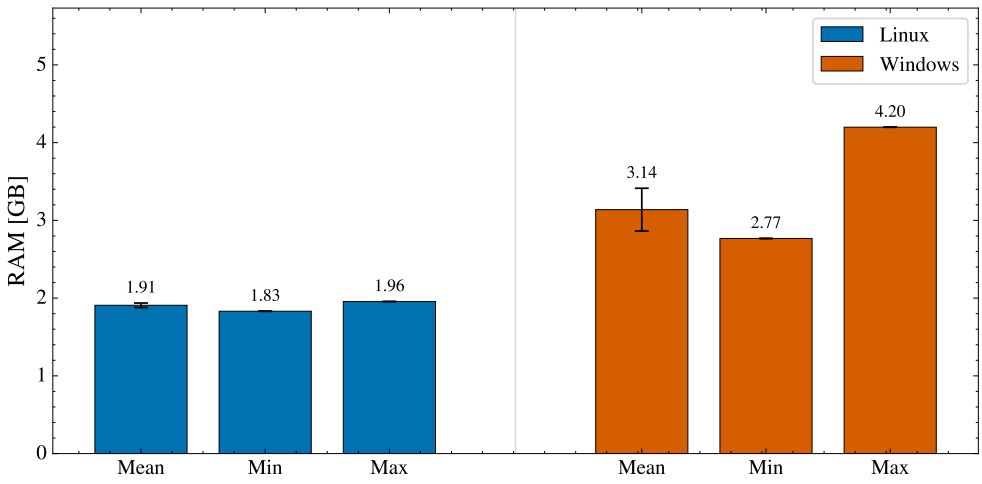

Abbildung A.31: RAM-Auslastung des FIFO_50-Schedulers auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

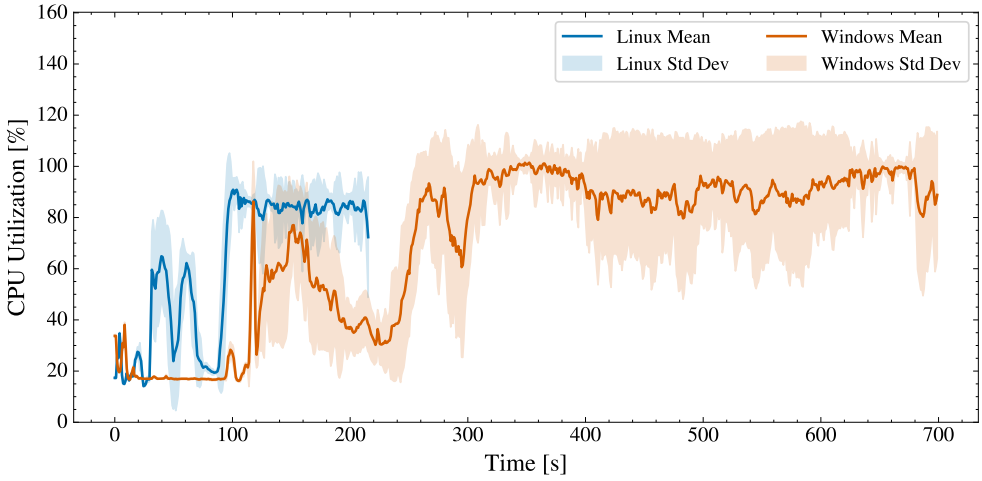

Abbildung A.32: Zeitlicher Verlauf der CPU-Auslastung des FIFO_50-Schedulers auf Linux und Windows bei zehn Threads

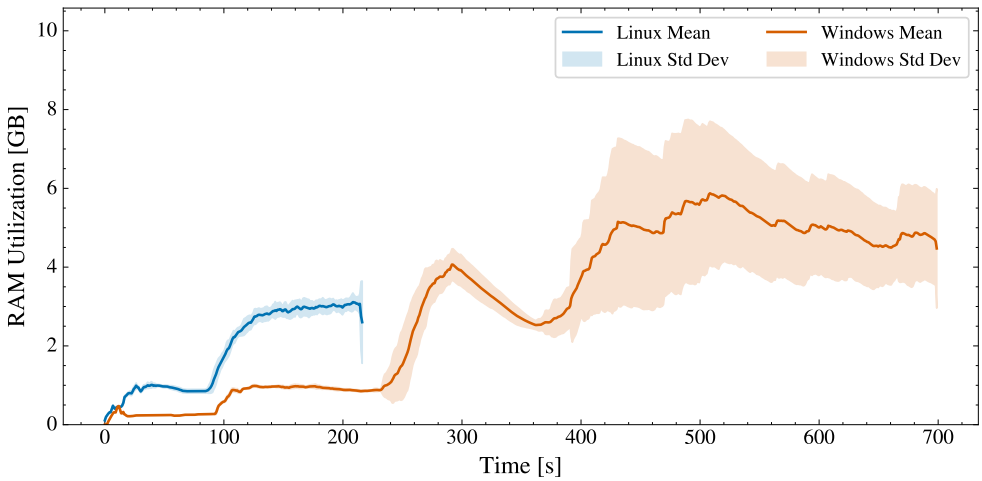

Abbildung A.33: Zeitlicher Verlauf der RAM-Auslastung des FIFO_50-Schedulers auf Linux und Windows bei zehn Threads

A.2.4 *Vergleich*

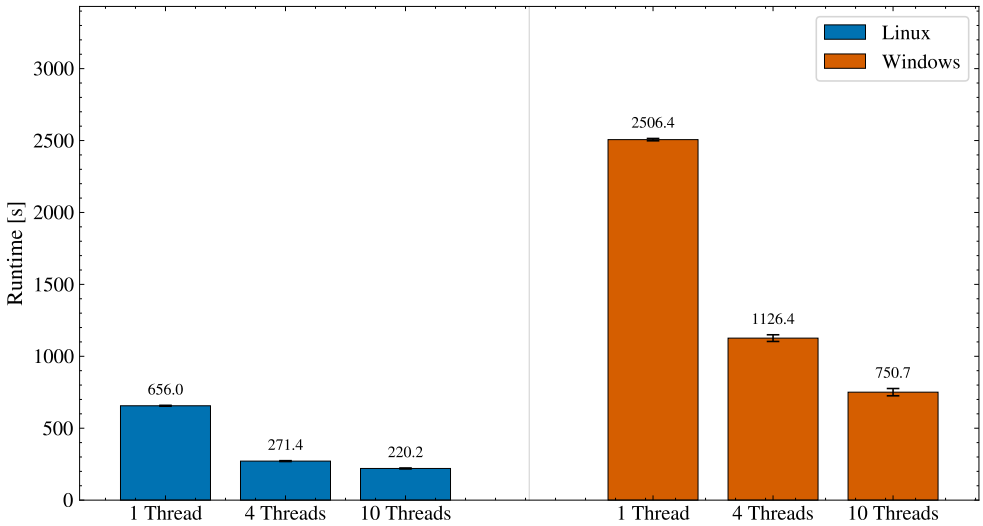

Abbildung A.34: Vergleich der Laufzeiten des FIFO_50-Schedulers auf Linux und Windows über ein, vier und zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

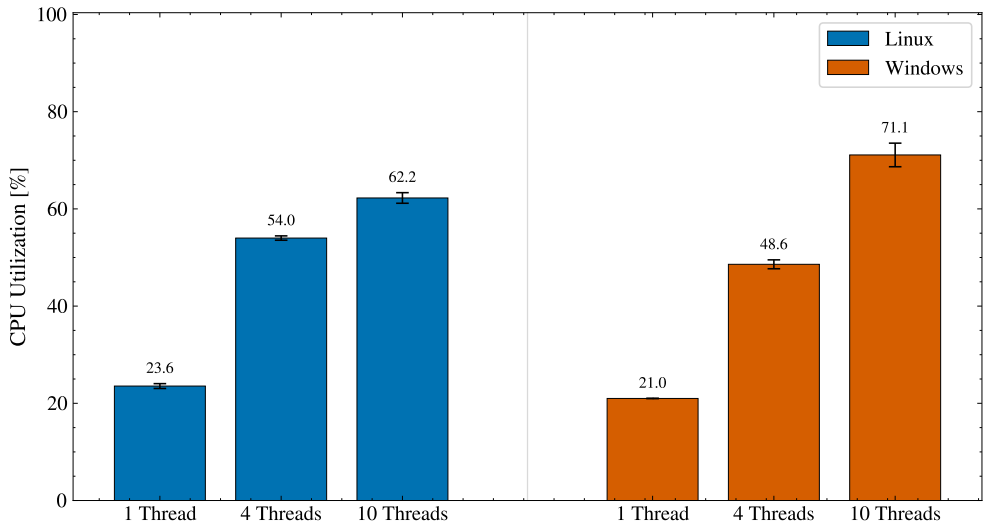

Abbildung A.35: Vergleich der CPU-Auslastung des FIFO_50-Schedulers auf Linux und Windows über ein, vier und zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

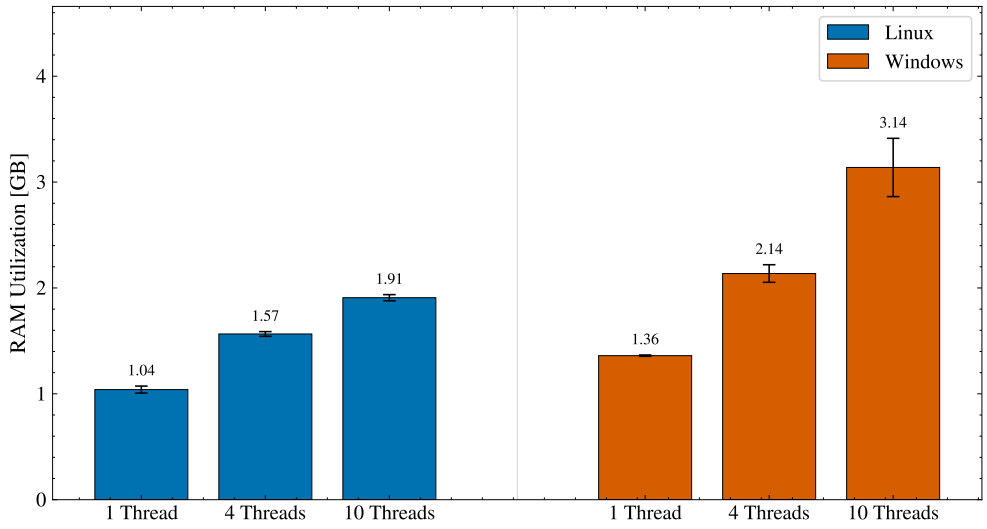

Abbildung A.36: Vergleich der RAM-Auslastung des FIFO_50-Schedulers auf Linux und Windows über ein, vier und zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

A.3 FIFO_500

A.3.1 *Ein Thread*

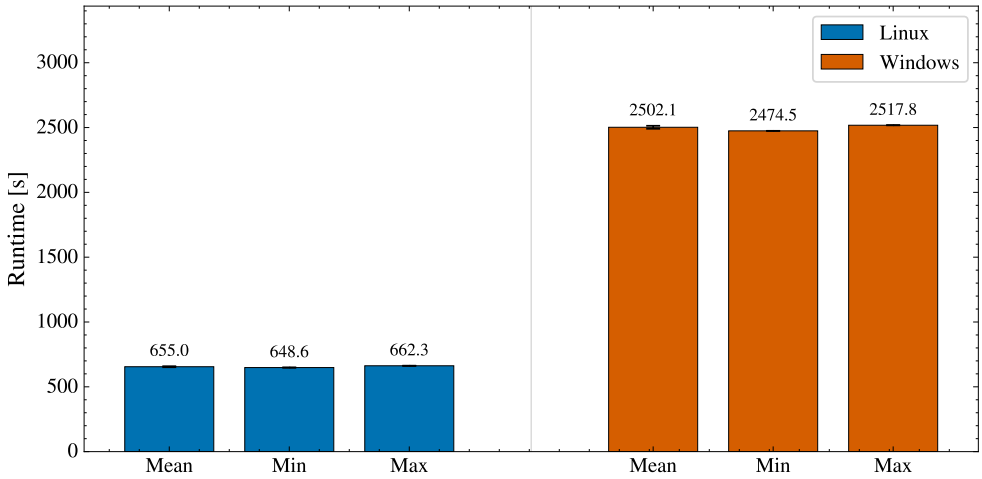

Abbildung A.37: Laufzeiten des FIFO_500-Schedulers auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

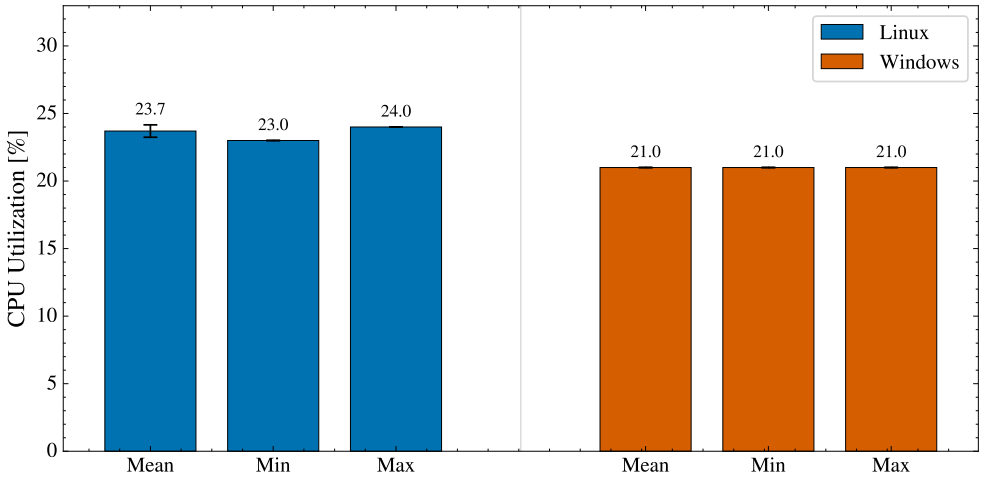

Abbildung A.38: CPU-Auslastung des FIFO_500-Schedulers auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

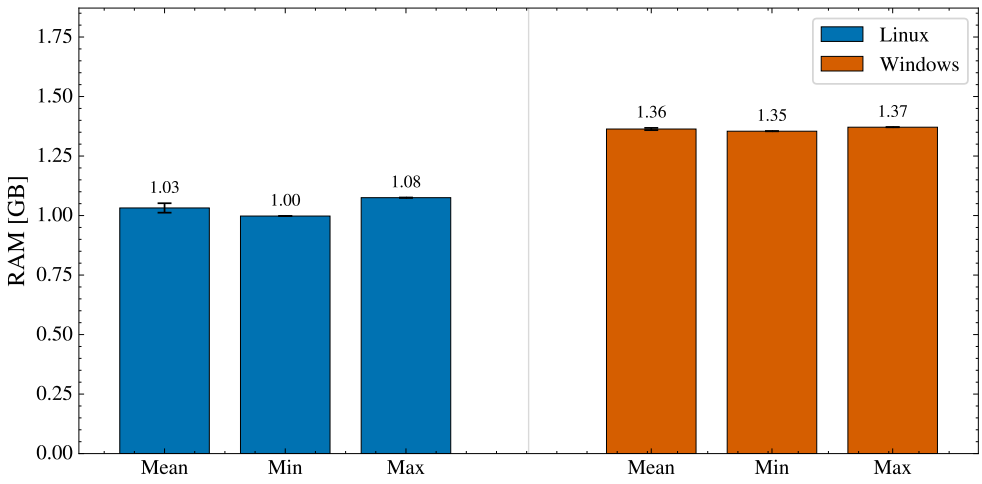

Abbildung A.39: RAM-Auslastung des FIFO_500-Schedulers auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

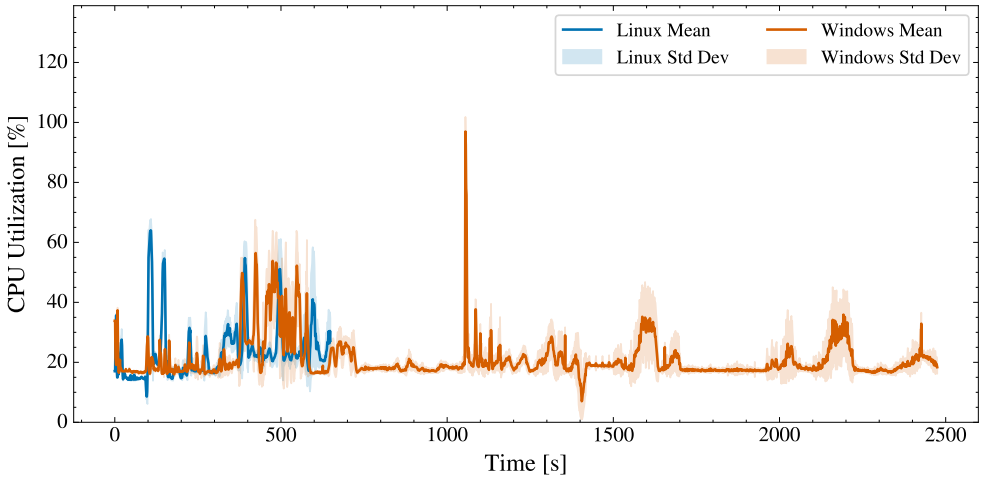

Abbildung A.40: Zeitlicher Verlauf der CPU-Auslastung des FIFO_500-Schedulers auf Linux und Windows bei einem Thread

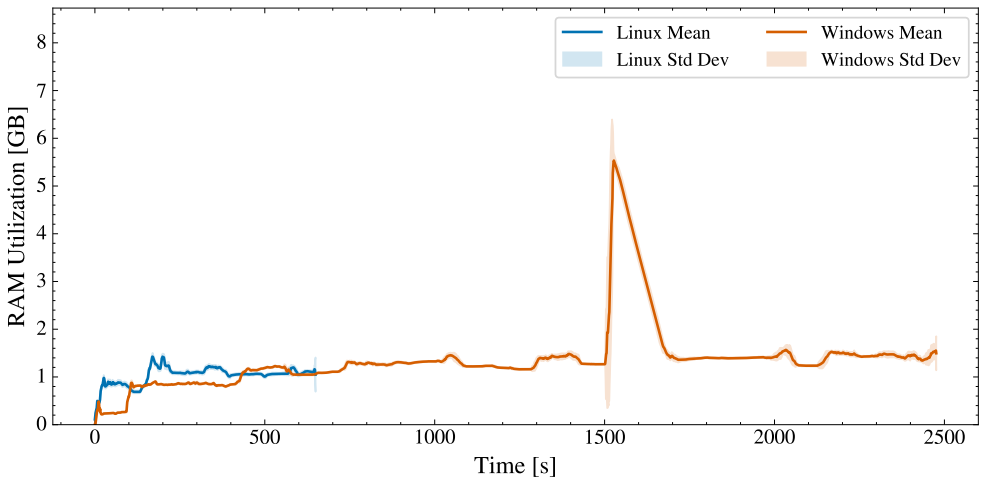

Abbildung A.41: Zeitlicher Verlauf der RAM-Auslastung des FIFO_500-Schedulers auf Linux und Windows bei einem Thread

A.3.2 *Vier Threads*

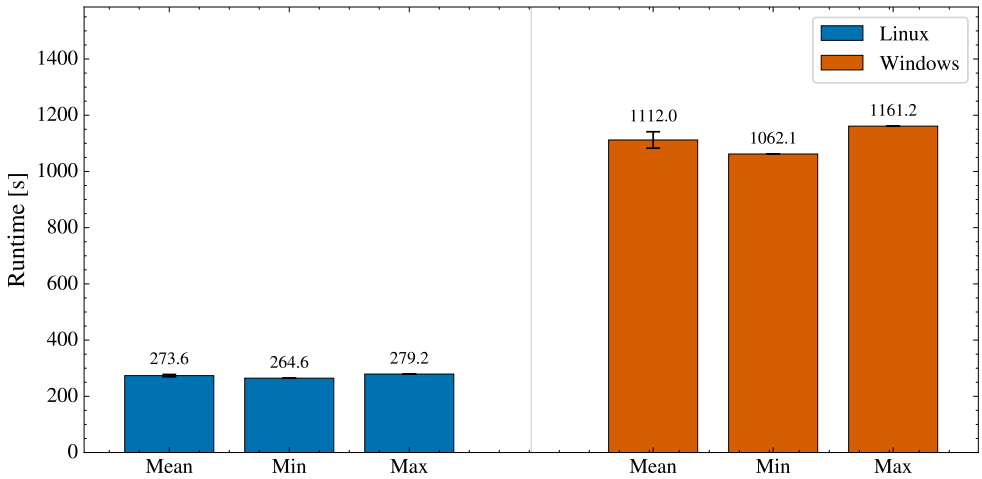

Abbildung A.42: Laufzeiten des FIFO_500-Schedulers auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

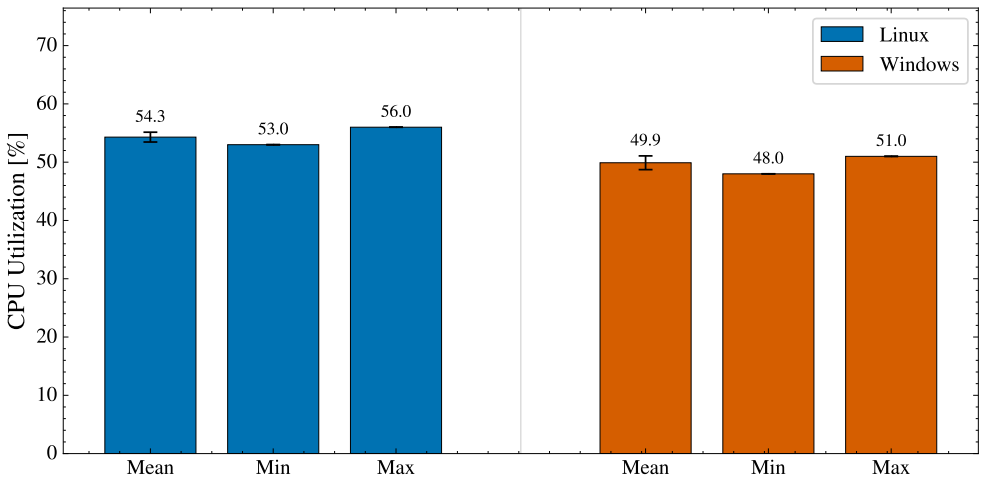

Abbildung A.43: CPU-Auslastung des FIFO_500-Schedulers auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

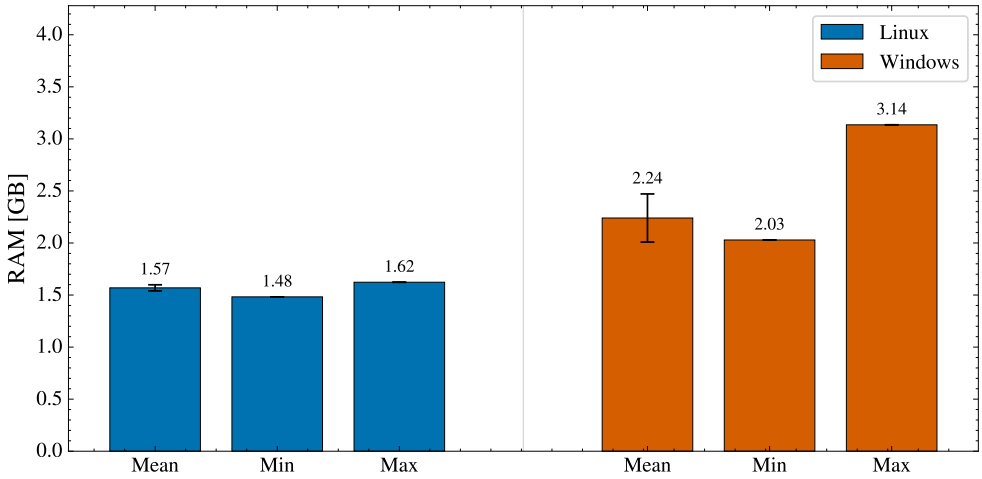

Abbildung A.44: RAM-Auslastung des FIFO_500-Schedulers auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

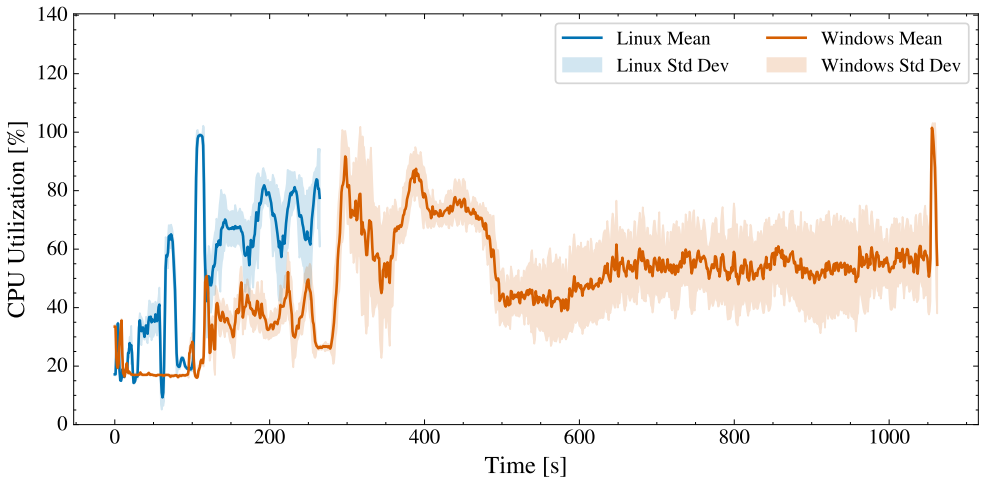

Abbildung A.45: Zeitlicher Verlauf der CPU-Auslastung des FIFO_500-Schedulers auf Linux und Windows bei vier Threads

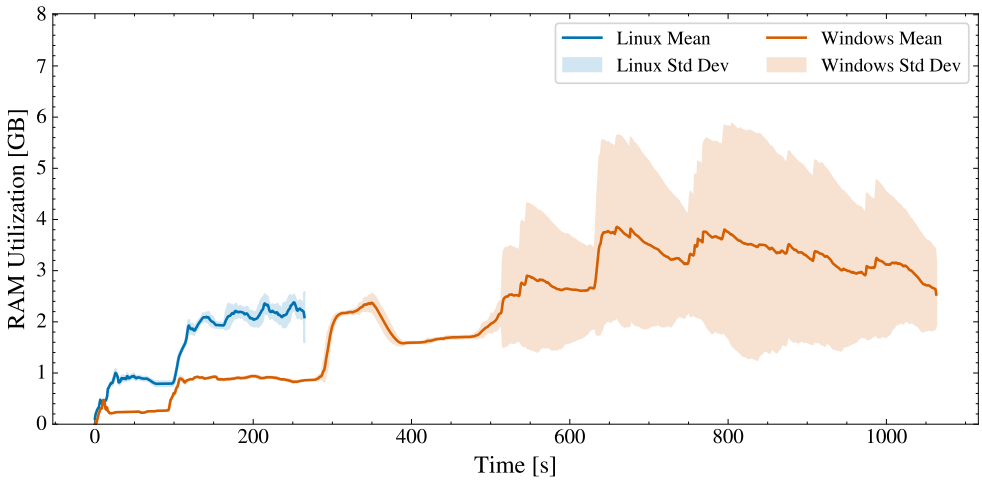

Abbildung A.46: Zeitlicher Verlauf der RAM-Auslastung des FIFO_500-Schedulers auf Linux und Windows bei vier Threads

A.3.3 *Zehn Threads*

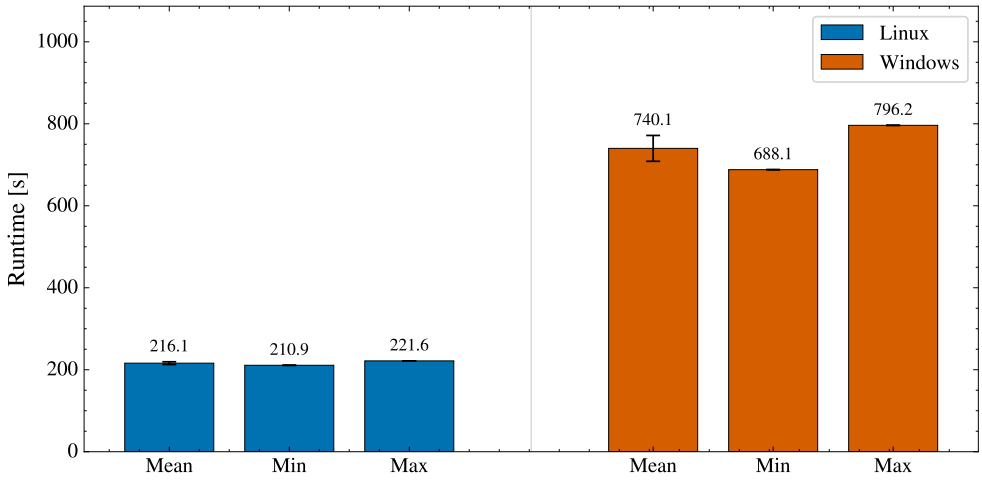

Abbildung A.47: Laufzeiten des FIFO_500-Schedulers auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

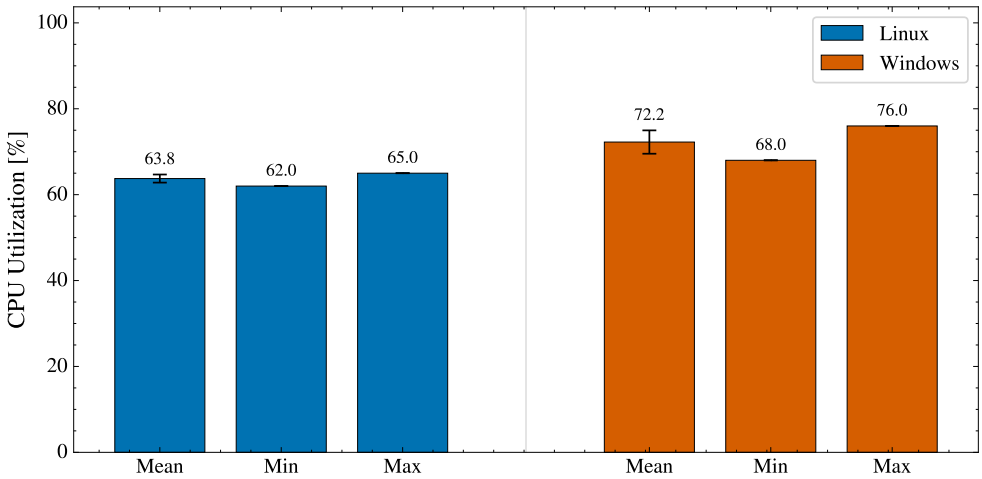

Abbildung A.48: CPU-Auslastung des FIFO_500-Schedulers auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

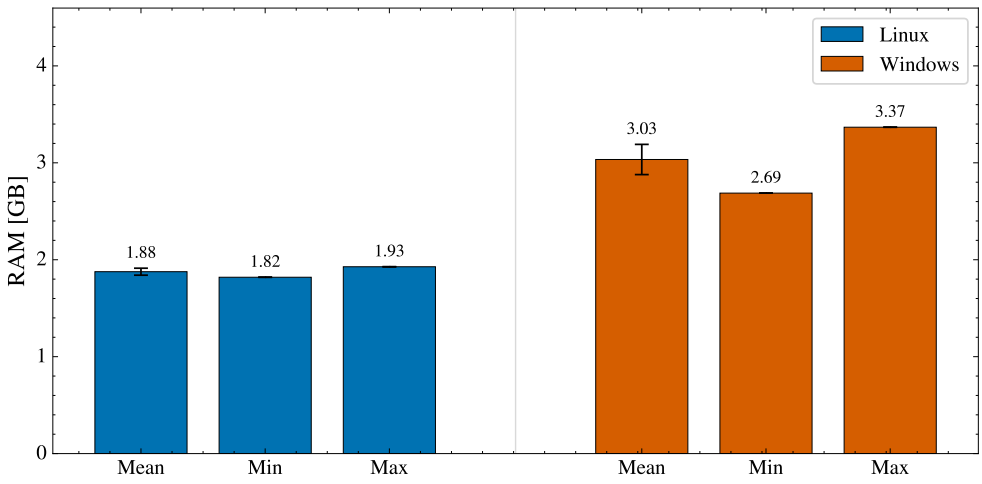

Abbildung A.49: RAM-Auslastung des FIFO_500-Schedulers auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

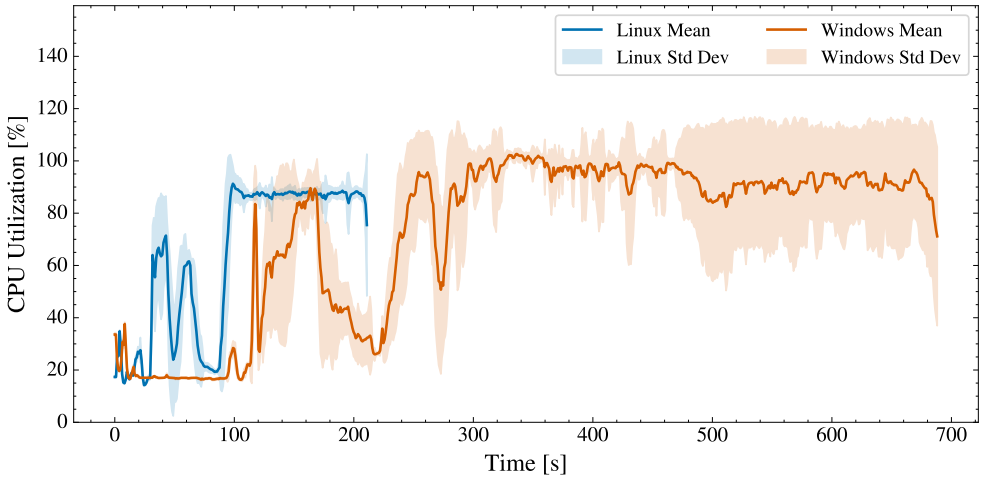

Abbildung A.50: Zeitlicher Verlauf der CPU-Auslastung des FIFO_500-Schedulers auf Linux und Windows bei zehn Threads

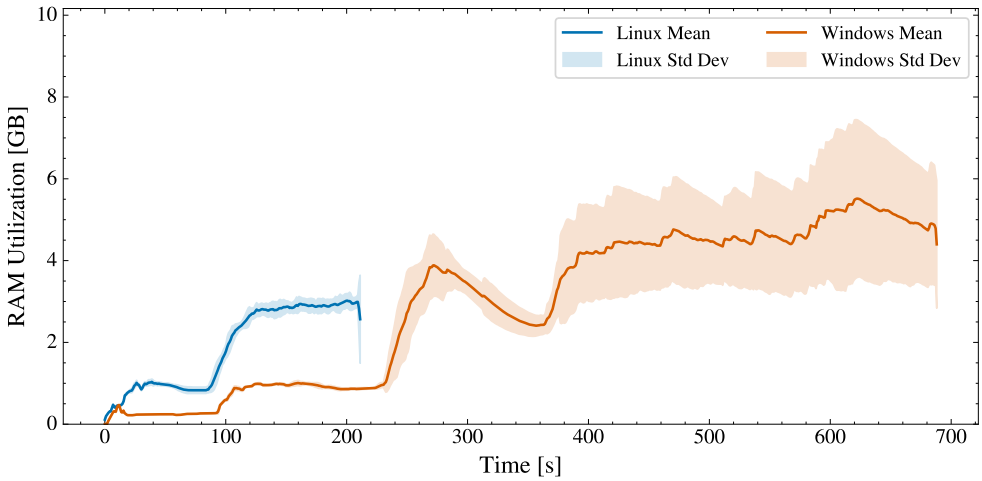

Abbildung A.51: Zeitlicher Verlauf der RAM-Auslastung des FIFO_500-Schedulers auf Linux und Windows bei zehn Threads

A.3.4 *Vergleich*

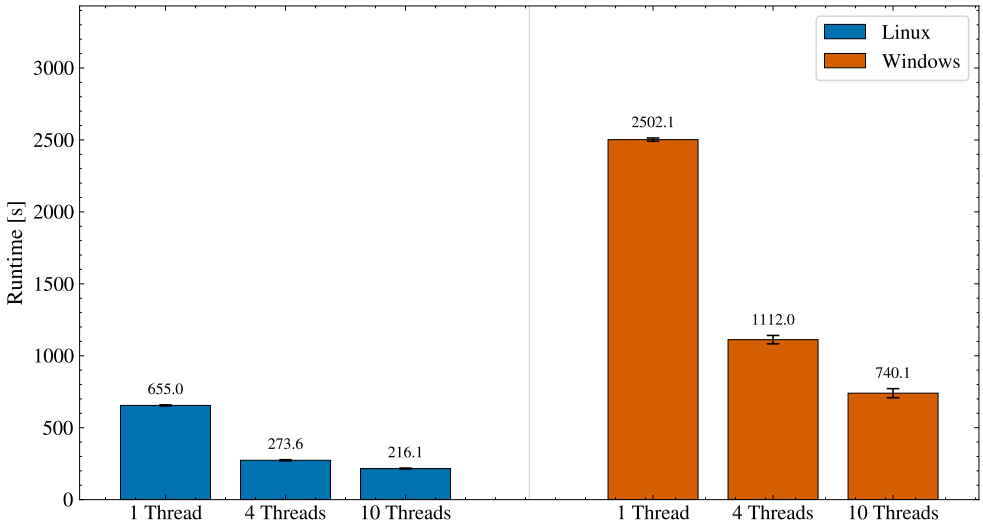

Abbildung A.52: Vergleich der Laufzeiten des FIFO_500-Schedulers auf Linux und Windows über ein, vier und zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

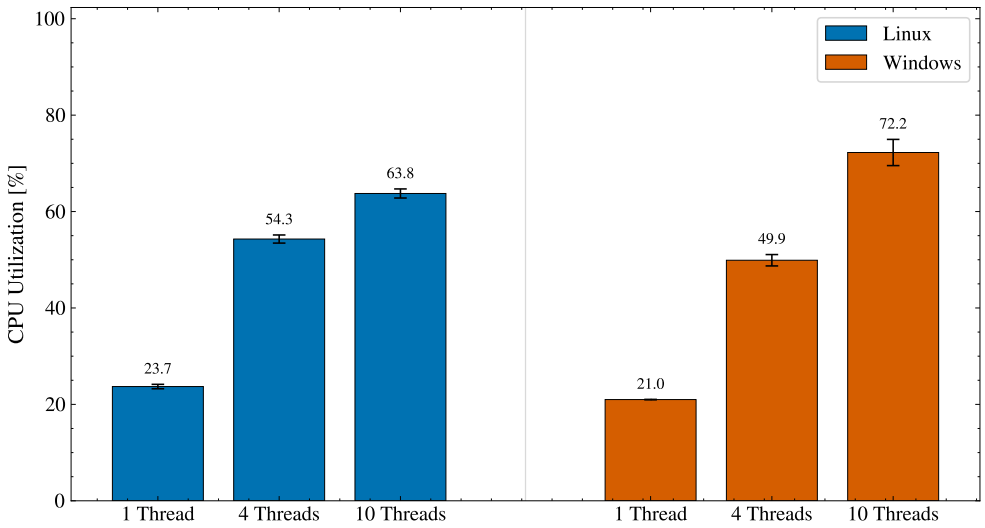

Abbildung A.53: Vergleich der CPU-Auslastung des FIFO_500-Schedulers auf Linux und Windows über ein, vier und zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

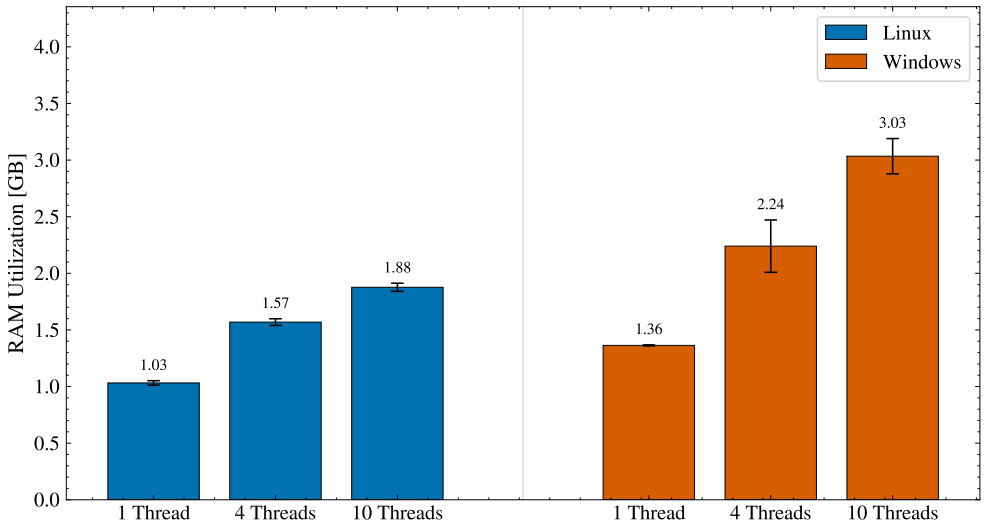

Abbildung A.54: Vergleich der RAM-Auslastung des FIFO_500-Schedulers auf Linux und Windows über ein, vier und zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

A.4 SHORTEST-SUBTREE-TIME-FIRST (SSTF)

A.4.1 *Ein Thread*

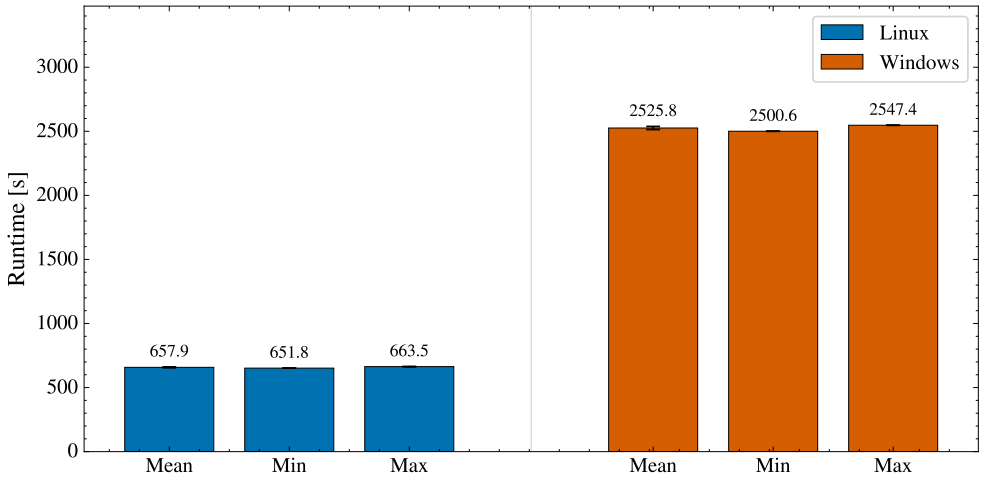

Abbildung A.55: Laufzeiten des SSTF-Schedulers auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

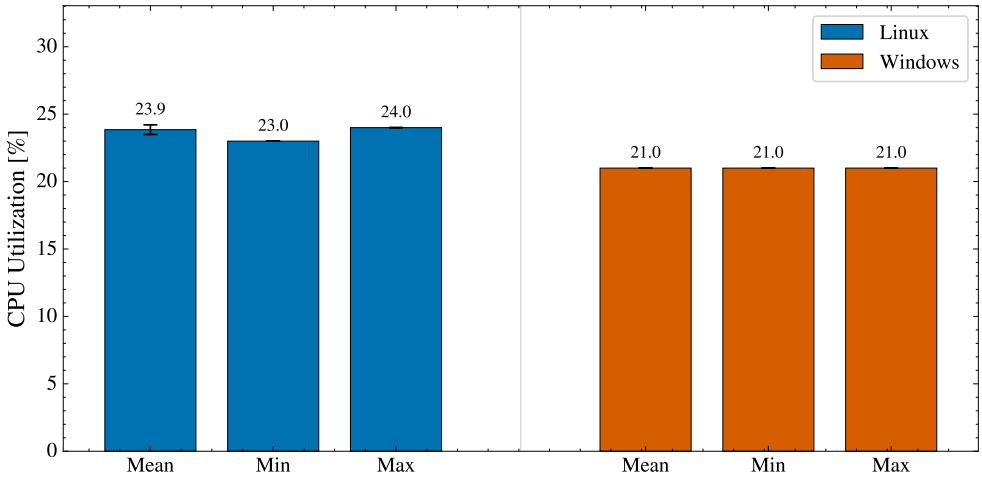

Abbildung A.56: CPU-Auslastung des SSTF-Schedulers auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

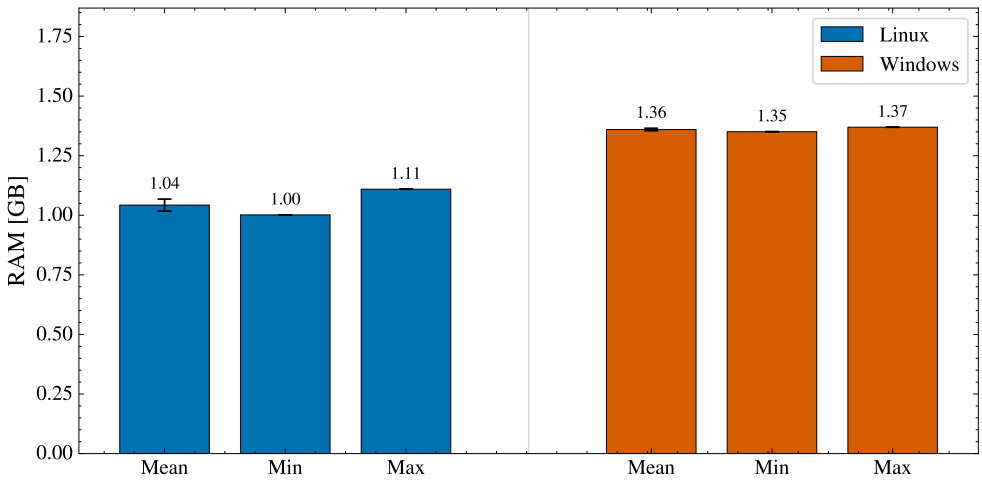

Abbildung A.57: RAM-Auslastung des SSTF-Schedulers auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

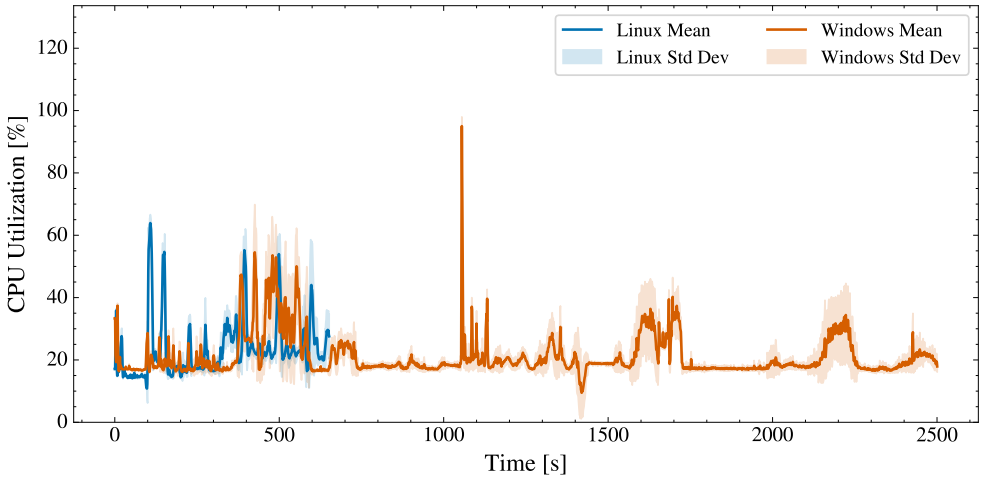

Abbildung A.58: Zeitlicher Verlauf der CPU-Auslastung des SSTF-Schedulers auf Linux und Windows bei einem Thread

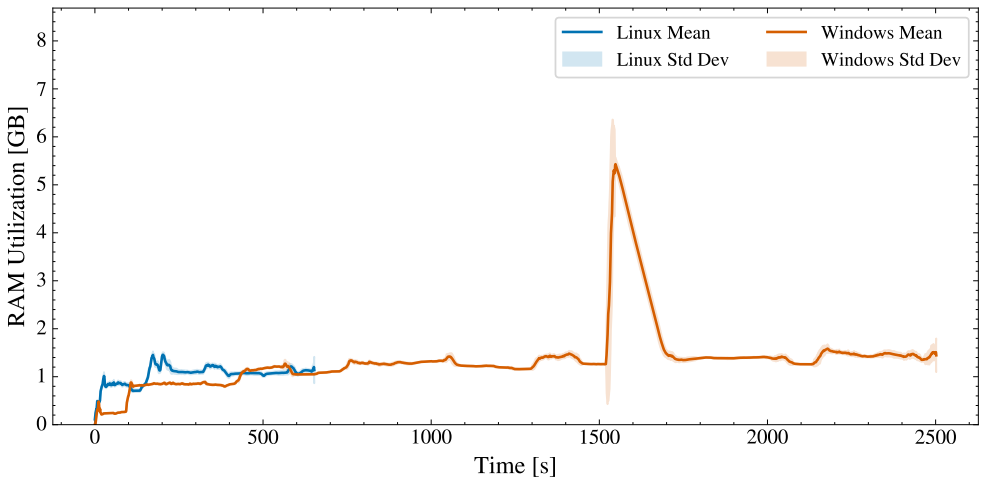

Abbildung A.59: Zeitlicher Verlauf der RAM-Auslastung des SSTF-Schedulers auf Linux und Windows bei einem Thread

A.4.2 *Vier Threads*

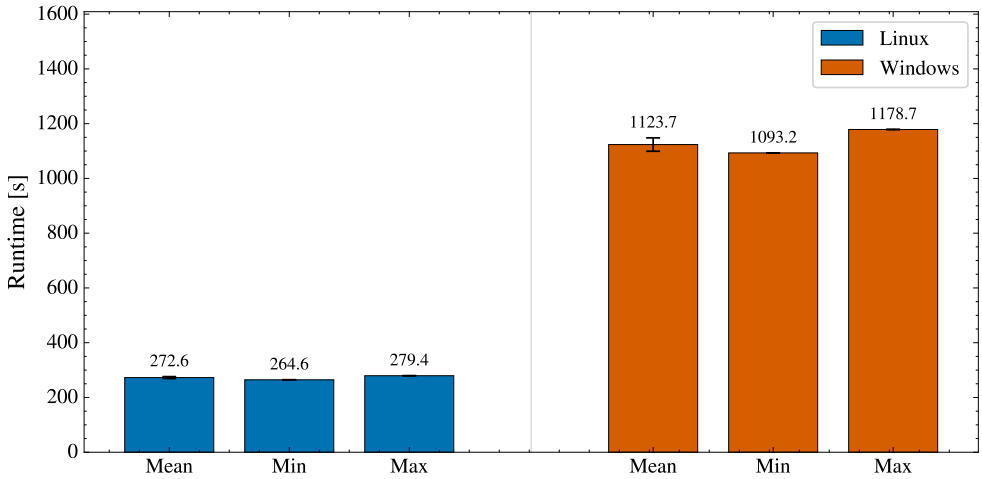

Abbildung A.60: Laufzeiten des SSTF-Schedulers auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

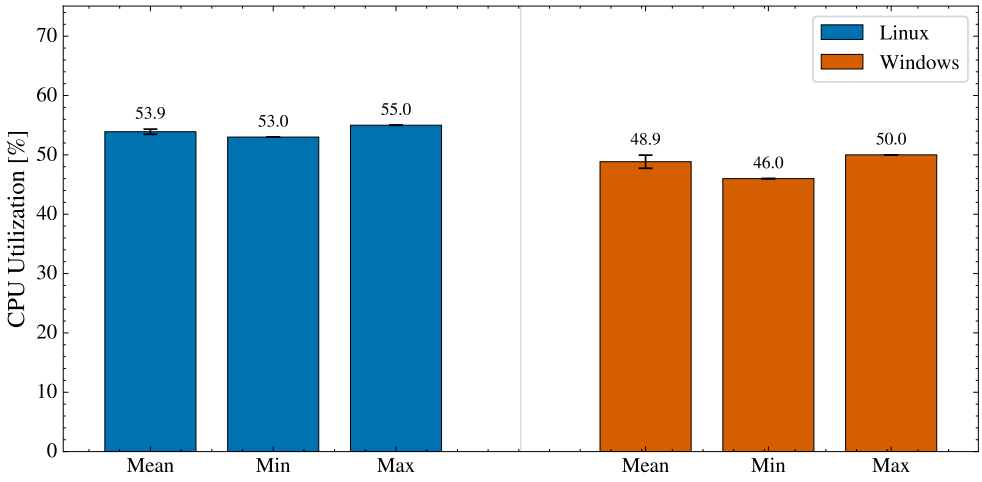

Abbildung A.61: CPU-Auslastung des SSTF-Schedulers auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

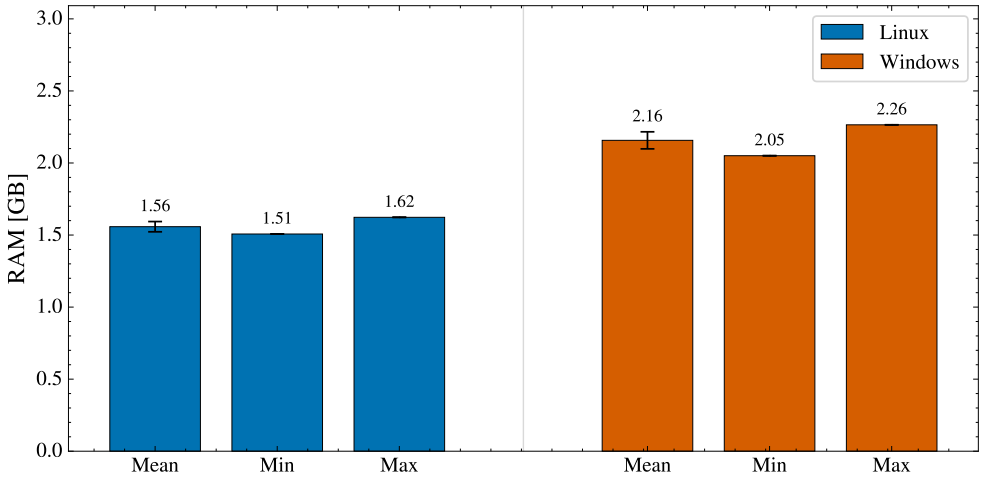

Abbildung A.62: RAM-Auslastung des SSTF-Schedulers auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

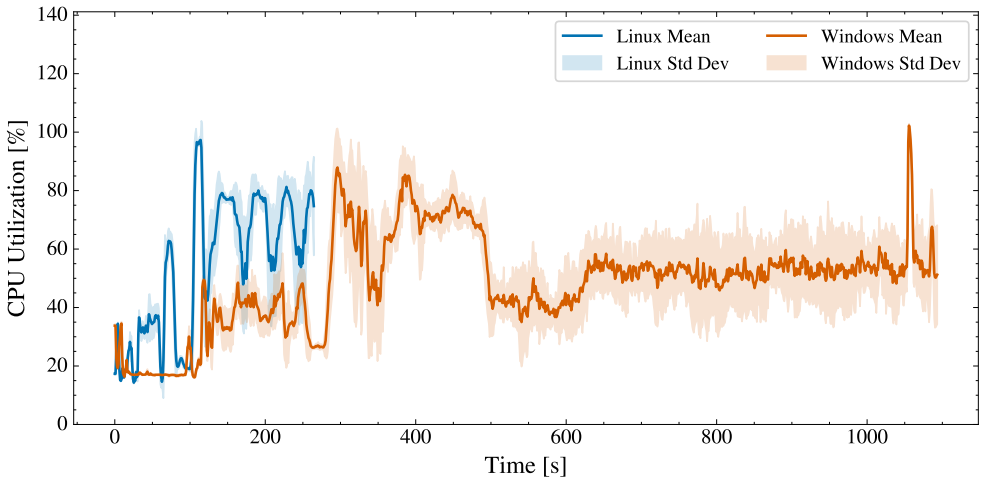

Abbildung A.63: Zeitlicher Verlauf der CPU-Auslastung des SSTF-Schedulers auf Linux und Windows bei vier Threads

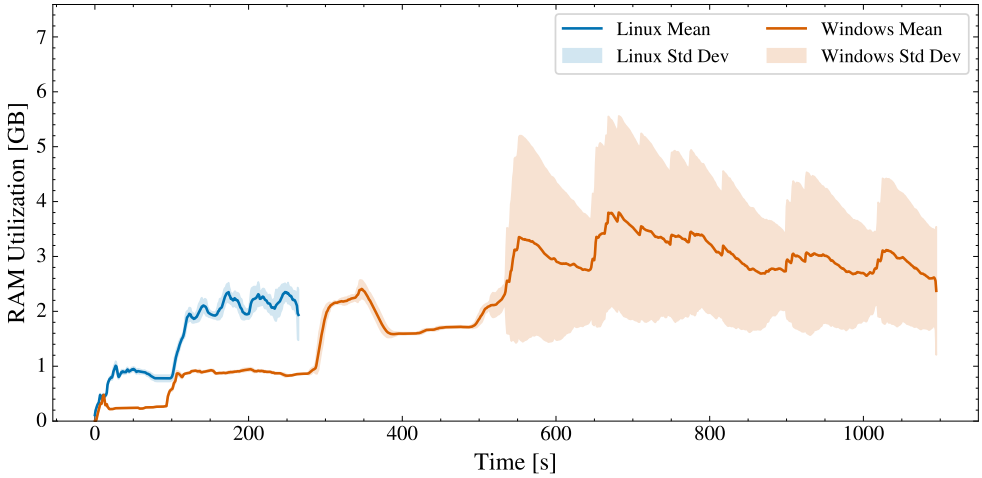

Abbildung A.64: Zeitlicher Verlauf der RAM-Auslastung des SSTF-Schedulers auf Linux und Windows bei vier Threads

A.4.3 *Zehn Threads*

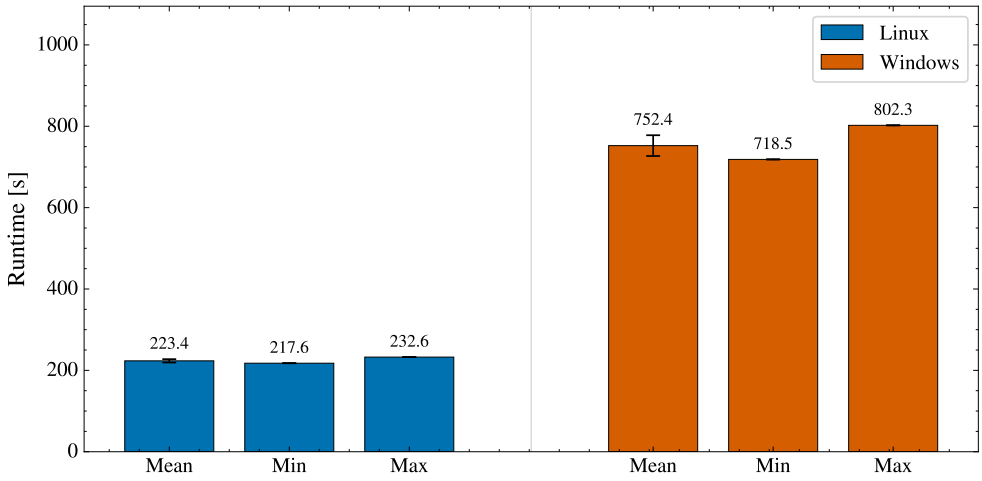

Abbildung A.65: Laufzeiten des SSTF-Schedulers auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

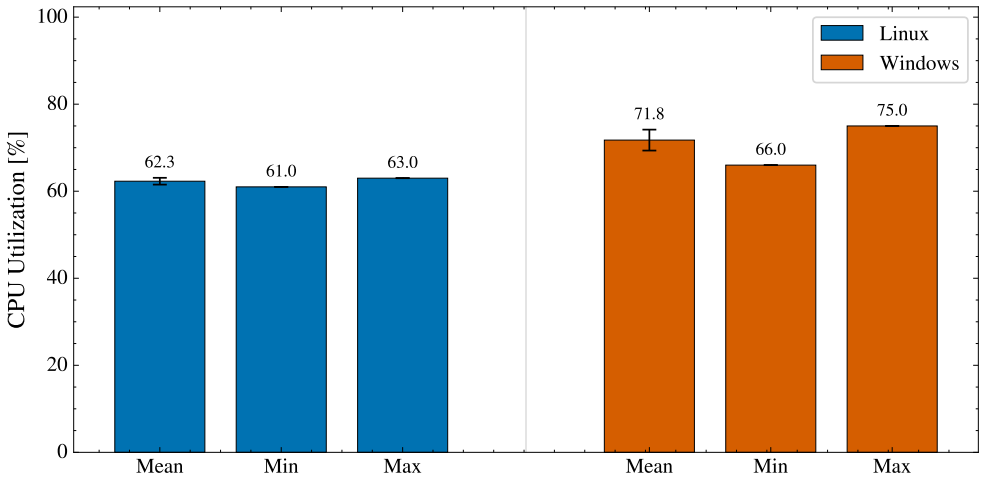

Abbildung A.66: CPU-Auslastung des SSTF-Schedulers auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

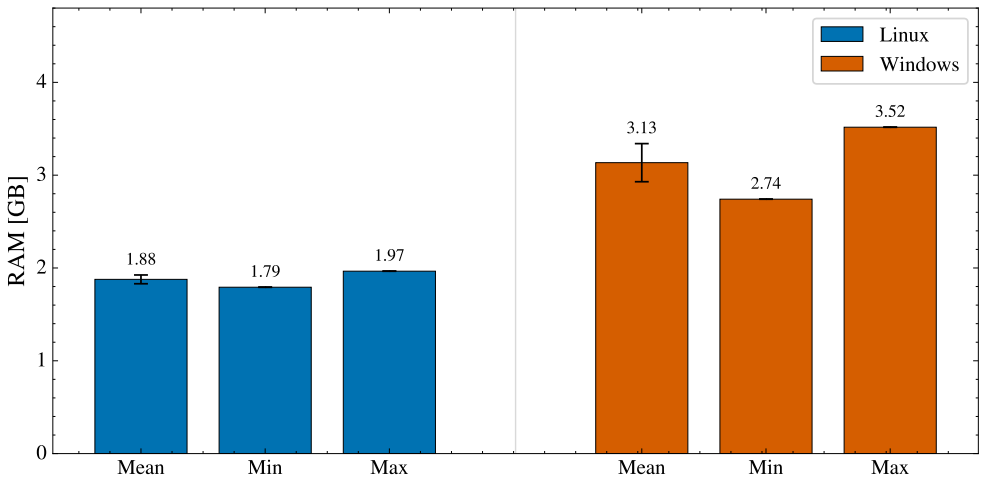

Abbildung A.67: RAM-Auslastung des SSTF-Schedulers auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

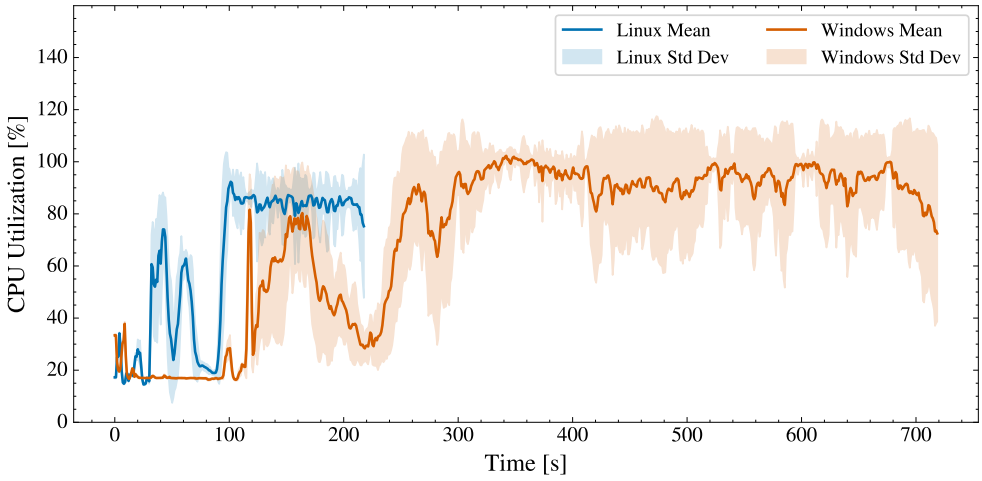

Abbildung A.68: Zeitlicher Verlauf der CPU-Auslastung des SSTF-Schedulers auf Linux und Windows bei zehn Threads

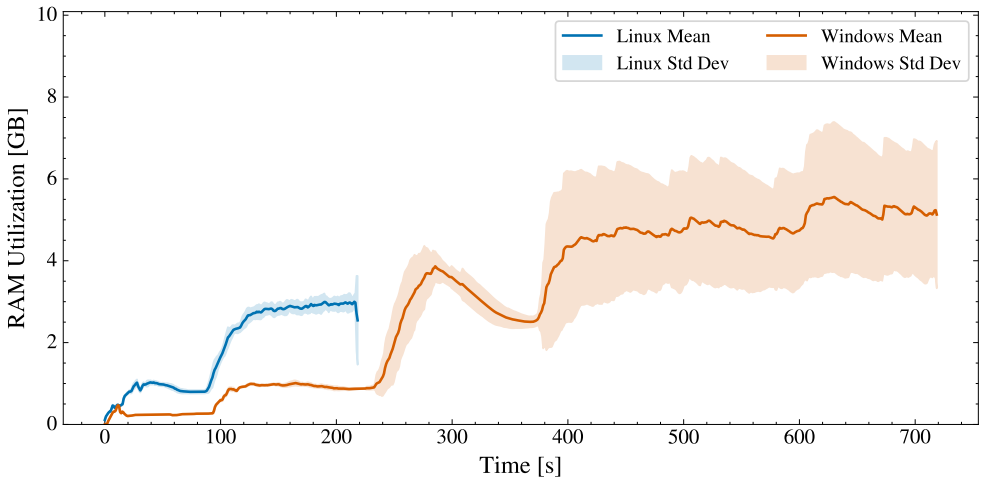

Abbildung A.69: Zeitlicher Verlauf der RAM-Auslastung des SSTF-Schedulers auf Linux und Windows bei zehn Threads

A.4.4 *Vergleich*

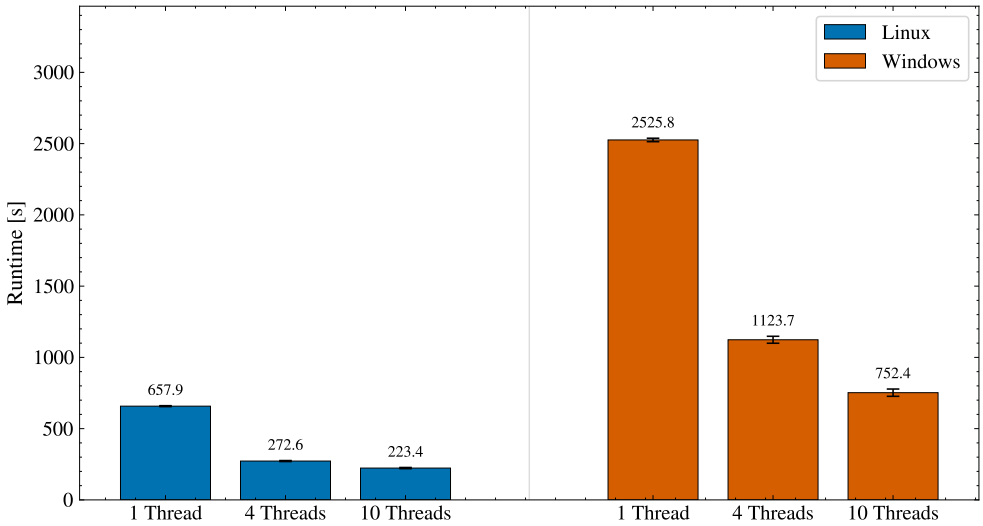

Abbildung A.70: Vergleich der Laufzeiten des SSTF-Schedulers auf Linux und Windows über ein, vier und zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

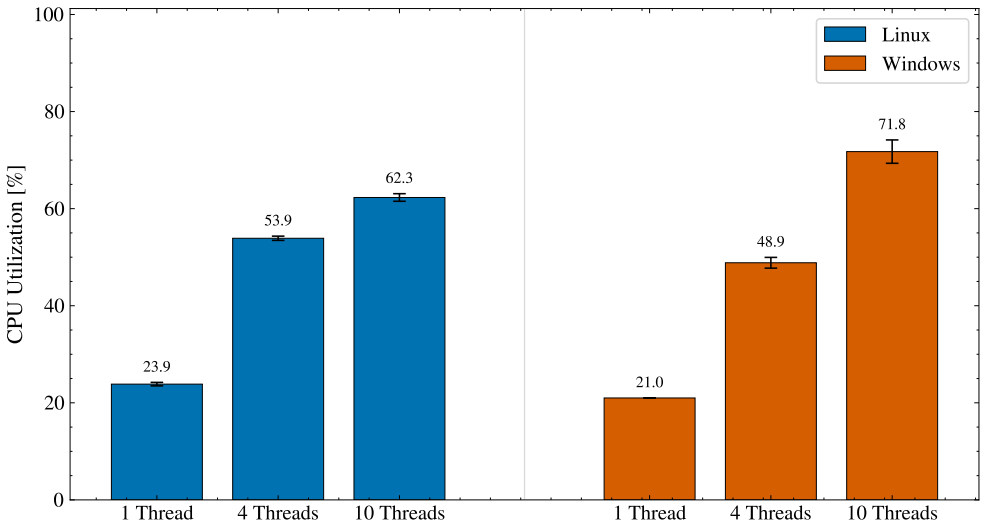

Abbildung A.71: Vergleich der CPU-Auslastung des SSTF-Schedulers auf Linux und Windows über ein, vier und zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

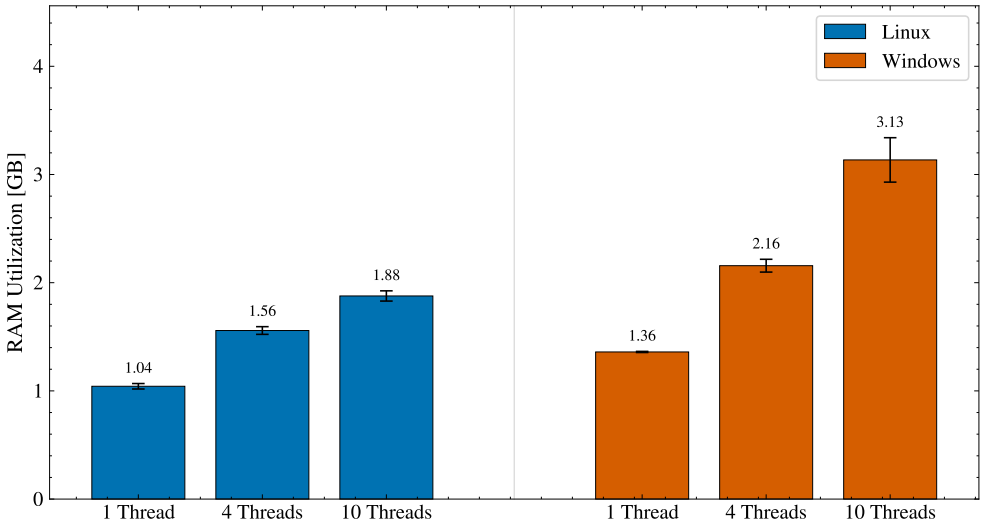

Abbildung A.72: Vergleich der RAM-Auslastung des SSTF-Schedulers auf Linux und Windows über ein, vier und zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

A.5 WORK-STEALING MIT SSTF-PRIORISIERUNG

A.5.1 *Ein Thread*

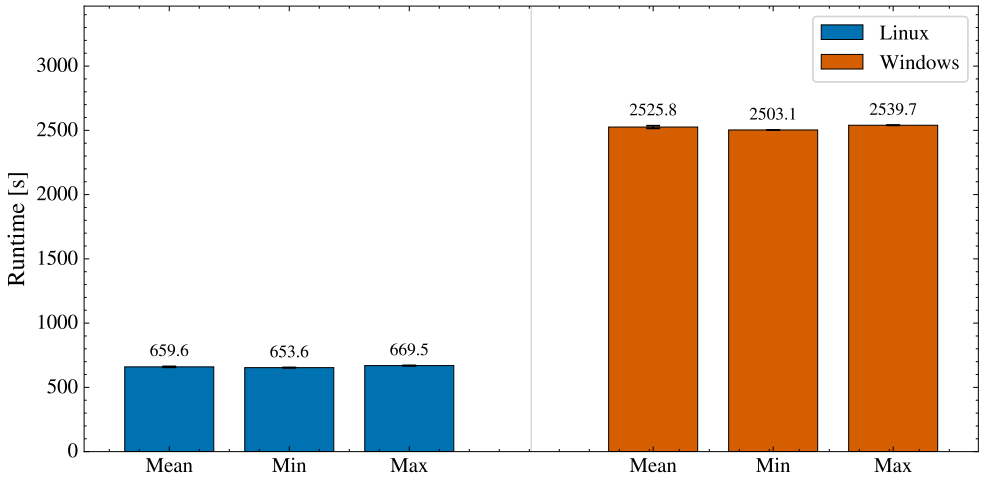

Abbildung A.73: Laufzeiten des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

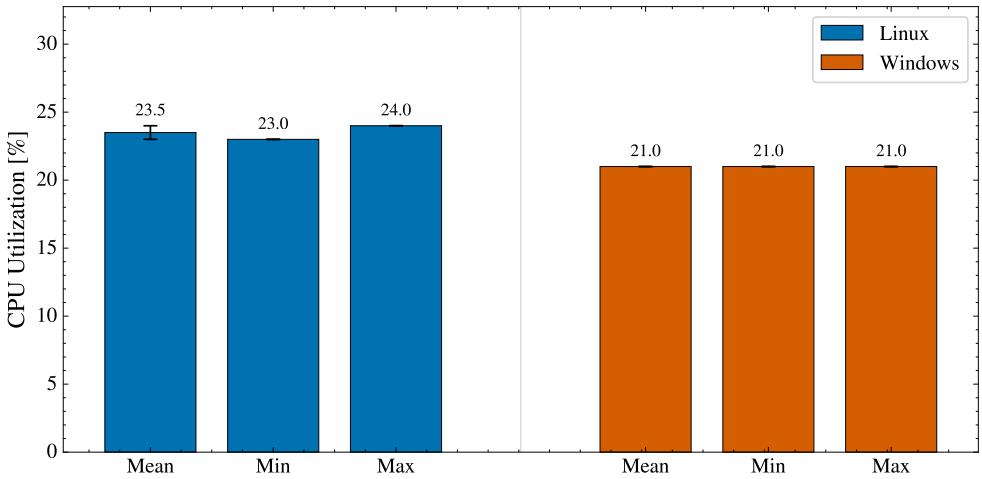

Abbildung A.74: CPU-Auslastung des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

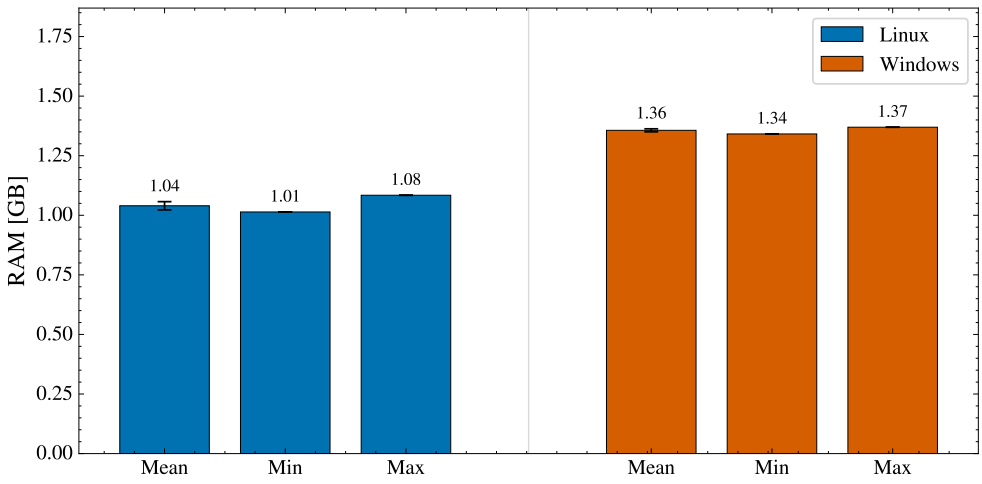

Abbildung A.75: RAM-Auslastung des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows bei einem Thread, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

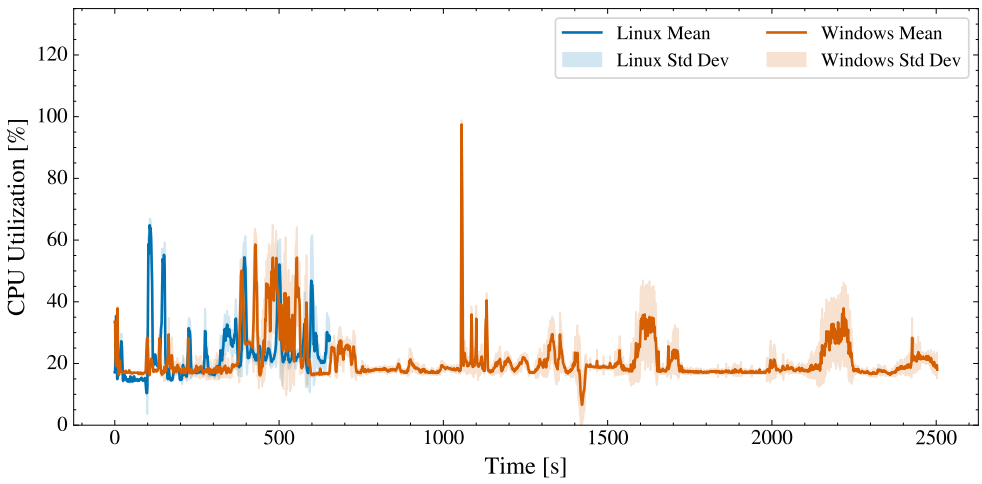

Abbildung A.76: Zeitlicher Verlauf der CPU-Auslastung des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows bei einem Thread

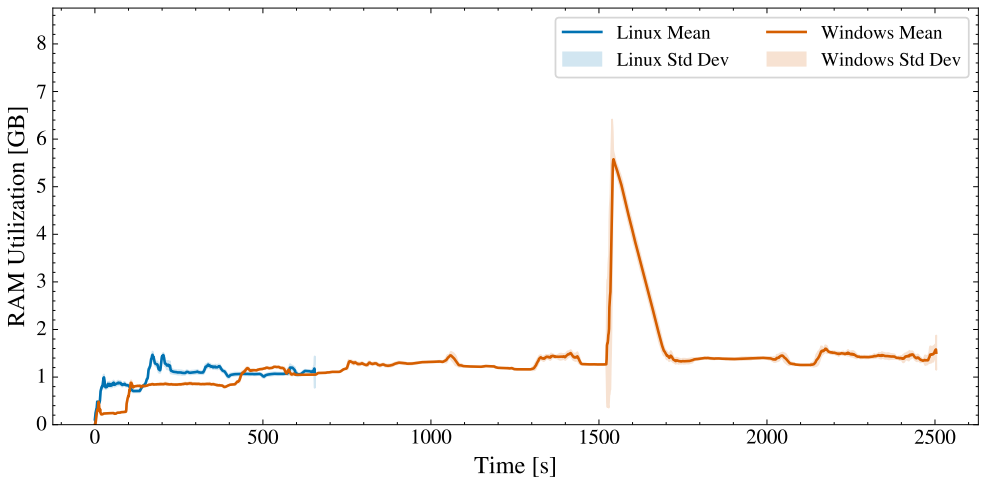

Abbildung A.77: Zeitlicher Verlauf der RAM-Auslastung des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows bei einem Thread

A.5.2 *Vier Threads*

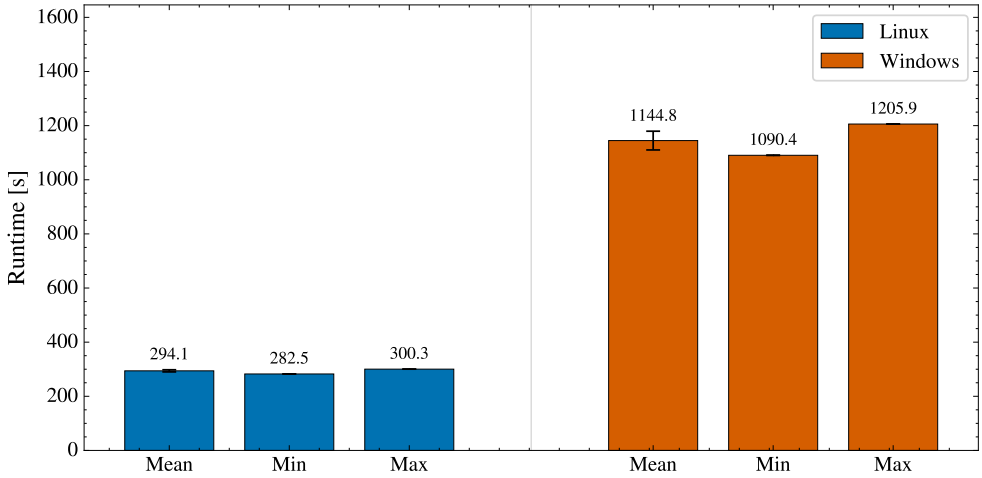

Abbildung A.78: Laufzeiten des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

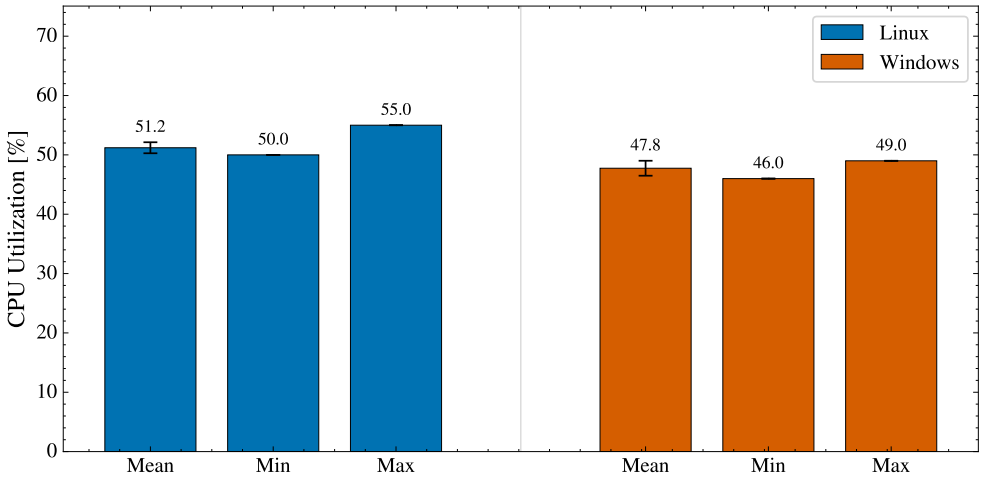

Abbildung A.79: CPU-Auslastung des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

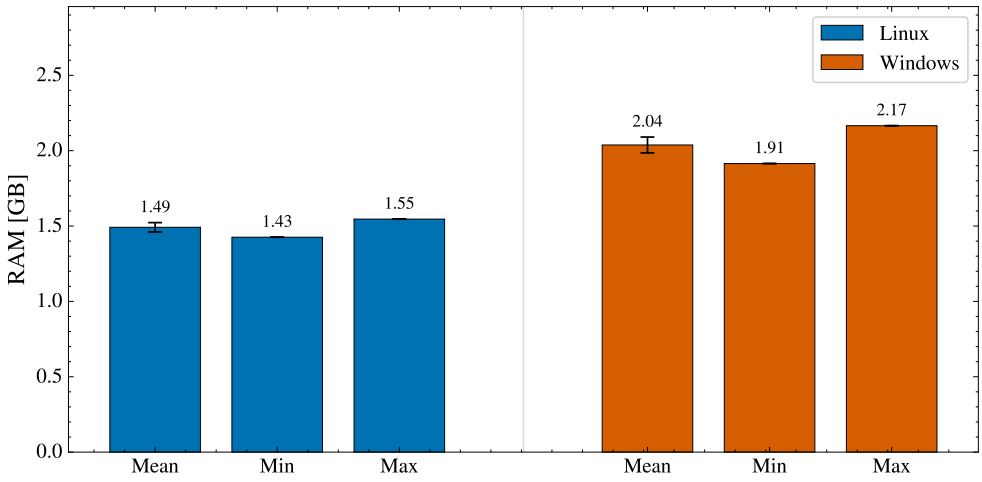

Abbildung A.80: RAM-Auslastung des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows bei vier Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

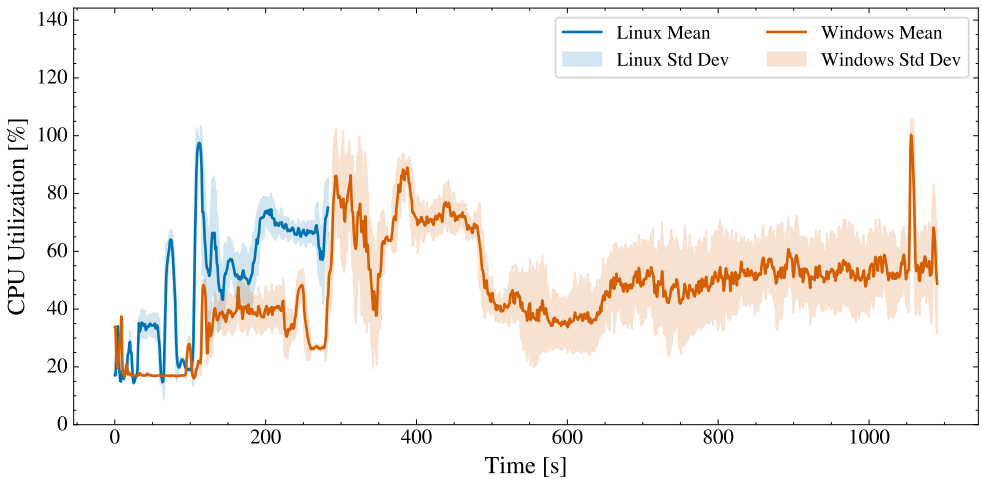

Abbildung A.81: Zeitlicher Verlauf der CPU-Auslastung des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows bei vier Threads

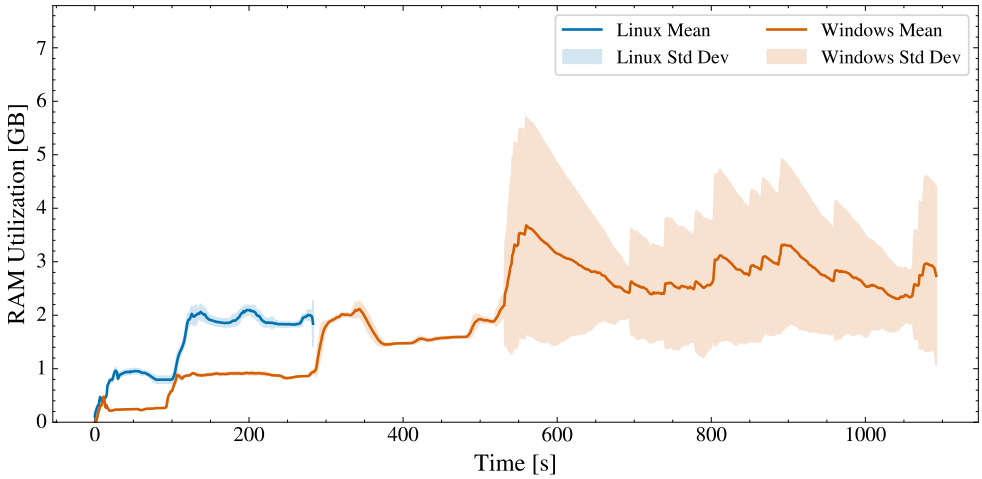

Abbildung A.82: Zeitlicher Verlauf der RAM-Auslastung des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows bei vier Threads

A.5.3 *Zehn Threads*

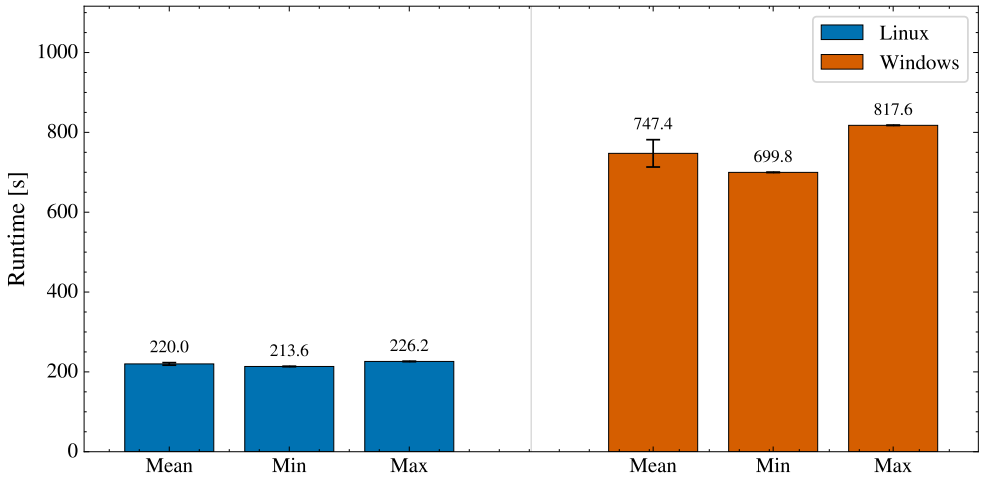

Abbildung A.83: Laufzeiten des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

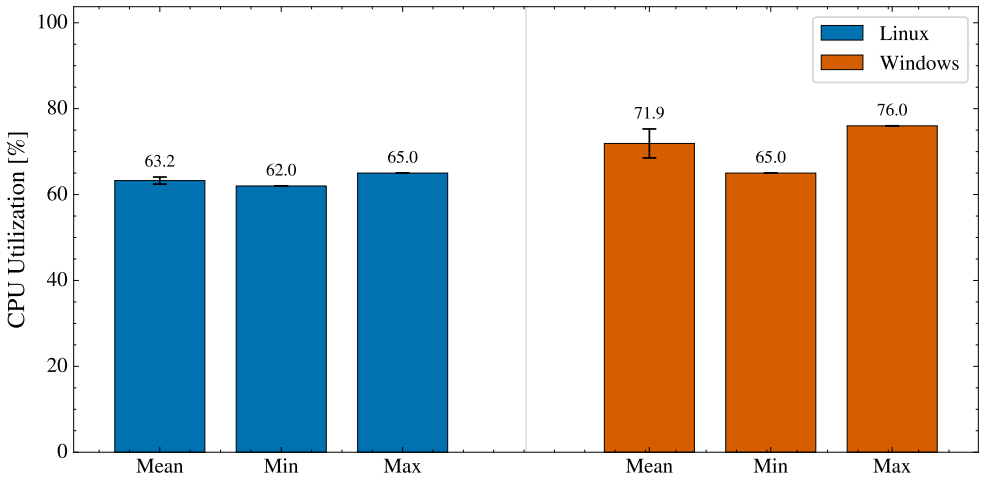

Abbildung A.84: CPU-Auslastung des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

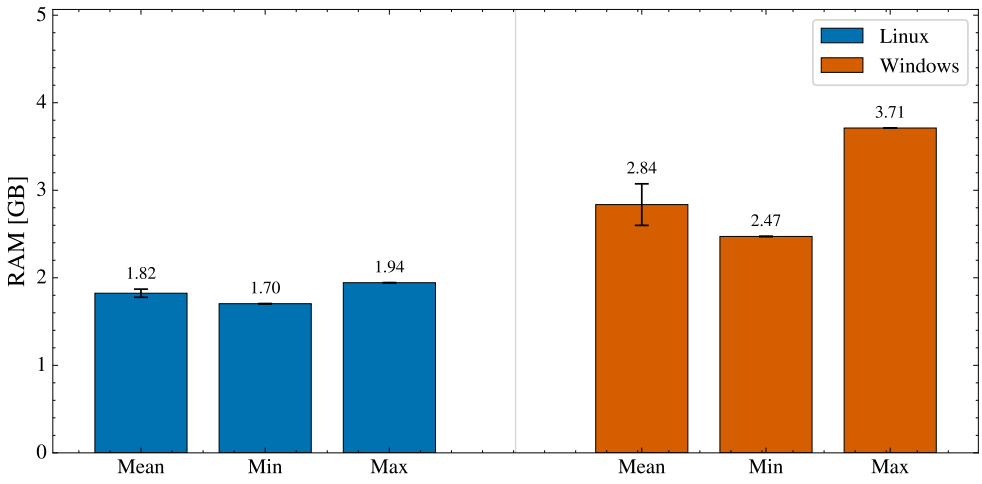

Abbildung A.85: RAM-Auslastung des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows bei zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

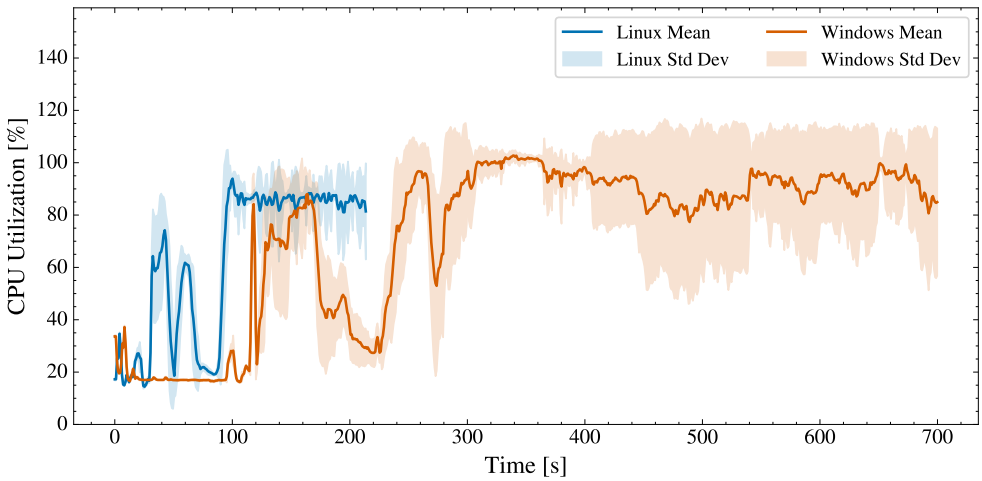

Abbildung A.86: Zeitlicher Verlauf der CPU-Auslastung des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows bei zehn Threads

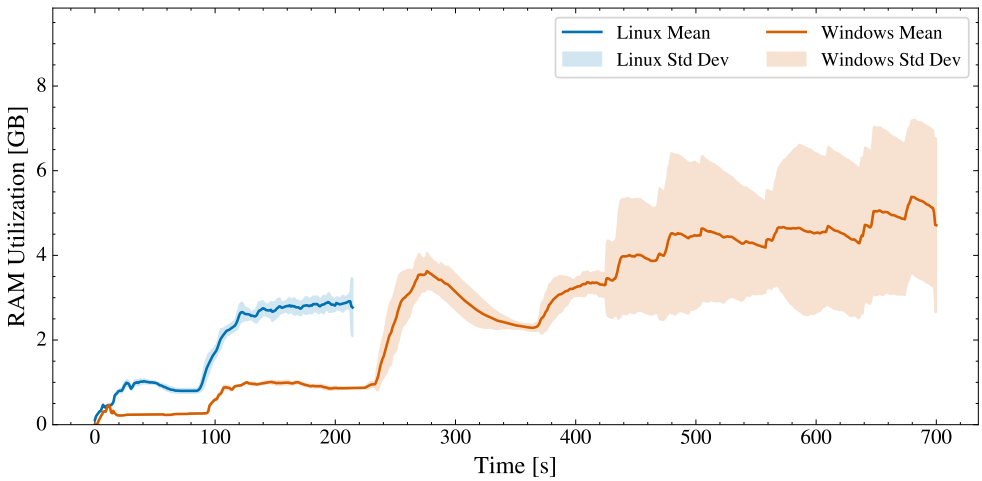

Abbildung A.87: Zeitlicher Verlauf der RAM-Auslastung des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows bei zehn Threads

A.5.4 *Vergleich*

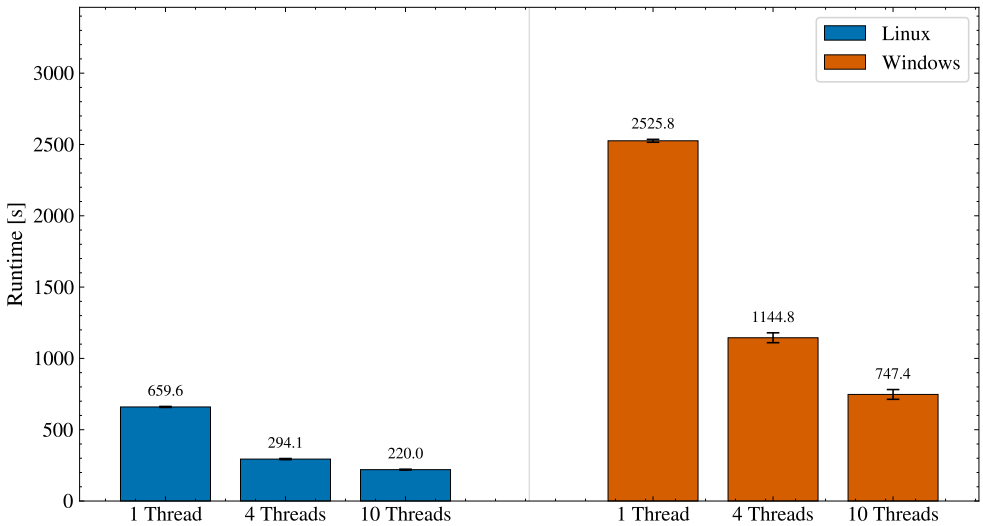

Abbildung A.88: Vergleich der Laufzeiten des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows über ein, vier und zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

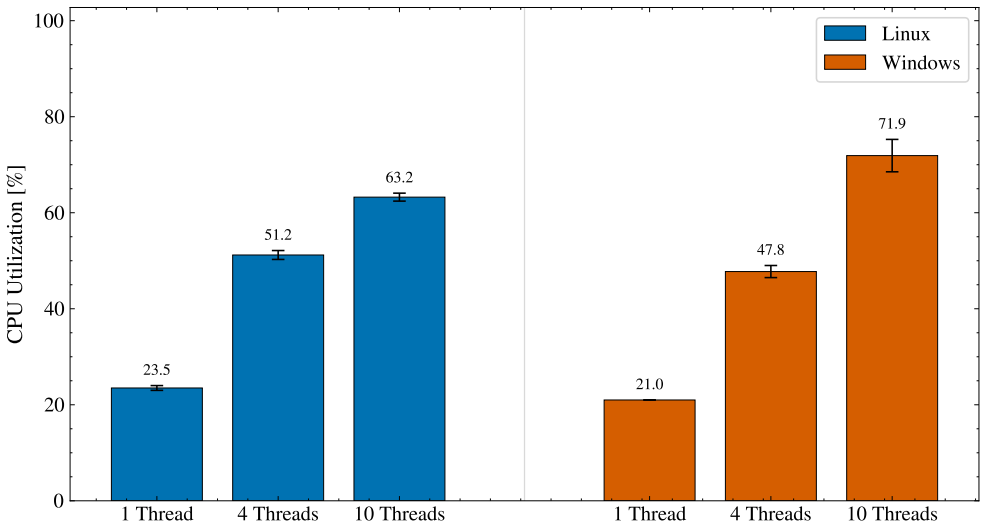

Abbildung A.89: Vergleich der CPU-Auslastung des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows über ein, vier und zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

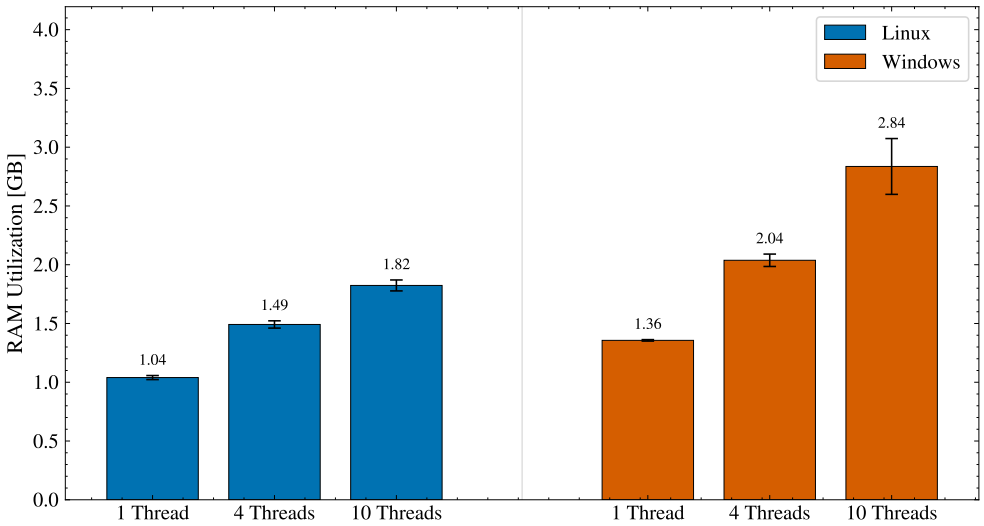

Abbildung A.90: Vergleich der RAM-Auslastung des Work-Stealing-Schedulers mit SSTF-Priorisierung auf Linux und Windows über ein, vier und zehn Threads, dargestellt als Balkendiagramm mit Mittelwerten und Standardabweichungen aus 20 Messläufen

LITERATUR

[1] R. Buchta, G. Gkoktsis, F. Heine und C. Kleiner, "Advanced Persistent Threat Attack Detection Systems: A Review of Approaches, Challenges, and Trends", *Digital Threats: Research and Practice*, Jg. 5, Nr. 4, S. 1–37, 2024. DOI: 10.25968/opus-3597. Adresse: https://doi.org/10.25968/opus-3597.

[2] P. B. G. Abraham Silberschatz und G. Gagne, "Operating System Concepts", 2018.

[3] A. S. Tanenbaum, *Modern operating systems*, Fourth edition. Boston: Pearson, 2015, ISBN: 9780133591620.

[4] B. Gregg, *Systems performance: Enterprise and the cloud* (Addison-wesley professional computing series), Second. Boston: Addison-Wesley, 2020, ISBN: 9780136820154.

[5] John L. Hennessy und David A. Patterson, "Computer Architecture: A Quantitative Approach 6th Edition", 2019.

[6] *3.6. Modules — THOR Manual documentation*, 1/19/2026. Adresse: https://thor-manual.nextron-systems.com/en/v11/scanning/modules.html.

[7] *3.7. Features — THOR Manual documentation*, 1/19/2026. Adresse: https://thor-manual.nextron-systems.com/en/v11/scanning/features.html.

[8] R. W. Conway, W. L. Maxwell und L. W. Miller, *Theory of Scheduling*. Addison-Wesley, 1967.

[9] R. D. Blumofe und C. E. Leiserson, "Scheduling multithreaded computations by work stealing", *Journal of the ACM*, Jg. 46, Nr. 5, S. 720–748, 1999, ISSN: 0004-5411. DOI: 10.1145/324133.324234.

[10] P. B. Hansen, *Operating System Principles*. 1973.

[11] *Virtio balloon memory statistics — QEMU documentation*, 3/1/2026. Adresse: https://www.qemu.org/docs/master/interop/virtio-balloon-stats.html.

[12] *Performance Tweaks - Proxmox VE*, 12/18/2025. Adresse: https://pve.proxmox.com/wiki/Performance_Tweaks.

[13] *Hyper-Threading vs No Hyper-Threading; Fixed vs Variable Memory*, 3/2/2026. Adresse: https://forum.proxmox.com/threads/solved-hyper-threading-vs-no-hyper-threading-fixed-vs-variable-memory.20265/.

[14] *8.4. Guest Virtual Machine Memory Allocation | Virtualization Administration Guide | Red Hat Enterprise Linux | 6 | Red Hat Documentation*, 3/2/2026. Adresse: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/6/html/virtualization_administration_guide/sect-mem-dump-off.

[15] *Hugepages - Debian Wiki*, 3/2/2026. Adresse: https://wiki.debian.org/Hugepages.

[16] Intel Corporation, *KVM/Qemu Virtualization Tuning Guide on 3rd Generation Intel® Xeon® Scalable Processors Based Platforms*, Revision 0.1, Document Number: 686407, Santa Clara, CA: Intel Corporation, 2021. Adresse: https://cdrdv2-public.intel.com/686407/kvm-tuning-guide-icx.pdf.

[17] "The Black Magic of Systematically Reducing Linux OS Jitter - High Scalability -", *High Scalability*, 4/8/2015. Adresse: https://highscalability.com/the-black-magic-of-systematically-reducing-linux-os-jitter/.

[18] *Disk cache modes | SLES 15 SP6*, 2024. Adresse: https://documentation.suse.com/de-de/sles/15-SP6/html/SLES-all/cha-cachemodes.html.

[19] *Invocation — QEMU 7.2.19 documentation*, 7/24/2025. Adresse: https://qemu.readthedocs.io/en/v7.2.19/system/invocation.html.

[20] *Optimizing KVM Performance with CPU Pinning and Memory Ballooning*, 2025. Adresse: https://dohost.us/index.php/2025/09/09/optimizing-kvm-performance-with-cpu-pinning-and-memory-ballooning/.

[21] *CPU Performance Scaling — The Linux Kernel documentation*, 3/1/2026. Adresse: https://docs.kernel.org/admin-guide/pm/cpufreq.html.

[22] *IBM Cloud Docs*, 3/4/2026. Adresse: https://cloud.ibm.com/docs/vpc?topic=vpc-disabling-hyper-threading.

[23] *Finer-grained kernel address-space layout randomization*, 3/17/2020. Adresse: https://lwn.net/Articles/812438/.